PROJECT ENGINEERING DOCUMENT V3.0

Benno Weideman 602904

Mohau Drew Modiselle 602295

Tammy Fourie 601264

Submission due date: 16 October 2026

Table of Contents

Submission Requirements 6
Submission Information Block 6
Version History 6
1. Problem & Business Need 7
1.1 Problem Statement 7
1.2 Business Value & Stakeholder Linkage: 7
2. Stakeholder Analysis 8
2.1 Purpose and business context 8
2.2 Stakeholder Register 8
2.3 Influence-Interest Positioning and Engagement 8
2.4 Competing Expectations and Resolutions 9
Submission simplicity versus reporting depth 9
3. Scope Baseline 10
3.1 Scope objective 10
3.2 In-scope capabilities 10
3.3 Out-of-scope items 10
3.4 Deferred / future candidates 11
3.5 Deliberate exclusion defense: automated informal-channel intake 11
3.6 Scope-to-Constraint Interactions 11
3.7 Scope Acceptance and Governance 12
4. Requirements & Acceptance Criteria 13
4.1 Functional Requirements 13
4.2 Non-functional Requirements 15
5. Constraints 16
5.1 Purpose 16
5.2 Application of Constraints 16
5.3 Scope constraints 16
5.4 Schedule constraints 16
5.5 Cost and resource constraints 17
5.6 Quality constraints 17
5.7 Security constraints 18
5.8 Technology and governance constraints 19
5.9 Proposed measurable quality boundaries 19
5.10 Constraint interactions and ripple effects 20

6. Architecturally Significant Requirements 21
7. Architecture Decision & Diagrams 22
7.1 Architectural drivers 22
7.2 Architecture alternatives considered 23
7.3 Selected architecture 23
Dependency rules 25
7.4 Physical view and environments 26
7.5 Request lifecycle state model (D10) 26
7.6 Key runtime interaction: changing a request's status 28
7.7 Availability position (ASR-004) 29
8. Data & Persistence Baseline 30
8.1 Entity Overview 30
8.1.1 Changes from the M2 baseline 31
8.2 Entity Relationship Diagram 32
8.3 Justification of Persistence Model 33
8.4 Conclusion 34
8.5 Bottlenecks, SPOF, Scalability, Availability, and Backup/Recovery 34
8.5.1 Single Point of Failure (SPOF) 34
8.5.2 Scalability 34
8.5.3 Availability 34
8.5.4 Backup and Recovery 35
9. Technology Stack Decision 36
9.1 Selection Criteria and ASR Alignment 36
9.2 Comparative Technology Matrix 36
9.3 Baseline Version and Compatibility Matrix 37
10. Information Architecture 37
Wireframes and Rationale 37
10.1 Screen inventory 38
10.2 Wireframes 38
10.3 Usability rationale 45
10.4 Accessibility rationale 46
10.5 Proposed measurable UI acceptance 47
10.6 Deliberately deferred UI decisions 47
11. Initial Design Decisions 48
11.1 DP-01: requester notifications and channel volatility (D11) 48
Problem in CivicConnect terms 48

A2 evidence and alternatives 48
Final decision 48
11.2 DP-02: status changes and their side effects (D12) 50
Problem in CivicConnect terms 50
A2 evidence and alternatives 51
Final decision 51
Rules the team commits to 52
11.3 Additional decision: email verification at registration (D13) 53
Why verify the email 53
Options considered 53
Final decision 54
Trade-offs the team accepts 56
11.4 Research-to-decision summary 56
12. API / Integration Decisions 58
12.1 Interaction Boundary and Integration Mechanism Selection 58
12.2 Initial REST API Endpoint Contract Baseline 58
12.3 Layered Validation, Integrity and Caching Exclusion 59
12.4 Error Semantics, Security Boundary and Versioning 60
13. Deployment Compatibility 61
13.1 Target Deployment Direction and Environment Strategy 61
13.2 State, Persistence and Networking Implications 61
13.3 Configuration Parity and Secrets Control 61
Deployment Risks and Deliberately Deferred Decisions 61
14. Requirements Traceability Matrix 63
15. Risk Register 70
16. Forward Engineering Considerations 73
17. Engineering Decision Log 74
18. Change Impact Analysis 78
19. GitHub Controls 81
19.1 Governance Rules 81
19.2 Branch Strategy (M1) 81
19.3 Pull Request Workflow 82
20. Team Working Agreement 83
20.1 Team Members & Roles 83
20.2 Primary Goals & Standards 83
20.3 Communication & Meetings Protocol 83

20.4 Git & Workflow Standards 83
20.5 Deadlines & Conflict Resolution 83
20.6 Signatures & Agreement 83
21. AI Usage Register 84
Features 84
22. Milestone 3 Overview 92
22.1 Purpose 92
22.2 Release candidate 92
22.3 Implemented scope 92
22.4 Current status 93
22.5 Where to find things 93
23. Milestone 2 Baseline Conformance 94
23.1 Approach 94
23.2 Conformance table 94
23.3 Significant deviations 95
23.4 Summary 97
24. Controlled Changes 98
24.1 Change summary 98
24.2 Decision record: D15 98
24.3 Decision record: D17 98
25.1 Repository and change control 99
25.2 Configuration and secrets 99
25.3 Staging environment 99
26. CI & Automated Verification 101
26.1 Pipeline summary 101
26.2 Quality gate rules 101
26.3 CI run evidence 101
26.4 Static analysis and security checks 101
27. Quality Strategy Applied 102
28. Black-box/Integration/System Validation 103
28.1 Technique selection 103
28.2 Test evidence records 103
29. Performance/Reliability 108
29.1 Test design 108
29.2 Results (PT-01) 108
29.3 Interpretation and limits 108

30. Defects & Quality Interpretation 109
30.1 Defect register 109
30.2 Static and security findings 109
30.3 Quality metrics interpretation 109
31. Release Readiness 110
31.1 Release Evidence Summary 110
31.2 Classification 110
31.3 Blockers and accepted limitations 111
31.4 Residual risk statement 111
32. Technical Debt Register 112
33. Baseline Sign-Off 114
References 115
Peer Review 117

Submission Requirements

Submission Information Block

Item	Detail
Team name	Group F
Team members	Benno Weideman 602904 Mohau Drew Modiselle 602295 Tammy Fourie 601264
GitHub repository	https://github.com/Stelselopsiener/CivicConnect-Application
M3 release candidate	f313e2818d4ecb571f1b74244c3db7b38cd8c901
PED version	v3.0
RTM location	14. Requirements Traceability Matrix, p.56 - 60
CI/test evidence	https://github.com/Stelselopsiener/CivicConnect-Application/actions
Staging/test environment	https://civic-connect-staging-application-4qxrp0sax-group-f3.vercel.app/sign-in
Date submitted	16 October 2026

Version History

Version	Milestone	Summary of change
v1.0	M1	Requirements, initial risk register, initial decision log, stakeholders ,and initial AI register.
v2.0	M2	Architecture, technology and initial design baseline.
v3.0	M3	Implementation, integration, verification evidence, release readiness.

1. Problem & Business Need

1.1 Problem Statement: Local municipalities face systemic inefficiencies due to fragmented communication channels and manual administrative processes for handling infrastructure faults. This disconnected workflow creates severe service delivery backlogs, prolonged resolution times, and widespread public frustration, compounding the well-documented adoption challenges and administrative barriers present in South African municipal e-government initiatives (Mashau, N.L. & Kroeze, J.H., 2023).

1.2 Business Value & Stakeholder Linkage: CivicConnect resolves these inefficiencies by functioning as a centralized e-governance platform. It standardizes issue ingestion and accountability routing, effectively integrating digitization directly into basic service delivery frameworks to eliminate operational bottlenecks (Hofisi & Chigova, 2023).

- **For Requesters:** The platform rebuilds public confidence through transparent, real-time status updates and meaningful feedback (FR-004, FR-006). Local authority trust, transparency, and platform security are the primary determinants of successful citizen consumption of e-government services (Netshirando & Ngoepe, 2024).
- **For Staff:** It replaces scattered, paper-based communication with a unified digital dashboard to filter, assign, and resolve tasks (FR-007, FR-011). This digital transformation removes administrative friction, allowing staff to manage operational workflows more efficiently (Hofisi & Chigova, 2023).
- **For Management:** It provides essential oversight through measurable summary statistics and staff audit histories (FR-018, FR-021). Centralizing this data supports data-driven governance, allowing municipalities to successfully balance citizen aspirations with structural and financial constraints (Mashau, 2023).

2. Stakeholder Analysis

2.1 Purpose and business context

The purpose of the stakeholder analysis is to identify the people who affect or are affected by CivicConnect, determine their needs, influence and interest, and examine meaningful conflicts between their expectations. These findings provide traceable inputs to the project's scope, requirements, constraints, risks and engineering decisions.

2.2 Stakeholder Register

ID	Stakeholder	Category	Primary needs / expectations	Influence	Interest	Engagement
STK-001	Requester	Primary user	Submit quickly, understand categories, see own status/history, receive meaningful feedback, trust request is not lost.	L	H	Keep informed
STK-002	Frontline staff	Primary user	Prioritized worklist, sufficient request detail, clear ownership, fast action logging, controlled resolution/closure.	M	H	Keep informed / involve
STK-003	Service desk / triage coordinator	Primary user	Reliable categorization, identify duplicates, reassign misrouted requests, maintain ownership and audit trail.	M	H	Keep informed / involve
STK-004	Department supervisor / team lead	Secondary user	Visibility of team workload and overdue work, rebalance assignments, review performance and exceptions.	M	H	Keep informed / consult
STK-005	Senior management / executive sponsor	Oversight / sponsor	Organization-level reporting, accountability, business value, sustainable operating cost, controlled scope.	H	H	Manage closely
STK-006	System administrator / security custodian	Technical / operational	Controlled identities and roles, least privilege, secure configuration, auditability, manageable maintenance burden, single-time notification after user registration.	H	M	Keep satisfied / consult
STK-007	Client or product owner	Governance / approval	Clarify priorities, approve business scope, accept or reject baseline changes, validate delivered stakeholder value.	H	H	Manage closely
STK-008	Lecturer / assessor	Academic governance	Evidence of controlled engineering, compliance with milestone and integrity requirements and defensible individual understanding.	H	M	Keep satisfied
STK-009	Project team	Delivery stakeholder	Achievable, stable and testable requirements, clear decisions, controlled changes and feasible delivery within skills and schedule.	H	H	Manage closely

2.3 Influence-Interest Positioning and Engagement

Position	Stakeholders	Engagement approach
Manage closely	STK-005, STK-007, STK-009	Review scope, priorities, constraints and material changes which obtains documented decisions and baseline approval.
Keep satisfied	STK-006, STK-008	Provide security, maintainability, governance and evidence reviews at agreed checkpoints which resolve veto or compliance risks early.
Keep informed / actively involve	STK-001, STK-002, STK-003, STK-004	Use interviews, workflow walkthroughs and prototype reviews which validates usability, information needs and acceptance criteria.
Monitor	None currently	Reassess if new peripheral or low-impact stakeholder classes emerge.

2.4 Competing Expectations and Resolutions

Submission simplicity versus reporting depth

Conflict: Requesters need a quick, comprehensible submission process, while management needs structured category, location and urgency information for accountability and reporting.

Baseline resolution: Use a small controlled set of mandatory submission fields. Permit staff to enrich or correct operational details during triage. Formal requirements must define the mandatory fields and validation rules.

Controlled artefacts affected: FR for request submission, mandatory fields and controlled categorization; usability and data-quality NFR; and RTM links to STK-001, STK-003 and STK-005.

Staff autonomy versus accountability

Conflict: Frontline staff need to act quickly, while management and governance stakeholders need traceable ownership and status changes.

Baseline resolution: Use controlled states and authorized transitions with an audit history. Do not assume one strictly linear path: requirements must define rejection, cancellation, reopening and invalid-transition handling.

Controlled artefacts affected: FR for request ownership and status updates; security and auditability NFR; later workflow/state modelling; and a risk-register entry if ownership or status-transition rules remain unclear.

Oversight visibility versus privacy and least privilege

Conflict: Supervisors and management require useful oversight, while requesting information may be sensitive and must not be exposed unnecessarily.

Baseline resolution: Apply role-based access and least privilege. Requesters see only their own requests; staff see requests relevant to their authorized functions; oversight access is limited to justified operational detail and aggregated information where sufficient.

Controlled artefacts affected: Security and privacy NFR; role and permission requirements; privacy and unauthorized access risks; and acceptance criteria or tests confirming that users cannot access information outside their authorized responsibilities.

Broad intake convenience versus delivery feasibility

Conflict: Stakeholders may prefer WhatsApp, telephone and email intake, while the team has limited schedule, skills and integration capacity.

Baseline resolution: Commit to one responsive web intake channel. Exclude automated external-channel intake from the baseline and reconsider it only through formal impact analysis.

Controlled artefacts affected: The out-of-scope/deferred-scope entry for external intake channels; schedule and resource constraints; integration and privacy risks; and the engineering decision record supporting the decision to defer automated external-channel intake.

3. Scope Baseline

3.1 Scope objective

The CivicConnect baseline commits the team to a controlled, responsive web platform that provides a reliable and traceable way to submit, manage, monitor and report on service requests without creating an unsustainable technical, operational or financial burden. The boundary prioritizes the minimum business capabilities in the Master Project Brief and a smaller, complete, secure and testable solution over uncontrolled feature growth (Belgium Campus, 2026a).

3.2 In-scope capabilities

Scope ID	Capability	Committed boundary
S-IN-001	Responsive web submission	A requester can submit a service request with validated essential information and a controlled category: facility fault, equipment damage, security concern, IT support, maintenance, lost property or other.
S-IN-002	Requester status and history	An authenticated requester can view the current status and lifecycle history of an individual request and a list of their previously submitted requests.
S-IN-003	Meaningful requester feedback	The platform provides in app feedback when a request is accepted, rejected, materially updated, resolved or closed. External email/SMS delivery is not committed, however, a requester will receive notification via email for 6-digit code to confirm user registration once.
S-IN-004	Staff worklist	Authorized staff can view full details of relevant requests and search, filter and sort them using agreed useful criteria.
S-IN-005	Triage and ownership	Authorized staff can correct categorization where permitted, assign a request or accept responsibility and view current ownership and ownership history.
S-IN-006	Controlled request lifecycle	Authorized staff can update status through valid transitions and record actions, comments and resolution information. Actor and timestamp information is retained for auditability.
S-IN-007	Resolution and closure	Authorized staff can resolve or close requests and provide resolution information. The detailed state model must address rejection, cancellation and reopening before requirements sign-off.
S-IN-008	Role-based access	Requester, staff, supervisor/management and administrator capabilities are separated using role-based access and least privilege.
S-IN-009	Oversight views	Authorized oversight users can identify open, overdue, resolved and closed requests and filter information by category and status.
S-IN-010	Due-date/overdue basis	The system stores or derives a target resolution date using an approved rule so that overdue work can be identified. Automatic escalation is excluded.

3.3 Out-of-scope items

Scope ID	Excluded item	Boundary statement
S-OUT-001	Automated intake from WhatsApp, telephone or email	The baseline provides a single controlled web intake channel. Manual staff capture, if permitted by the client, must be specified separately and is not an automated integration.
S-OUT-002	Native mobile applications	A responsive web interface is sufficient for this baseline.
S-OUT-003	Multi-language localization	The baseline supports English only.
S-OUT-004	Payment or billing	No financial transaction or invoicing workflow is included.
S-OUT-005	AI-driven triage or routing	Categorization and assignment are controlled user actions where no predictive or generative routing is committed.
S-OUT-006	Advanced analytics / BI	Only the oversight views in S-IN-009 are committed where forecasting, custom BI and data warehouses are excluded.

Scope ID	Excluded item	Boundary statement
S-OUT-007	External asset-management or HR integration	No external enterprise-system synchronization is included.
S-OUT-008	Multi-organization / multi-tenant operation	The baseline serves one organization and one controlled deployment context.

3.4 Deferred / future candidates

Scope ID	Candidate	Conditions before consideration
S-DEF-001	SLA-based automatic escalation	Requires approved service targets, exception paths and notification rules.
S-DEF-002	Email/SMS notifications	Requires provider, cost, consent, privacy, failure-handling and delivery decisions.
S-DEF-003	Exportable analytics/reports	Requires approved formats, access rules, data minimization and reporting acceptance criteria.
S-DEF-004	Attachment/photo upload	Requires storage limits, malware/content handling, access controls, retention and privacy analysis.
S-DEF-005	Multi-channel automated intake	Requires external APIs, identity/correlation rules, security, availability and cost analysis.
S-DEF-006	Registration Email verification	Six-digit email ownership verification at registration is deferred to the next milestone. The implementation has been partially completed and retained in the codebase, but the M3 registration flow creates an immediately usable requester account. Reactivation requires regression testing of code expiry, resend behaviour, attempt limits, SMTP configuration and login gating.

3.5 Deliberate exclusion defense: automated informal-channel intake

Excluded item: Automated intake from WhatsApp, telephone and email channels (S-OUT-001).

- **Why it is attractive:** These channels are named sources of fragmentation. Integrating them could reduce user behaviour change and appear to consolidate the current process directly.
- **Schedule and resource rationale:** A three-person academic team working across four milestones cannot responsibly commit to multiple third-party integrations while also specifying, constructing, securing, testing and deploying the core platform. The integration would materially increase dependency and schedule risk.
- **Technology and security rationale:** Messaging and email integrations introduce external authentication, rate limits, availability dependencies, identity matching, duplicate detection, consent and sensitive-data handling. Telephone intake additionally requires a defined transcription or staff-capture process.
- **Value rationale:** The core business value is a single controlled request record with visible ownership, status, history and oversight. A controlled web channel can deliver that value without external integrations. Multi-channel automation is therefore an enhancement, not a precondition for the baseline solution.
- **Condition for reconsideration:** After the core platform is stable, the client may request reconsideration. The team must analyse requirements, architecture, UI/API/data, privacy, quality/testing, scope, schedule/resources, cost, deployment, risk and technical debt before approval (Belgium Campus, 2026a).

3.6 Scope-to-Constraint Interactions

Constraint area	Interaction with scope	Required control/evidence
Schedule	External integrations, native apps and advanced analytics would compete with core workflow completion.	Keep excluded/deferred unless approved impact analysis shows capacity and no unacceptable milestone risk.
Cost/resources	Providers, hosting, storage and maintenance can create recurring costs beyond the academic team's capacity.	Record cost assumptions; prefer sustainable platform choices; approve new recurring costs explicitly.

Constraint area	Interaction with scope	Required control/evidence
Quality	Every scope item creates specification, usability, test, deployment and maintenance obligations.	Give committed capabilities measurable acceptance criteria and trace them through the RTM.
Security/privacy	Request data may be sensitive; broader access, uploads and integrations increase exposure.	Apply RBAC, least privilege, validation, secrets controls and security testing; record residual risks.
Operational burden	Complex configuration or multiple intake paths may undermine the stated need for sustainability.	Review administrative workflow and maintainability with STK-006 before approval.

3.7 Scope Acceptance and Governance

This scope becomes the approved CivicConnect baseline after team review and PED v1.0 baseline sign-off. After approval, additions, removals or material changes must follow the project's change-management process. Approved changes must be reflected in the scope baseline, requirements, Requirements Traceability Matrix and other affected controlled artefacts (Belgium Campus, 2026a).

4. Requirements & Acceptance Criteria

4.1 Functional Requirements

ID	Source	What should the system do?	Priority	Acceptance criteria
FR-001	STK-001, Project Master Brief, Min Business Capabilities, p.7	The system should allow the requester to enter and submit service request information.	Medium	Meaningful messages guide user input. Able to enter title, description, and street address. Validation check ensures no empty fields. Information must be submitted to the database.
FR-002	STK-003, Project Master Brief, Min Business Capabilities, p.7	The system should allow the requester to select a predefined category for a service request.	High	Predefined categories can be selected via dropdown. Validation check ensures that a category was selected.
FR-003	STK-001, Project Master Brief, Min Business Capabilities, p.7	The system should allow the requester to see the status of a service request.	Medium	Message “pending” displays if request not assigned. Message “accepted” displays if request was assigned. Message “in progress” displays if request is underway. Message “completed” displays if request is complete. Message “rejected” displays if request was rejected. Message “updated” displays if request was updated. Message “cancelled” displays if request was cancelled.
FR-004	STK-001, Project Master Brief, Min Business Capabilities, p.7	The system should allow the requester to view the history of her/his service requests.	Medium	All submitted service requests are visible in history.
FR-005	STK-001, Project Master Brief, Min Business Capabilities, p.7	The system should allow the requester to view meaningful feedback for a request when it is accepted, rejected, updated, or completed.	Medium	Meaningful feedback is provided for a service request is provided whenever it is accepted, rejected, updated, or completed.
FR-006	STK-002, Project Master Brief, Min Business Capabilities, p.7	The system should allow the staff to view a list of service requests according to their category.	High	Visible service requests appear on the list subject to staff category.
FR-007	STK-002, Project Master Brief, Min Business Capabilities, p.7	The system should allow the staff to search, filter, and sort service request using criteria information.	High	Service requests can be searched, filtered, and sorted according to the provided criteria.
FR-008	STK-002, Project Master Brief, Min Business Capabilities, p.7	The system should allow the staff to view the full details of a service request.	High	Full details of request displays whenever pressed.
FR-009	STK-002, Project Master Brief, Min Business Capabilities, p.7	The system should allow the staff to assign service request to another staff or accept it themselves.	High	Display authorized staff for service request. Staff can assign service request to authorized staff. Staff can take ownership of service request.
FR-010	STK-002, Project Master Brief, Min Business Capabilities, p.7	The system should allow the staff to select a request status option.	High	Staff can update request statuses using the following vocabulary: pending, in progress, accepted, completed, and rejected.
FR-011	STK-002, Project Master Brief, Min Business Capabilities, p.7	The system should allow the staff to enter and submit service request feedback.	High	Able to submit actions, comments, date, and resolution information. Validation check ensures no empty fields. Submitted feedback must be displayed on requester’s side.
FR-012	STK-002, Project Master Brief, Min Business Capabilities, p.7	The system should allow the staff to resolve or close a service request.	High	Staff can resolve a service request and status is changed to “completed”. Staff can close a service request and status is changed to “rejected”.
FR-013	STK-005, Project Master Brief, Min Business Capabilities, p.7	The system should allow management to view overall statistics of service requests.	High	Visible display of total summary statistics of service requests.
FR-014	STK-005, Project Master Brief, Min Business Capabilities, p.7	The system should allow management to view service requests with work status.	High	Displays a visible list of all service requests. Message “open” displays if request is not taken yet. Message “closed” displays if request was rejected. Message “resolved” displays if request is completed. Message “overdue” displays if request is past its due date.

FR-015	STK-005, Project Master Brief, Min Business Capabilities, p.7	The system should allow management to filter and sort request information by status, category, start date, and end date.	High	Display filtered and sorted service request when criteria information such as work status, category, start/end dates are provided.
FR-016	STK-003, Project Master Brief, Min Business Capabilities, p.7	The system should allow management to view audit of staff actions.	High	Display visible audit of staff with the actions for a service request. Visible audit of staff updating status requests.
FR-017	STK-006	The system shall allow a requester to create an account by entering a valid name, email address and password.	High	A requester can enter name, email and password. Passwords between 8 and 64 characters are accepted, subject to the implemented UTF-8 byte limit. Valid registration creates a requester account with the password stored as a hash. Duplicate email addresses are rejected. After successful registration, the requester can sign in using the registered email address and password. Email ownership verification by six-digit code was implemented during M3 but formally deferred to the next milestone due to release-scope and verification effort.
FR-018	STK-003, Project Master Brief, Project Scenario: CivicConnect, p.6	The system should allow staff to correct category for a request.	High	Staff can select the correct category for a service request. Staff can see the updated category.

4.2 Non-functional Requirements

ID	Source	How should the system behave?	Priority	Acceptance criteria
NFR-001	STK-008	The system should not delay processing by more than 10 seconds.	Medium	When a button is pressed for any system related functionality, the loading should not take more than 10 seconds.
NFR-002	STK-008	The system should support Email to allow notifications to be sent regarding status updates.	Medium	Status update notification is sent to Email.
NFR-003	STK-001, Project Master Brief, Project Scenario: CivicConnect, p.6	The system should be intuitive for requesters to use and should receive meaningful feedback.	Medium	At least 80% of beginner requesters should be able to submit a service request and check its status without assistance in a moderated walkthrough of the M3 build.
NFR-004	STK-001, Project Master Brief, Project Scenario: CivicConnect, p.6	The system should ensure that service requests do not become corrupted or duplicated during submission.	Medium	Service request is submitted and stored in database without corruption. Rollback occurs for corrupted data and recovery message is displayed. Requester cannot submit duplicate service requests.
NFR-005	STK-006 Project Master Brief, Project Scenario: CivicConnect, p.6	The system should protect user data and enforce role-based access.	High	A requester can only see their requests. A staff can only see requests inside their jurisdiction. Senior management has access to oversight information. No one can gain access to another user's data unless under strict, authorized reasons.
NFR-006	STK-006, STK-008	The system should encrypt passwords and verification tokens once a user creates a new account.	High	Hashing should convert passwords and tokens to hashed values. Hashed passwords and tokens must be stored in the database.
NFR-007	STK-001	The system should allow users to use the platform via any device.	Medium	The platform can be used with a phone, tablet, laptop, and desktop computer.
NFR-008	STK-001, STK-002, STK-003, STK-004, STK-005, STK-006	The system should ensure that it is available 99.9995% of the time.	High	The platform should have a maximum RTO of 4 hours (City and County of SAN FRANCISCO, 2023).
NFR-009	STK-001	The system should allow the requester to submit service requests quickly.	Medium	The median time from opening the form to confirmed submission is 60 seconds or less for a typed description of about 20 words.
NFR-010	STK-006	The system should ensure that a backup database is ready if the main database fails.	High	Standby database takes main database's place during database failure with a RPO of less than 15 minutes (City and County of SAN FRANCISCO, 2023).
NFR-011	Past similar system (Airport Company South Africa, n.d.), Project Master Brief, 2. Project Scenario: CivicConnect, p.6	The system should ensure that 210 concurrent users can use the system.	Medium	If 210 users use the system at the same time, the system itself should not crash.
NFR-012	STK-008	The system should ensure that a single-time email notification is sent to the user to confirm successful registration.	High	A user receives 6-digit code notification via email and can confirm registration and land to home page.

5. Constraints

5.1 Purpose

This section defines the conditions within which CivicConnect must be engineered and delivered. The constraints cover scope, schedule, cost, resources, quality, security, technology and governance, as required by the Milestone 1 Brief and CivicConnect Master Project Brief (Belgium Campus, 2026a; Belgium Campus, 2026b).

The final technology stack, architecture, database, hosting platform and deployment model are not baselined during Milestone 1. These matters are recorded as Forward Engineering Considerations and will be evaluated through controlled engineering decisions in later milestones.

5.2 Application of Constraints

Each constraint is linked to its source, engineering implications and required evidence. Where a constraint creates a measurable expectation for system quality or security, the corresponding non-functional requirement is recorded in the Requirements Register and traced through the RTM. Constraints and requirements remain linked controlled artefacts and are not duplicated.

5.3 Scope constraints

ID	Constraint	Engineering implication	Source and evidence
CON-SCP-01	The committed system scope must be baselined at the end of Milestone 1. Baselined scope may not be expanded or silently altered without formal change control.	Divide requirements into in-scope, out-of-scope and deferred scope. Assess every proposed change for its effects on requirements, schedule, cost, resources, quality, security, testing, deployment and risk.	Master Brief pp. 8 and 13-14 Evidence: Approved scope baseline, change requests, impact analyses, PED history and RTM updates.
CON-SCP-02	The final solution must deliver the minimum requester, staff and management capabilities specified in the Master Project Brief.	Translate the stated capabilities into detailed, prioritized and testable requirements. The team may refine them but may not remove them merely to simplify implementation.	Master Brief pp. 6-7 Evidence: Functional requirements, acceptance criteria, RTM and final verification evidence.
CON-SCP-03	Additional features may be included only when stakeholder value justifies their schedule, cost, quality, security, testing, deployment and maintenance impact.	Protect the minimum capabilities by deferring lower-value optional features when capacity is limited.	Master Brief p. 7 Evidence: Prioritized requirements, scope statement, risk register and approved change record.
CON-SCP-04	Detailed architecture, stack selection, persistence, API, user-interface implementation, Continuous Integration implementation and production deployment are outside the required Milestone 1 decisions.	Record future concerns and selection criteria now, but defer final technical choices until evidence is sufficient.	M1 Brief pp. 4-5 Evidence: Forward Engineering Considerations Register and Decision Log.

5.4 Schedule constraints

ID	Constraint	Engineering implication	Source and evidence
CON-SCH-01	The project must progress through four formal milestones within the SEN381 delivery period.	Plan milestone-appropriate work so that the M1 baseline supports later architecture, construction, verification and deployment.	Master Brief p. 8 Evidence: Milestone plan, backlog, issues, repository history and sign-offs.

CON-SCH-02	Milestone 1 evidence must be ready by 9 September 2026.	Complete drafting, review, correction, two-reviewer approval and baseline sign-off before the official date.	Official lecturer schedule to be added Evidence: Project board, issue dates, PR reviews and signed baseline record.
CON-SCH-03	Repository and engineering evidence must be produced progressively. Bulk reconstruction shortly before assessment is not acceptable.	Commit requirements, risks, decisions and documentation throughout the milestone.	M1 Brief p. 4 Evidence: Authentic sequence of issues, commits, branches, PRs, reviews and merges.
CON-SCH-04	Schedule pressure may not be handled by silently removing testing, review, security work or required scope.	Re-prioritise optional scope, record schedule variance, assess risk and control any material baseline change.	Master Brief p. 15 Evidence: Variance record, updated plan, risk entry and change decision.

5.5 Cost and resource constraints

ID	Constraint	Engineering implication	Source and evidence
CON-CST-01	The team should prefer free or low-cost development, repository, testing and hosting services where practical.	Technology evaluation must include free-tier quotas, expiry conditions and potential charges. A service must not be described as free without evidence.	Master Brief pp. 8 and 15 Evidence: Pricing research and the Milestone 2 technology decision matrix.
CON-CST-02	The team has approved a direct development budget of R0. Any proposed paid service requires formal approval and impact analysis.	Paid tools or services require an affordability and access assessment for all team members.	Team decision to be confirmed Evidence: Cost register and approved decision or change record.
CON-CST-03	Likely operational costs beyond the educational project must be identified even when the prototype uses free-tier services.	Consider hosting, storage, database use, traffic, backups, monitoring, notifications and maintenance in later selection.	Master Brief pp. 8 and 15 Evidence: Cost forecast and final operational-cost evaluation.
CON-RES-01	The team must consist of three registered students.	Plan scope for three people and manage the reduced capacity caused by a member becoming unavailable.	M1 Brief p. 1 and Master Brief pp. 8 and 10 Evidence: Team registration, working agreement, task ownership and risk register.
CON-RES-02	All three members remain collectively responsible for the engineered product even when roles are assigned.	Share knowledge and documentation so every member can locate, explain and defend the complete baseline.	Master Brief p. 10 Evidence: Peer reviews, shared documents and individual defence.
CON-RES-03	Each student must produce authentic individual contribution evidence.	Allocate meaningful issues, commits, authored PRs, reviewed PRs and controlled-document contributions to every member.	Master Brief pp. 10-11 Evidence: Repository history, project board and document history.
CON-RES-04	Each substantive change entering protected main requires two approvals from team members other than the author. Self-approval is not accepted.	In a three-person team, both non-authors must review every substantive PR. Reviewer availability must be included in planning.	Master Brief p. 11 Evidence: Branch rules, PR reviews, author responses and merge history.
CON-RES-05	Technology must suit team capability, available equipment and the institutional environment. Institutional support is not guaranteed for every technology.	Investigate compatibility, learning effort and fallback options before commitment.	Master Brief pp. 15-16 and 24 Evidence: Decision matrix, official documentation and proof of concept where appropriate.

5.6 Quality constraints

ID	Constraint	Engineering implication	Source and evidence

CON-QUA-01	Functional and non-functional requirements must have stable unique identifiers, sources, priorities and appropriate acceptance criteria.	Do not baseline ambiguous or untestable requirements.	Master Brief p. 12 and M1 Brief p. 3 Evidence: Requirements catalogue, review evidence and RTM.
CON-QUA-02	Quality attributes must be measurable and later supported by traceable evidence. Unsupported claims such as fast, reliable, usable or scalable are insufficient.	Define measurable NFRs without prematurely prescribing a stack.	Master Brief pp. 8-9 and 14 Evidence: NFRs, acceptance criteria, test plans, results and quality reports.
CON-QUA-03	Changes entering the controlled baseline must undergo meaningful peer review.	Review correctness, requirement alignment, maintainability, security, testing, dependencies and traceability.	Master Brief p. 11 Evidence: PR comments, corrections, re-review and approvals.
CON-QUA-04	Quality Assurance and Quality Control evidence must be retained throughout the project.	Later define process controls, suitable tests, quality gates and a defect register. Coverage alone does not prove correctness.	Master Brief p. 14 Evidence: Review procedure, test strategy, test results, gates and defect register.
CON-QUA-05	The PED must remain one coherent, evolving and controlled engineering record.	Preserve baselined history and connect changes to approved records instead of silently overwriting content.	Master Brief pp. 8-9 Evidence: PED history, approvals, repository history and milestone sign-offs.

5.7 Security constraints

ID	Constraint	Engineering implication	Source and evidence
CON-SEC-01	Security must be addressed throughout requirements, design, implementation, repository control, testing, deployment and operation.	Identify security requirements and sensitive-data implications during Milestone 1.	Master Brief pp. 8 and 14 Evidence: Security requirements, risk register, later threat review and tests.
CON-SEC-02	Passwords, tokens, API keys, private keys and other secrets may not be committed to the repository.	The future stack must support externalised and environment-specific configuration. Revoke any exposed secret and treat the event as an incident.	Master Brief pp. 11 and 14-15 Evidence: Repository rules, scanning evidence, configuration guidance and incident records.
CON-SEC-03	Service-request information and actions must be limited to authorised users according to their responsibilities.	Support authentication, authorisation, controlled status transitions and least privilege without prescribing the identity technology.	Master Brief pp. 7 and 14 Evidence: Security requirements, permission model and later authorisation tests.
CON-SEC-04	The system should collect, display and retain only the information needed to manage and audit a service request. Sensitive information must not be unnecessarily exposed.	Review data fields, access, logging and retention during design.	CivicConnect scenario and Master Brief pp. 6, 12 and 14 Evidence: Data classification, privacy requirements, access tests and risk treatment.
CON-SEC-05	External libraries, services and platforms must be assessed for maturity, support and known vulnerabilities before adoption.	Do not choose a stack only because it is familiar. Unsupported or vulnerable dependencies require rejection or mitigation.	Master Brief pp. 14-16 Evidence: Dependency research, decision matrix, scans and ADR.
CON-SEC-06	Confidential data, credentials and inappropriate personal information may not be submitted to external AI systems. Material AI work must be recorded and independently verified.	AI output is not authoritative; the team remains accountable for every submitted artefact.	Master Brief p. 12 Evidence: AI Usage Register, human review, references and technical verification.

5.8 Technology and governance constraints

ID	Constraint	Engineering implication	Source and evidence
CON-TEC-01	No programming language, framework, architecture, database, hosting platform or deployment model is prescribed.	Keep Milestone 1 technology-neutral. Select the eventual stack against requirements, constraints and documented trade-offs.	Master Brief p. 8 Evidence: Deferred decision followed by a Milestone 2 ADR and weighted matrix.
CON-TEC-02	The eventual stack must support the approved functional, quality and security requirements.	Eliminate candidates that cannot meet mandatory requirements or produce suitable verification evidence.	Master Brief pp. 15-16 Evidence: Requirements-to-technology evaluation and proof-of-concept results.
CON-TEC-03	The engineering and deployment approach must distinguish the purposes, configurations, data and risks of development, test, staging and production environments.	Candidates must support controlled configuration, testing, production-like staging and repeatable deployment.	Master Brief pp. 14-15 Evidence: Deployment analysis and later environment evidence.
CON-TEC-04	The platform must support controlled configuration, secrets, logging, monitoring, recovery and rollback appropriate to the project.	Use these as later selection criteria although final implementation is deferred.	Master Brief p. 15 Evidence: Forward Engineering Considerations, decision matrix and operational evidence.
CON-TEC-05	Vendor lock-in and the effect of a technology becoming unavailable or unsuitable must be considered before commitment.	Prefer portability where practical and document accepted switching costs.	Master Brief p. 16 Evidence: ADR, decision matrix, risk register and contingency plan.
CON-GOV-01	The team must use one controlled repository, protect main, prohibit substantive direct work on main and use PRs for substantive changes.	Configure governance in Milestone 1 before substantial coding starts.	Master Brief p. 11 Evidence: Live repository settings, branch rules and PR history.
CON-GOV-02	Baseline artefacts must use consistent identifiers and remain traceable.	Link requirements, risks, decisions, changes, tests and defects across the lifecycle.	Master Brief pp. 9 and 12-13 Evidence: PED, RTM, registers and repository links.
CON-GOV-03	External claims must be cited, while implementation claims require authentic project evidence.	Use research to justify why a practice matters and project evidence to prove what the team did.	M1 Brief p. 8 Evidence: In-text citations, reference list, repository evidence and tests.

5.9 Proposed measurable quality boundaries

Quality area	Proposed boundary
Performance	Under an agreed normal test load, 95 percent of ordinary user actions should be completed within 5-10 seconds, excluding delays caused by external notification services.
Reliability	Once the system confirms successful submission, the request shall be stored once with a unique reference and remain retrievable after a page refresh and normal application restart.
Data integrity	Status and assignment changes must preserve an attributable chronological history.
Accessibility	Core workflows should support keyboard use, readable contrast, clear labels and understandable validation feedback.
Recovery	Define a Recovery Point Objective and Recovery Time Objective after persistence and hosting options are understood.
Maintainability	Require automated checks, peer review and documentation for substantive changes before merge.
Compatibility	Decide the supported device and browser range from stakeholder evidence and verify it later.

Security	Make authentication, role-based authorization, input validation, secure secret handling and sensitive-data protection verifiable through tests and review.

5.10 Constraint interactions and ripple effects

Interaction	Ripple effect and required response
Schedule and scope	Adding features without changing the deadline reduces time for review, security and testing. Defer optional scope or formally change the baseline.
Team size and governance	Both non-authors must review each substantive PR. This improves quality and shared understanding but can delay merges when a member is unavailable.
Cost and technology	A free tier may impose storage, runtime, traffic, backup or monitoring limits that reduce availability or create later migration work.
Security and usability	Stronger authentication protects sensitive requests, but unnecessary friction may discourage users. Compare options against security risk and accessibility.
Auditability and privacy	Action histories support accountability, but excessive logging may expose personal information. Record important actions without unnecessary sensitive content.
Quality and schedule	Review, testing, correction and regression checks need time. If implementation falls behind, revisit scope or schedule rather than removing quality controls.
Technology and resources	A capable but unfamiliar stack may create a learning burden that the three-person team cannot absorb within the schedule.

6. Architecturally Significant Requirements

ID	ASR / Quality Attribute	Source	Measurable/Testable Expectation	Architecture / Data / Technology / Design Influence
ASR-001	The system must utilize recurring cost of R0 on a single API process and a single PostgreSQL instance, making use of free tools only.	CON-CST-01, CON-CST-02	The CI and staging cost are R0. If a tool must be paid for, the request must be recorded in the Change Impact Analysis. Deployment will make use of a single API instance and a single database.	Influences technology since software components should be free tier and instead of using a costly architecture required for deployment, the team will use one instance of a deployment architecture instead of distributed.
ASR-002	Requesters, front-line staff, department supervisors and senior management must only view the service request information which they may see depending on their roles and may not retrieve sensitive data of other users.	CON-SEC-03, REQ-023	API calls for requester only return their own requests; API calls for front-line staff only return requests that belong to their jurisdiction. For all endpoints that are guarded, 100% of unauthorized calls are declined.	Makes access control a cross-cutting concern that is enforced in one place. Every guarded route passes through an Express middleware chain, which is the Chain of Responsibility pattern: verify JWT, then authorise (user_type, ownership, staff category), then the controller. Each check is one reusable function that either passes the request on or stops it with 401 or 403, so role checks are not repeated inside controllers. Repositories also filter every query by owner and category, so a query never returns a whole table.
ASR-003	Passwords and verification tokens should only appear as hashes and should never be recorded in the API calls and logging table. The same hashing constraint remains applicable to verification tokens when the deferred email-verification capability is re-enabled.	REQ-024, REQ-030	Plaintext passwords or tokens should not exist in user, logging and API responses, must be verified by automated test and database inspection.	Influences the technology to ensure that the backend stack has a hashing library for passwords and tokens, and the data-model is influenced that the password attribute and token attribute of users will not contain string data type but hashed data type. The “logging” entity must not include tokens and passwords when SQL triggers occur.
ASR-004	When database experience failure, its service must be restored with a RTO of 4 or less and a maximum of 15 minutes of data may be lost.	REQ-026, REQ-028	Backup restoration or standby failover must meet RT0 <= 4 hours and RPO <= 15 minutes.	Influences the architecture as well as data choices to have a replicated database having the backup function, and the Data & Persistence Baseline must address the backup and recovery feature.
ASR-005	A service request must be persisted after processing with slowness of quality removed from the request path.	REQ-019, REQ-027	API establishes persistence within 10 seconds with normal load, and the user journey takes a median time of 60 seconds or less.	Slow work must be kept off the request path. The status change is saved synchronously and the requester email is sent afterwards by an event handler that the HTTP response does not wait for. This is the Observer pattern applied in-process (D12, Section 11.2).
ASR-006	The system should handle 210 simultaneous users on the platform.	REQ-029	Service request operation should not go beyond 10 seconds, and the API will not crash.	Impacts the architecture to pivot towards a backend that is stateless
ASR-007	The system must ensure that requests that are submitted are neither corrupted, lost, nor duplicated.	REQ-022	If failure occurs during the middle of a submission, zero records must be persisted. If a double-submit occurs, a zero duplicate request must be made.	Influences persistence to make use of transactional writes for submission of service requests and any other CRUD operations that can cause corrupted/loss data.
ASR-008	Notification channels can be placed in or swapped without affecting request and staff logic.	REQ-020, REQ-030	Placing a second notification channel on the platform should not change request and staff modules. The fakeNotificationChannel will be added to test second channel addition.	The Notification module is separated from request and staff logic by a NotificationChannel interface, which is the Strategy pattern (D11, Section 11.1). The email library is wrapped by a MailSender Adapter, so a channel or a provider can be added or replaced without changing the core logic.
ASR-009	All staff actions that makes changes to a request must be recorded and may not be altered.	REQ-013, REQ-016	All status, category, and assignment changes must be recorded as as action row in the action entity. Actions may neither be deleted nor updated.	Influences data to keep an action/audit table for staff actions, and this ASR also influences design to separate read-only queries from transactional writes for the activity overview module.Admin module only allows management to read audit data read-only.
ASR-010	All user journeys are able to be performed on phone, tablet, laptop and desktop computer with a responsive web client.	REQ-025	User journeys can be completed in a browser with estimate 360 pixels with tablet and desktop widths, it should not be native app.	Influences the choice of using frontend technology to make the website responsive for any device, also ensuring it stays within a budget of R0, and the team of 3 people can work on it.

7. Architecture Decision & Diagrams

This section answers the M2 question "how should CivicConnect be structured, and why?". It starts from the approved ASRs in Section 6, compares realistic structural alternatives and records the selected structure with its diagrams. Technology choices are recorded in Section 9 (D05), the data model in Section 8, the API contract in Section 12 (D06) and environment direction in Section 13 (D07); this section references them rather than repeating them. Decisions made here are logged in Section 17 as **D09** (architecture style) and **D10** (request lifecycle state model).

7.1 Architectural drivers

Not every ASR changes the *structure* of the system. The table shows which Section 6 ASRs shape the architecture and what each one forces. ASR-003 and ASR-008 are affected by change requests raised in M2 (CR-004 and CR-005, Section 18) and are shown with their proposed direction.

Table 7-1. *Section 6 ASRs and their structural consequence*

ASR	Driver in short	Structural consequence in this architecture
ASR-001	R0 budget, free tools	One deployable API process and one database, not a distributed system. Free, open-source components only (D05).
ASR-002	Role-scoped data access	A single enforcement point: every request passes through the API middleware chain (Chain of Responsibility: verify JWT, then role, ownership and category check), and repositories filter by user_type and ownership (Repository pattern). No page reads the database directly.
ASR-003	No secret stored in plain text	Owned by the User module: passwords hashed with bcrypt (D05). Email confirmation tokens (D13, CR-004) are also stored only as hashes.
ASR-004	99.9% availability, failover	Stateless API so an instance can be restarted or replaced; all state in PostgreSQL. Standby database and failover are deferred to M3 (Section 13); see 7.6 and CR-006.
ASR-005	Fast submit and processing	Slow side effects (email) run after the transaction commits, in an event handler that the HTTP response does not await (Observer pattern, D12). Oversight counts are computed by SQL, not in the browser.
ASR-006	210 concurrent users	No in-memory session state in the API (JWT), shared pg.Pool, so the API tier can scale horizontally behind one address.
ASR-007	No lost or corrupted requests	One relational store with ACID transactions. A status change and its action row commit together or not at all. This part is deliberately not event-driven: it is explicit orchestration inside one service method (D12).
ASR-008	Notifications, replaceable channels	A separate Notification module behind a `NotificationChannel` interface (Strategy pattern), with the SMTP library behind a `MailSender` interface (Adapter pattern), so channels and providers are added or swapped without touching request logic (D11).
ASR-009	Oversight and staff audit	Every staff change writes an action row. A read-only Admin module serves summaries and audit views, separate from write use cases.
ASR-010	Any device	One responsive React client (no native apps), shared by all roles.

7.2 Architecture alternatives considered

Four realistic structures were scored against the drivers above. Weights reflect the tightest limits identified in Section 5: team capacity and schedule (CON-RES-01, CON-SCH-01), then data integrity and access control.

Table 7-2. *Weighted architecture decision matrix (1 = poor, 5 = strong; total = Σ weight × score ÷ 5)*

Criterion (weight)	Layered modular monolith	Microservices	Serverless functions + hosted backend	Unstructured monolith (routes query the DB directly)
Team capability and schedule, CON-RES-01 / CON-SCH-01 (25)	5	1	3	5
Transactional integrity and audit, ASR-007 / ASR-009 (20)	5	2	3	4
One security boundary, ASR-002 (20)	4	2	3	2
Running cost, ASR-001 (15)	4	1	4	5
Testability (10)	4	3	3	2
Replaceable parts, ASR-008 (10)	4	4	2	1
Weighted total (out of 100)	**89**	**38**	**61**	**70**

- **Microservices (rejected).** Splitting request, staff and oversight into separately deployed services would put a network boundary inside one business transaction (status change plus action row), which then needs distributed transactions or an outbox to satisfy ASR-007. It multiplies hosting, secrets and monitoring work against ASR-001 and a three-person team. A2 Task 1 already found a message broker disproportionate for in-process collaboration, and Section 12.1 reached the same view for the client–server boundary.
- **Serverless functions with a hosted backend (rejected).** Low cost, but authorisation ends up split between function code and provider-specific data rules, weakening ASR-002, and it raises the vendor lock-in risk recorded in Section 16 (Heroku, n.d.).
- **Unstructured monolith (rejected).** Lowest set-up effort, but route handlers that each query the database scatter validation and role checks across files. That is the "god service" failure described in A2 Task 1, and the audit guarantee in ASR-009 would depend on every developer remembering to write the action row.
- **Layered modular monolith (selected, D09).** One process and one database keep transactions simple, while module and layer boundaries give the separation ASR-002 and ASR-008 need. This follows the common advice to start with a well-modularised monolith and split only when evidence demands it (Fowler, 2015; Richards and Ford, 2020).

7.3 Selected architecture

CivicConnect is a **layered modular monolith**: one React client and one Node.js/Express API process backed by one PostgreSQL database, plus an external SMTP email provider. Inside the API, code is organised first by business module and then by layer. Figure 7-1 shows the system boundary and Figure 7-2 the internal structure.

ADDED 4 the Design Patterns

Architecture style and design patterns. "Layered modular monolith" (D09) is the architecture style: it decides how the whole system is divided into modules and layers. Design patterns work one level lower: they are named,

reusable solutions for how specific classes inside those layers cooperate. CivicConnect uses six of them, each for a reason that traces to an ASR. Section 11.5 (Table 11-8) lists all six with their code locations; the dependency rules below show where each one sits in the layers.

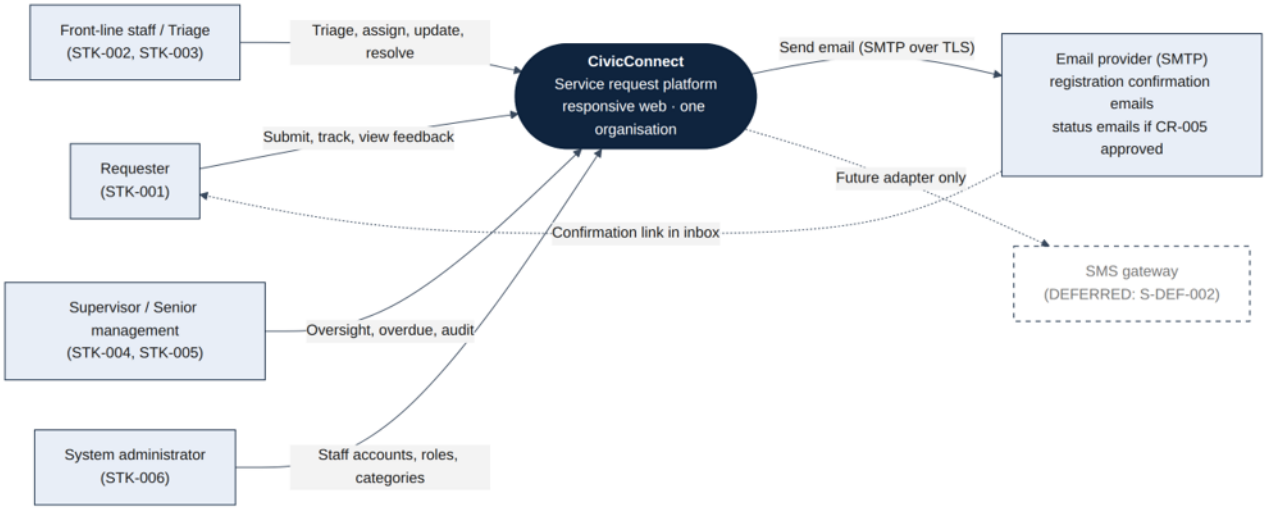


Figure 7-1. *System context. The SMTP provider sends registration confirmation emails (D13) and status emails if CR-005 is approved. SMS is drawn dashed because it stays deferred (S-DEF-002).*

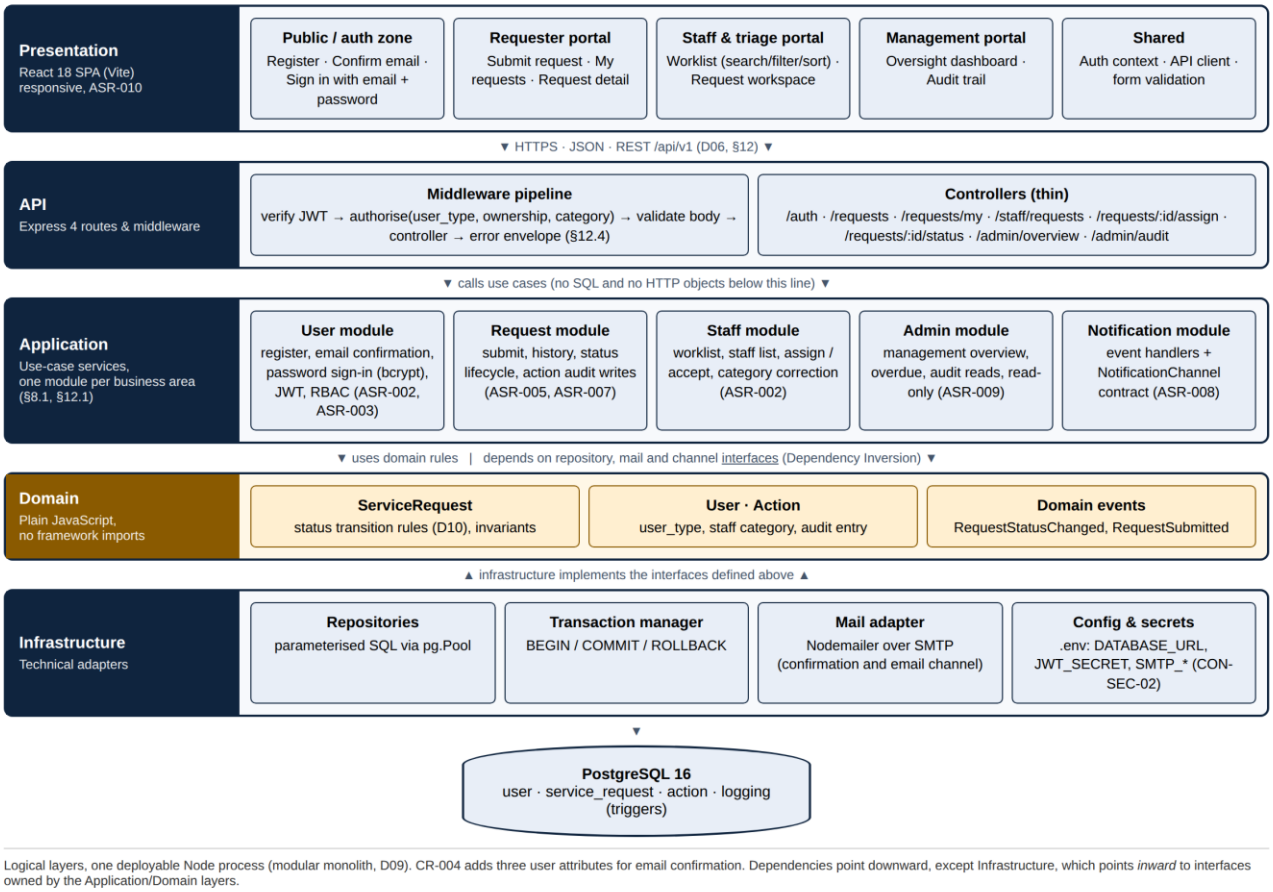


Figure 7-2. *Logical architecture: five layers and five business modules. Arrows show the allowed dependency direction.*

Dependency rules

These rules are checked in pull-request review now and by an ESLint import rule from M3:

1. The React client calls the API only over HTTPS/JSON (/api/v1, Section 12). It holds no business rules beyond form checks for fast feedback.
2. Controllers are thin: they translate HTTP to a use-case call and back and never contain SQL. Before a controller runs, the request passes through the middleware chain (**Chain of Responsibility**): verify JWT, authorise, validate body. Any step can stop the request.
3. Application services own the use cases and the transaction boundary. They depend on repository, mail and channel *interfaces*, never on pg or Nodemailer directly. The concrete objects are created once in the composition root (app.js) and passed into each service through its constructor (**Dependency Injection**), which is what lets tests pass in fakes.
4. Domain code imports nothing from Express, React or pg. Status transition rules live here so they can be unit tested without a database.
5. Infrastructure implements the interfaces. Repositories hold all SQL (**Repository** pattern) and NodemailerMailSender wraps the email library (**Adapter** pattern). Only this layer knows SQL, connection strings, SMTP settings and secrets.
6. Modules call each other only through their application services, never through another module's repository.

Table 7-3. *Module responsibilities and requirement allocation*

Module			Responsibility	Requirements	Data it writes (Section 8)
User	Registration with email confirmation, email + password sign-in, JWT issue, role and category policy used by every other module	REQ-023, REQ-029, REQ-030, REQ-033	user (incl. email_verified, verification_token_hash, verification_expires_at; CR-004)		
Request	Submit a request once, requester history, status lifecycle and the action rows it produces	REQ-001 to REQ-006, REQ-013 to REQ-017, REQ-028, REQ-031, REQ-035	service_request, action		
Staff	Category-scoped worklist, full details, staff list, assign / accept, category correction	REQ-007 to REQ-012, REQ-024	service_request (owner, category), action		
Admin (management)	Read-only summaries, overdue detection, all-requests view, audit trail	REQ-018 to REQ-022	none (reads only, ASR-009)		
Notification	Subscribes to domain events (the observer side of the **Observer** pattern, D12) and delivers requester messages through interchangeable NotificationChannel implementations (**Strategy** pattern, D11).	REQ-006, REQ-026	none		

Naming. Sections 8.1 and 12.1 name User, Request, Staff and Admin modules; this section keeps those names and adds the Notification module that ASR-008 asks for. The "Persistence module" in Section 8.1 (trigger-written logging table) is part of the Infrastructure layer here, because no application code writes to it. The Admin module serves the management role; system-administrator tasks (staff accounts and categories) are handled by the User module.

7.4 Physical view and environments

The five layers in Figure 7-2 are logical: they are folders and import rules, not servers, so adding a layer costs no network hop. Physically there are four runtime parts: the browser, one Node.js process, one PostgreSQL server and the email provider. Section 13 sets out the environment strategy; Figure 7-3 shows how this architecture maps onto it.

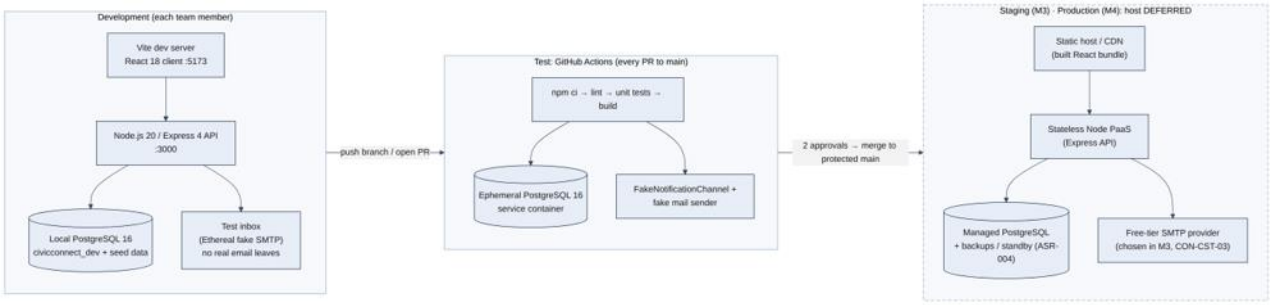


Figure 7-3. *Environment mapping. Development and CI are defined now; the staging/production host and SMTP provider are chosen in M3 (dashed).*

- **No real email outside production.** Development uses a fake SMTP inbox (Ethereal) and CI uses the FakeNotificationChannel and a fake mail sender, so tests never email real people and need no SMTP secret.
- **Stateless API.** Authentication is a signed JWT and all domain state is in PostgreSQL, so an API instance can restart or be duplicated (ASR-004, ASR-006). Email confirmation tokens are also stored in PostgreSQL, never in memory.

7.5 Request lifecycle state model (D10)

Scope item S-IN-007 requires a state model covering rejection, cancellation and reopening. The baseline currently uses several status vocabularies: REQ-004 (pending, accepted, rejected, updated, completed), REQ-013 (pending, in progress, completed), REQ-022 (open, closed, resolved, overdue), Section 8.1 (Pending → Accepted/Assigned → In Progress → Completed/Resolved, or Rejected/Closed) and Section 12.2 (Pending, Accepted, In Progress, Completed, Rejected). REQ-017 also sets a *closed* request to "Rejected". Workflow, audit, requester feedback and oversight must all read the same values, so the state model is an architectural decision.

D10 keeps the names the team already uses in Sections 8.1 and 12.2 and adds only what S-IN-007 requires.

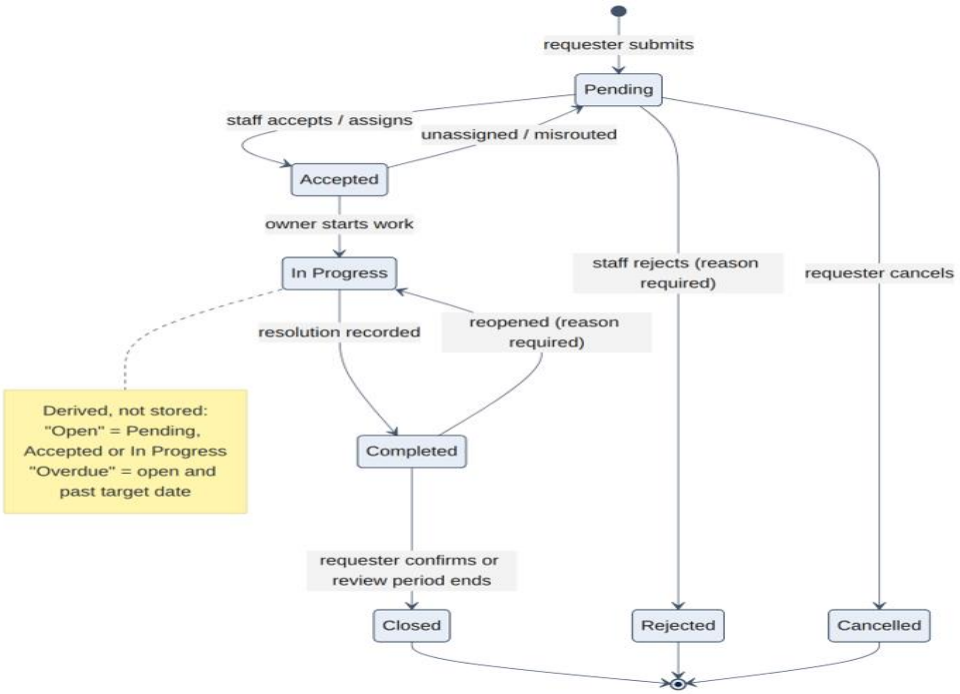


Figure 7-4. *Request lifecycle. Seven stored states. "Open" and "Overdue" are derived when read, never stored.*

Table 7-4. *Mapping of existing status wording to the D10 state model*

Existing wording (source)		D10	Change needed
pending (REQ-004, REQ-013, §8.1, §12.2); open (REQ-022)	Pending. "Open" is derived: Pending, Accepted or In Progress	None	
accepted (REQ-004); Accepted/Assigned (§8.1)	Accepted (a staff owner exists)	None	
in progress (REQ-013, §8.1, §12.2)	In Progress	None	
updated (REQ-004)	Not a state. An update is an action row shown in the timeline	REQ-004 criterion reworded (CR-003)	
completed (REQ-004, REQ-013, REQ-016); resolved (REQ-022)	Completed. Oversight labels it "Resolved"	None	
closed (REQ-017, REQ-022)	Closed. REQ-017's "set to Rejected" is replaced: rejection is its own transition from Pending	REQ-017 criterion corrected (CR-003)	
rejected (REQ-004, §12.2)	Rejected (reason required)	None	
overdue (REQ-022)	Derived: open and past the target date, which is itself calculated as created_at plus the approved number of days for the category (S-IN-010), so no extra database attribute is needed	None	

(new) cancel, reopen (S-IN-007)	Cancelled (requester, from Pending only); Completed → In Progress on reopen	Adds behaviour required by S-IN-007 (CR-003)

Section 8.3 requires that "Status must follow a valid transition sequence". D10 supplies that sequence as a **finite state machine**. It is implemented as a transition table (a map from each current state to the states it may move to) inside the ServiceRequest domain object, whose changeStatus(toStatus) method throws InvalidTransitionError for any move that is not in the table. The API turns that error into 409 Conflict (Section 12.4), and a CHECK constraint limits the stored value to the seven names.

This is **not** the Gang of Four State pattern. The State pattern gives each state its own class with its own behaviour; CivicConnect's states differ only in which moves are allowed, so one table is simpler, easier to unit test (Section 11.2) and easier to compare with Figure 7-4. The team would move to State classes only if states start to need different behaviour.

7.6 Key runtime interaction: changing a request's status

Figure 7-5 shows the most architecturally important flow because it exercises ASR-002 (authorisation), ASR-007 and ASR-009 (one transaction with its action row) and ASR-005 and ASR-008 (notification after commit, not blocking the response). It is also the one flow in which five of the six patterns appear together. The design reasoning is recorded in Section 11 (D12).

Reading Figure 7-5 by pattern:

- **Step 3, Chain of Responsibility.** The middleware chain verifies the JWT and checks user_type and category before the controller runs.
- **Steps 5, 11 and 12, Repository.** The status service reads and writes only through repositories, inside one transaction.
- **Steps 6 to 9, state machine (D10).** ServiceRequest.changeStatus accepts or rejects the move.
- **Steps 13 and 14, Observer.** After COMMIT the service publishes RequestStatusChanged to the EventDispatcher, which calls each registered handler without awaiting it.
- **Step 16, Strategy and Adapter.** NotifyRequesterHandler calls send() on each NotificationChannel; the email channel sends through the MailSender adapter.

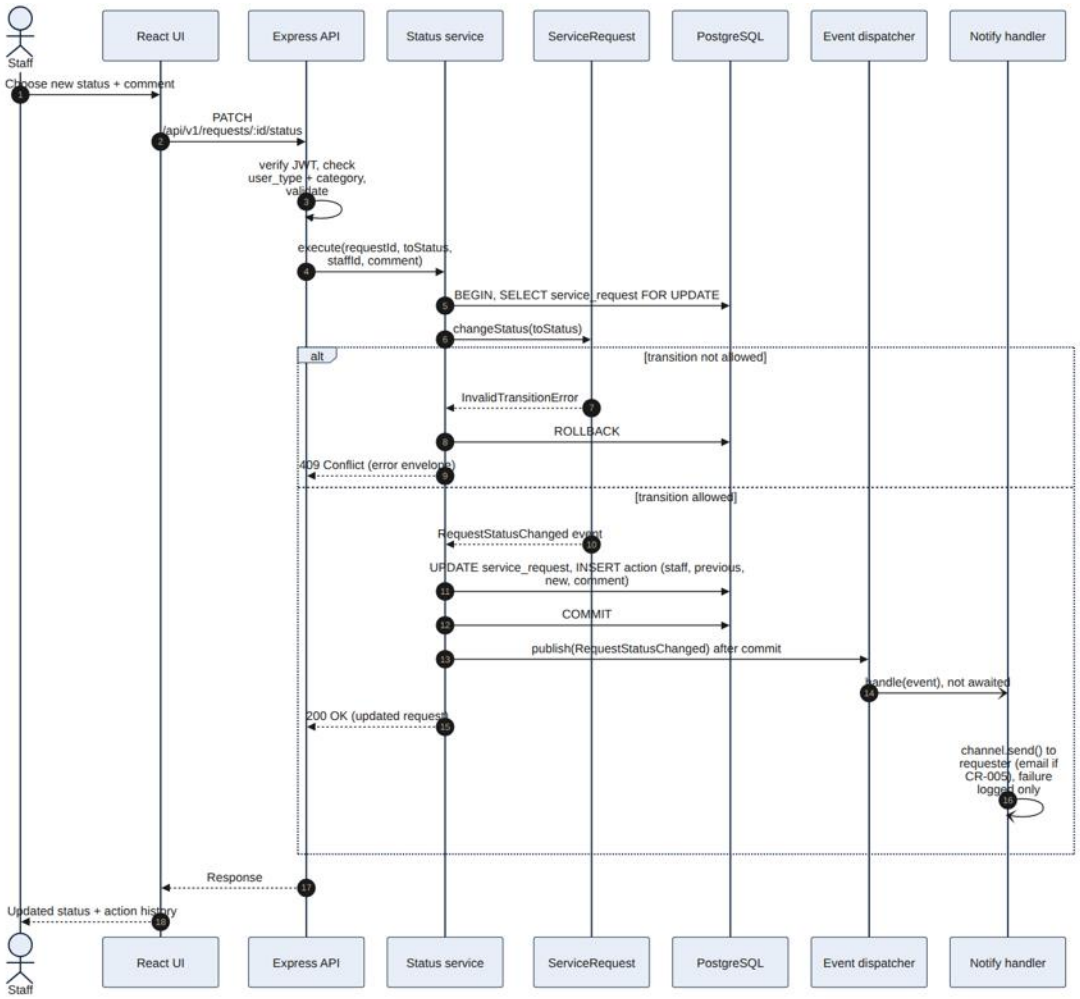


Figure 7-5. *Status change. The state change and the action row commit together; the notification handler runs after commit and is not awaited.*

7.7 Availability position (ASR-004)

ASR-004 asks for 99.9% availability (about 8.8 hours of downtime a year) and database failover. The architecture keeps that path open: the API is stateless, all state sits in one PostgreSQL database, and pg.Pool reconnects after a database switch. However, three baselined statements do not yet agree. Section 8.5.3 assumes automatic failover to a replica, Section 8.5.4 plans weekly manual backups (so up to seven days of data could be lost), and Section 13 defers failover to M3. None of these can be verified on free-tier hosting in M2. **CR-006** proposes restating ASR-004, REQ-034 and REQ-036 as measurable recovery targets (RPO and RTO) once the M3 host is chosen, with the standby database as the preferred way to meet them. Until the CR is decided, the ASR stands and is tracked as R05.

Decisions recorded from this section (Section 17)

D09 Layered modular monolith with five modules. **D10** One request lifecycle state model. Full records are in the Engineering Decision Log.

8. Data & Persistence Baseline

8.1 Entity Overview

From milestone 1, the separate roles such as requester, front-line staff, department supervisors, senior management, and system administrator have been merged into a single entity known as the "user" to ensure simplicity. However, to distinguish between different user types, a "user_type" attribute will be introduced to only include three roles which are requester, staff (front-line staff), and management (department supervisor, senior management, and system administrator).

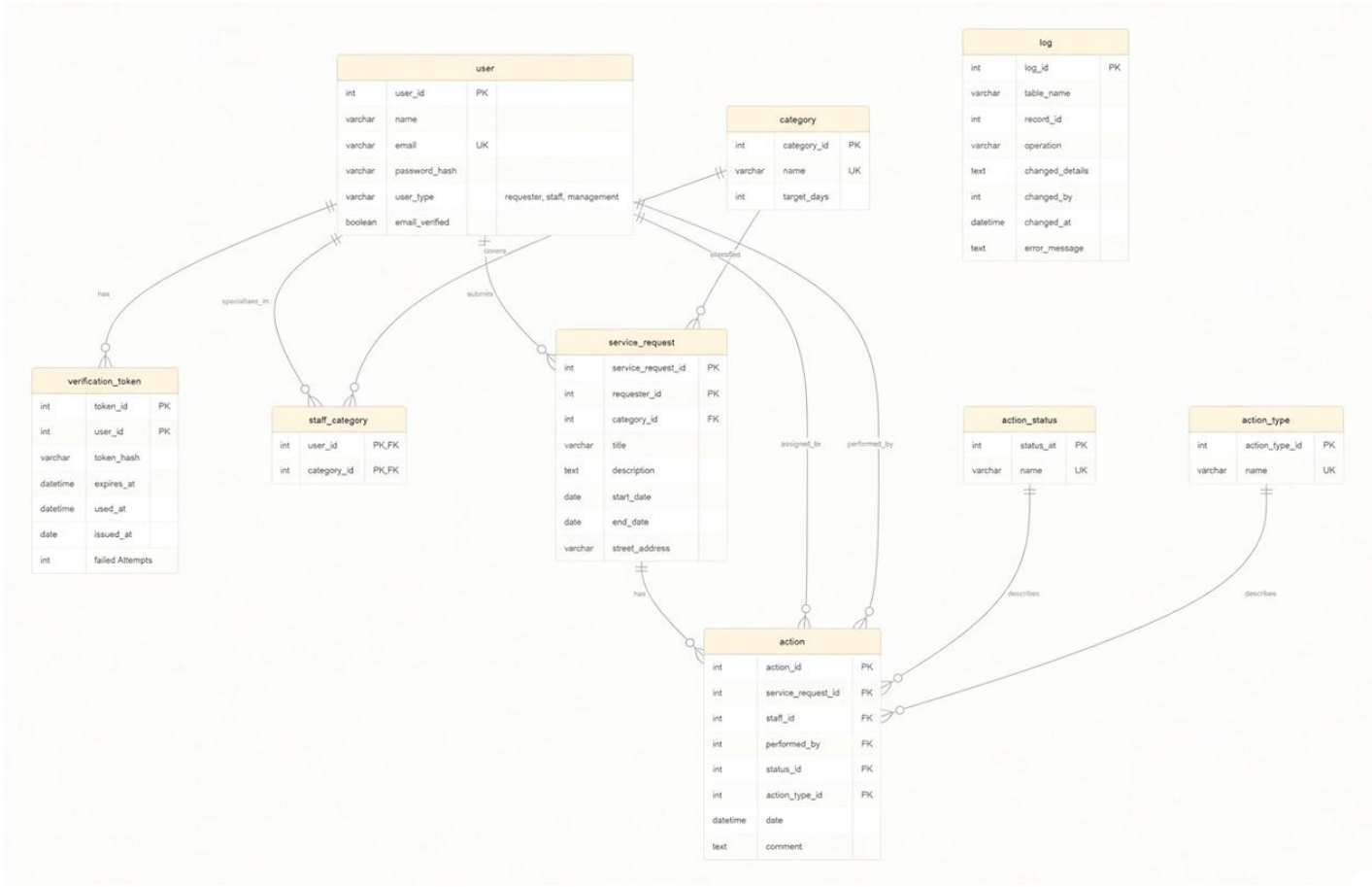
The "service_request" entity represents the submitted service request details. For every change made to that request, such as status, category change, and comment, it will be logged in the "action" entity for audit trail. Category data for both staff and requests are stored in the "category" entity. The "action_status" entity represents the status of a request whenever an action occurs, the "action_type" entity represents the operation that the action perform on the request, the "verification_token" entity represents the one-time token sent to new users before successful registration, and the "log" entity represents SQL triggers whenever a developer made a mistake in application code for diagnostics. The "category", "action_status", and "action_type" are lookup tables.

Entity	Purpose	Owning module	Lifecycle
user	Entity that represents the requester, staff, and management. Contains the secrets, user type, and if email was verified (REQ-017, 023, 024, 030).	User module	Registered (email_verified = false) → Verified → Active
verification_token	Entity that represents the single-time 6-digit verification token for registration once, contains token hash, expiry date, usage date, issue date, and failed attempts (REQ-017, 024, 030).	User module	Issued → Used or Expired
category	Entity that represents a lookup for request and staff categories. Makes links with requests and staff with the same categories (REQ-002, 006, 018).	Staff module	Seeded; changes rarely
staff_category	Entity that represents links to staff with their category service request. Defines staff jurisdiction (REQ-006, 023).	Staff module	In maintenance when staff are created.
service_request	Entity that represents a requester's service request containing the following attributes: title, description, start date, end date, street address, requester and category (REQ-001 to 004). Status and assigned staff are stored in other entities, not here.	Request module	Created once; progress is the sequence of its actions
action	Entity that represents the actions taken by staff for making changes to service requests. Who made the request, who is assigned, the status, the action, type of change, date, and comment (REQ-003, 005, 009 to 012, 016).	Request module	Insert only
action_status	Lookup statuses utilised by staff actions (REQ-003, 010).	Request module	Seeded
action_type	Lookup different action types persisted in history.	Request module	Seeded
log	Entity that represents SQL triggers that occur due to error CRUD operations made by the developer on user, service_request, and action entities. Shows who, what, where, when, and how.	None	Trigger-generated

8.1.1 Changes from the M2 baseline

Aspect	M2 baseline (PED v2.0, D08)	M3 model	Effect
Entities	Used to be 4 entities: user, service_request, action, logging	Now, there are 9 entities: add category, staff_category, verification_token, action_status, action_type; logging renamed log	More joins, but each fact is stored once. Reference data must be seeded.
Request status	The “status” column was originally part of the service_request entity.	The “status” column is not stored on the service_request entity; rather, the last action of a request shows the current status.	The last action on a service request must provide the current status of that request (see 8.5.2). [Update D10 and §7.5 wording]
Assignee	The “staff_id” column was originally part of the service_request entity.	The “staff_id” column is now moved to the “action” entity.	Service requests should be locked as soon as a staff ID is assigned to a request via action (REQ-009).
Category	The “category” column was originally part of the “user” and “service_request” entities.	A new distinct “category” entity exists with lookup values where service request has link with category through “category_id” column. In addition to that, the “staff_category” entity is a link only for staff to be linked with a category, to exclude requesters and management from having categories.	Dropdown that shows categories that only staff can be assigned to. [Confirm: a category correction (REQ-018) updates category_id and is also recorded as an action]
Verification	The “verification_token_hash” and “verification_expires_at” were columns part of the “user” entity.	A new distinct “verification” entity exists that now contain the following columns: “token_hash”, “expires_at”, “used_at”, "issued_at", and "failed_attempts". The “email_verified” column is moved to the “user” entity, since it has nothing to do with the verification token.	6 digit code can be reissued and is single-use; supporting ASR-003.
Action	The “new_status” and “action_type” columns were part of the “action” entity. The “staff_id” is in the “action” entity.	Status and action type are added in the two new entities, now their foreign IDs are added in the “action” entity along with “staff_id”. The “performed_by” column is for staff that accepts/assign the request to a staff whereas the “staff_id” is the assigned staff.	Both the staff who assigned or accepted the request and the staff who is assigned to do the request are recorded.
Creation date	The “service_request” entity used to have the “created_at” column.	No created_at column.[Confirm: creation time is the date of the request’s first action]	Every service request must be linked with an initial action to provide creation date.
Diagnostics	logging	log, columns unchanged	Rename only.

8.2 Entity Relationship Diagram



8.3 Justification of Persistence Model

Entity	Structure	Relationships	Access patterns	Integrity	Consistency	Sensitivity	Growth
user	Fixed, tabular: user_id, name, email, password_hash, user_type, email_verified.	One-to-many to service_request (requester), action (staff_id, performed_by), verification_token and staff_category.	Read at every login and on every protected request for RBAC. email_verified is checked at login.	The "email" column must be unique. The "password_hash" column must never be raw text. The "user_type" column is only used for requester, staff, and management. The "email_verified" column is true only if the requester confirmed code for registration.	User type is checked on every request to prevent wrong role usage.	Secrets and email of users may not be shared to other users and are not included in the "log" entity.	Linear with registrations; low.
verification_token	Fixed, tabular: token_id, user_id, token_hash, expires_at, used_at, issued_at, failed_attempts.	Many-to-one with user.	Written at registration or resend; read once at confirmation.	The hash token is stored, never the raw 6-digit code. Any used or expired codes will not be accepted. The user ID must exists.	A transaction should contain both "used_at" and "email_verified" columns and mark them together.	High, since it is only shown to the requester, it is never recorded in the "log" entity.	Low; expired rows can be purged.
category	Fixed, tabular: category_id, name.	One-to-many with service_request and staff_category.	Read-mostly lookup for the dropdown and for staff users. Written only by the seed data which shows predefined estimate duration of days.	Category names are unique. They cannot be deleted when referenced.	Not critical; can be cached.	None.	Negligible.
staff_category	Fixed, tabular: composite key (user_id, category_id).	Many-to-many bridge between staff users and categories.	Read when the staff list is made (REQ-006) and for area verifications (REQ-023).	This composite key will prevent duplicates. The user must be a user type of staff to be linked with the "staff_category" entity.	Low.	Low, since it only shows what category is linked to which staff.	Small.
service_request	Fixed, tabular: service_request_id, requester_id, category_id, title, description, start_date, end_date, street_address.	Many-to-one with user (requester) and category; one-to-many with action.	Request submitted once. Linked to the very last action to find current status. Management can filter requests by status, date, and category.	The "end_date" column must not be earlier than the "start_date" column. Both "requester_id" and "staff_id" columns must reference existing records with correct user types.	Both service request and action inserted in same transaction. Locking row (SELECT … FOR UPDATE) serialises simultaneous assignments and duplicate verifications (REQ-009, 022).	Seeable for requester who submitted it, staff who is assigned to request with same category, and management.	One row per request; high.
action	Fixed, tabular: action_id, service_request_id, staff_id, performed_by, status_id, action_type_id, date, comment.	Many-to-one with service_request, user (twice both for assigned staff and assignee staff), action_status and action_type.	Action is created once first request was submitted to indicate current status = "pending". In addition, actions are also created when a status is changed, a comment is added or if a category was made right. Last action per request is the current status. Management requires this for the audit trail of staff.	No alterations may occur on the action table as it is only meant to create new actions. Foreign keys must always reference real staff IDs.	E.g. if a transaction contains the changed status for a service request, in that same transaction, the action is also recorded (REQ-016, ASR-007, ASR-009).	Only management is allowed to view audit of staff actions whereas requesters may only see feedback (REQ-005).	Fastest growing; index on (service_request_id, date).
action_status, action_type	Fixed, tabular: id and name.	One-to-many with action.	Read-mostly lookups.	Each status and type must be unique; they cannot be deleted while referenced at the same time.	Not critical.	None.	Negligible.

log	Fixed, tabular: log_id, table_name, record_id, operation, changed_details, changed_by, changed_at, error_message.	There are no foreign keys; the "table_name" and "record_id" will say which rows was affected in log.	Written by SQL triggers and only read during debugging.	The "changed_by" and "changed_at" columns are filled in automatically. A record must not be ignored when a bug occurs.	Consistency is relaxed since the platform's operations will not be affected by background diagnostics.	Password and token hashes may not be stored in log (ASR-003).	Fast: one record per fault.

8.4 Conclusion

All nine entities will be fixed, structured, and are related to each other with foreign keys; when queries are executed, joins will be required, which relational databases can perform well. Merging requester, staff, and management into a single entity called user reduced complexity. However, RBAC must be enforced using the user attribute "user_type" to categorize user roles instead of relying on which user belongs to which table. In addition to that, categories are only linked to staff via the staff_category.

Categories, statuses, and action types are added into separate lookup tables to prevent repeated plain text and allow constraints to decline invalid values. The statuses of the service request are directly derived from the status of the last action recorded for that service request to keep consistency and proper audit trail, to prevent unnecessary queries from involving more than one status column. Access for category, service_request, and user is read-heavy, append-only for action, and diagnostics only for the log entity.

Taking this into consideration, these features fit relational databases which is strengthened by A2 research describing that relational databases require integrity constraints, security, strong consistency, fixed tables, and table dependencies. Therefore, based on these features listed and A2 research, PostgreSQL, which is a relational database, is best suited for the CivicConnect platform. However, a trade-off would be that changing requirements impact fixed schemas, and since the ERD is now normalized, more joins need to be included with careful indexing.

8.5 Bottlenecks, SPOF, Scalability, Availability, and Backup/Recovery

8.5.1 Single Point of Failure (SPOF)

If only a single database lives for the CivicConnect platform and it fails, all operations performed on the platform will fail as well; every user cannot log in, requesters cannot submit requests, staff cannot make updates to the service requests, and management does not receive full visibility of actions recorded for audit trail. The *log* entity will also fail, since it stores bugs that occurred in application development; it is not there to save records when database is down. Therefore, to solve this issue, a standby database can be added alongside the main database, mirroring precise actions of it, so that it can take the main database's place if failure occurs for it.

8.5.2 Scalability

Entities that are expected to grow fast are *service_request*, *action*, and *log*. The *service_request* grows for every added request, *action* grows more for every status update or new comment, and *log* entity grows for every bug found when developer attempted CRUD operations across 3 tables. If these are not addressed, it brings two bottlenecks:

Read bottleneck: staff and management should be able to filter service requests using category, status, start/end date(s), requester, and assignee. (Make a change to the requirement for management needing to filter requester and assignee as well) Dashboard query is a big risk, since assignee and status belong to the same last action. Indexes that can be planned for: action (service_request_id, date), action (staff_id), service_request (requester_id) and service_request (category_id).

Write bottleneck: If retention policies for the database are not taken into consideration, and *action* and *log* entities continue to perform many write volumes, records can be archived depending on their retention period, removing them from the primary database completely.

8.5.3 Availability

The RTO must follow 4 hours whereas the RPO must fall underneath 15 minutes (REQ-026, REQ-028, CR-006), supported by a standby database that takes over if the main database fails. Making use of the RTO of 4 hours, the availability of the CivicConnect platform should be 99.9995% (REQ-026).

8.5.4 Backup and Recovery

Backups should be frequent, manually, or automated to fall under the threshold of the defined RPO (15 minutes).

The *service_request* and *action* entities contain most write operations; they require the highest priority for backup, ensuring that their tables are restored in correct manner. If they fail, they will not be supporting the active and seeable audit trail that is managed by management personnel. Since *log* only concerns about diagnostics and not operational purposes, losing its data will not affect the platform data itself; lowering its backup/restoring priority to low, with the *user* and *verification_token* following. Lookup tables can also be added/changed with values.

Make sure you add a change request, decision, and risk if necessary to keep track of important changes.

9. Technology Stack Decision

9.1 Selection Criteria and ASR Alignment

With CON-TEC-02, CON-RES-05 and CON-CST-01/ CON- CST-02 in mind, technologies were evaluated against the approved the Architecturally Significant Requirements (ASR-001 to ASR-010), the teams skill set (CON-RES-01), the delivery schedule (CON-SCH-01), the budget constrains (ASR-001), supply-chain security (CON-SEC-05) and cross environment deployment compatibility (CON-TEC-03)

9.2 Comparative Technology Matrix

Architectural Tier / Concern	Selected Technology	Alternative Considered	Evaluation Against ASRs, Constraints & Research Evidence	Licensing, Cost & Dependency Risk
Client / Presentation Layer (User, Staff, Admin UI views)	**React** (SPA via Vite) + Responsive CSS	Native Mobile Apps (Android/iOS) or Angular	Satisfies ASR-010 (REQ-032, NFR-008) by delivering a single responsive web codebase across phone, tablet, laptop, and desktop devices while respecting S-OUT-002 (native apps excluded). Peer-reviewed mobile engineering research confirms that while native apps offer hardware-level optimization, responsive web applications eliminate multi-platform maintenance overhead for small teams (Horn et al., 2023). Aligns with existing team capability (AI-017).	**MIT License** (R0 cost, satisfies ASR-001). Mitigates R08 (cross-device rendering failure)
Application / API Runtime (User, Request, Staff, Admin, Notification modules)	**Node.js** + **Express.js** + **Nodemailer**	ASP.NET Core (C#) or Spring Boot (Java)	Satisfies ASR-005 (REQ-025 <10s processing, REQ-035 <30s submission) and ASR-006 (REQ-037 210 concurrent users) via an event-driven, non-blocking I/O execution model. Nodemailer implements the MailSender adapter (D11) for ASR-008 notifications. Sharing JavaScript/JSON across client and server reduces context-switching friction across the 3-person team (CON-RES-01, CON-RES-05).	**MIT / OpenJS License** (R0 cost). Requires lockfile pinning (package-lock.json) to mitigate transitive npm package risks (CON-SEC-05)
Data & Persistence Layer (user, service_request, action, logging tables)	**PostgreSQL** + node-postgres (pg)	MongoDB (Document NoSQL)	Satisfies ASR-007 (REQ-028 / NFR-004 uncorrupted transactional storage) and Section 8.4 persistence requirements. Enforces strict ACID transactions, relational foreign keys (requester_id, staff_id, service_request_id referencing user.user_id and service_request.service_request_id), domain constraints (CHECK, NOT NULL, UNIQUE), and automated AFTER audit triggers into the logging table (Hellerstein, Stonebraker and Hamilton, 2007).	**PostgreSQL License** (Open-source, R0 cost). Mitigates R06 (corrupted submission / rollback failure)
Authentication & Credential Security	**JSON Web Tokens (jsonwebtoken)** + **bcrypt** (and Node crypto for CR-004)	Server-side In-Memory Sessions / Paid Auth0 (ASR-001 violation)	bcrypt satisfies baseline ASR-003 (REQ-030 / NFR-006) by hashing user.password before persistence (while Node's built-in crypto SHA-256 supports hashed login_token secrets if CR-004/D13 is adopted).	**MIT License** (R0 cost). Mitigates R01 (unauthorized role access) and R02 (credential exposure).

			Stateless JWT middleware encodes user_id and user_type (requester, staff, management) to satisfy ASR-002 (REQ-029 RBAC) and ASR-006 (210 concurrent users without session-store memory bottlenecks) (Marquis, 2024).	
Build, Dependency & Quality Gate Tooling	**npm** (npm ci + package-lock.json), **ESLint**, **npm audit**	Unpinned manual installs (npm install)	Empirical research on package managers demonstrates that committed lockfiles (package-lock.json) paired with clean CI installations (npm ci) are essential to guarantee deterministic, repeatable builds across developer machines (Gamage et al., 2026). Satisfies CON-QUA-04 and CON-SEC-05	**Open-Source / Built-in** (R0 cost). Prevents dependency drift between development and testing environments

9.3 Baseline Version and Compatibility Matrix

The Following versions are baselined across the application repository

Component	Baseline Version	Compatibility & Configuration Rule
Node.js Runtime	v20.x LTS	Active Long-Term Support; enforced across all three developer environments.
Express.js	^4.19.x	REST routing, request validation, and RBAC middleware execution
PostgreSQL Engine	v16.x	Relational storage for Requester, Staff, Management, Service_Request, and Action schemas.
React & React-DOM	^18.3.x	Component-driven SPA rendering for Requester, Staff, and Management views
pg (node-postgres)	^8.11.x	Parameterized SQL query execution and connection pooling (pg.Pool).
bcrypt & jsonwebtoken	^5.1.x / ^9.0.x	Password hashing (ASR-003) and stateless RBAC token signing (ASR-002, ASR-006)
nodemailer	^6.9.x	SMTP adapter implementing MailSender(D11,D13,ASR-008)
npm & Lockfile	v10.x (lockfileVersion: 3)	Deterministic dependency tree resolution via npm ci (Gamage et al., 2026)

10. Information Architecture

Wireframes and Rationale

The IA above sets out the zones and modules. The subsections below turn it into screens: a screen inventory, low-fidelity wireframes and the usability and accessibility reasoning behind them. The wireframes fix structure, content and behaviour, not visual style, so reviewers judge what each screen does. Final styling is not required in M2.

Clickable prototype: It covers all eleven screens, a phone-width view and numbered notes that trace each design choice to a REQ, ASR or decision ID. The screenshots below are the controlled copy and are also stored in the repository under docs/design/wireframes/.

10.1 Screen inventory

Table 10-1. *IA zones mapped to wireframed screens*

IA zone (above)			Screens	Landing page	Requirements
Public / Auth zone	Register · Check your email · Sign in	Sign in	REQ-023, REQ-029, REQ-030, REQ-033 (D13, CR-004)		
Requester portal (STK-001)	Submit request · My requests · Request detail	My requests	REQ-001 to REQ-006, REQ-027, REQ-031, REQ-035		
Staff & triage portal (STK-002, STK-003)	Worklist · Request workspace	Worklist	REQ-007 to REQ-017, REQ-024		
Management portal (STK-004, STK-005)	Oversight dashboard · Audit trail · All requests (read-only list)	Oversight	REQ-018 to REQ-022		
Admin zone (STK-006)	Staff accounts and categories	n/a	Not wireframed in M2 (10.6)		

Navigation is built from the signed-in user's user_type. A user never sees a menu item they are not allowed to use, which removes "access denied" dead ends and applies least privilege (REQ-029, ASR-002).

10.2 Wireframes

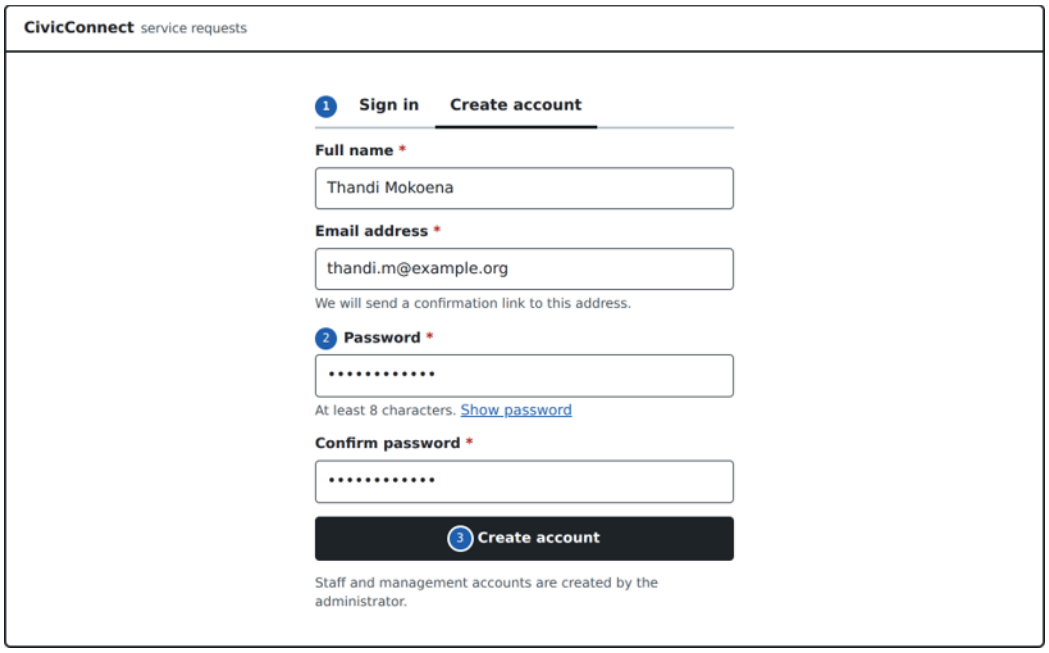


Figure 10-1. *Create account: name, email and password; the account is confirmed by email before first sign-in (REQ-023, REQ-030, D13).*

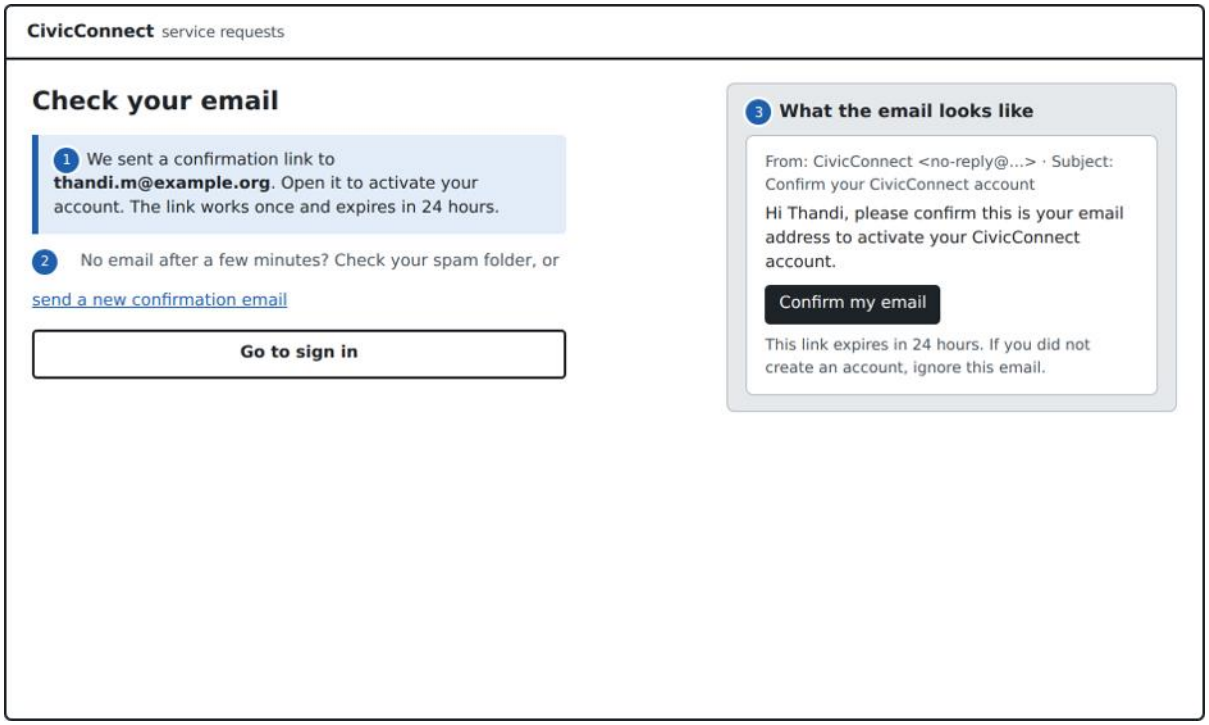


Figure 10-2. *Check your email: generic confirmation page, resend option and the confirmation email itself (D13).*

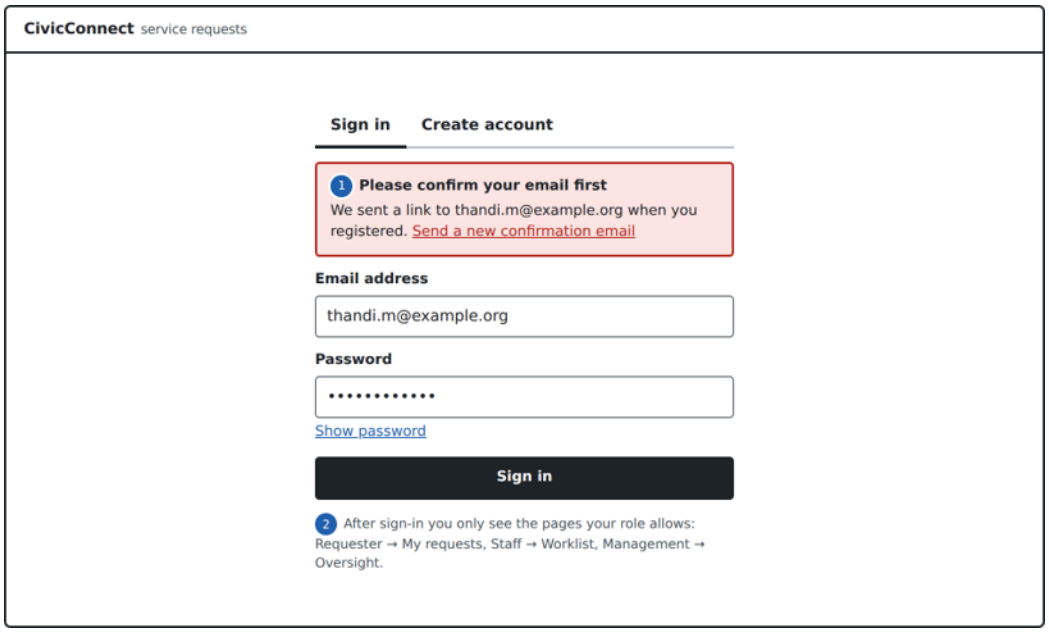


Figure 10-3. *Sign in with email and password, showing the message for an account whose email is not yet confirmed (REQ-029, D13).*

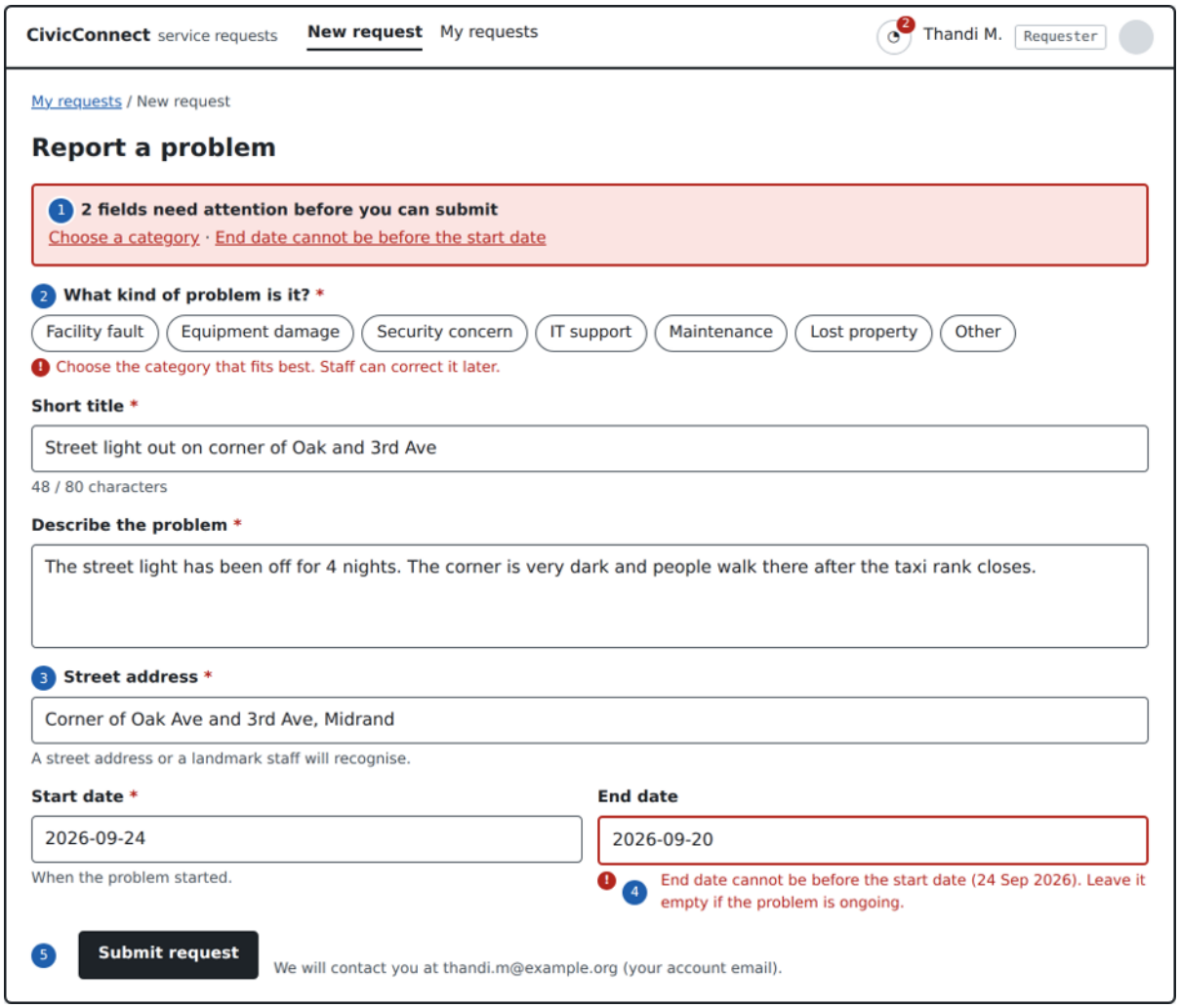


Figure 10-4. *Requester: submit request in its validation-error state (REQ-001 to REQ-003; CR-001 street address, CR-002 no contact number).*

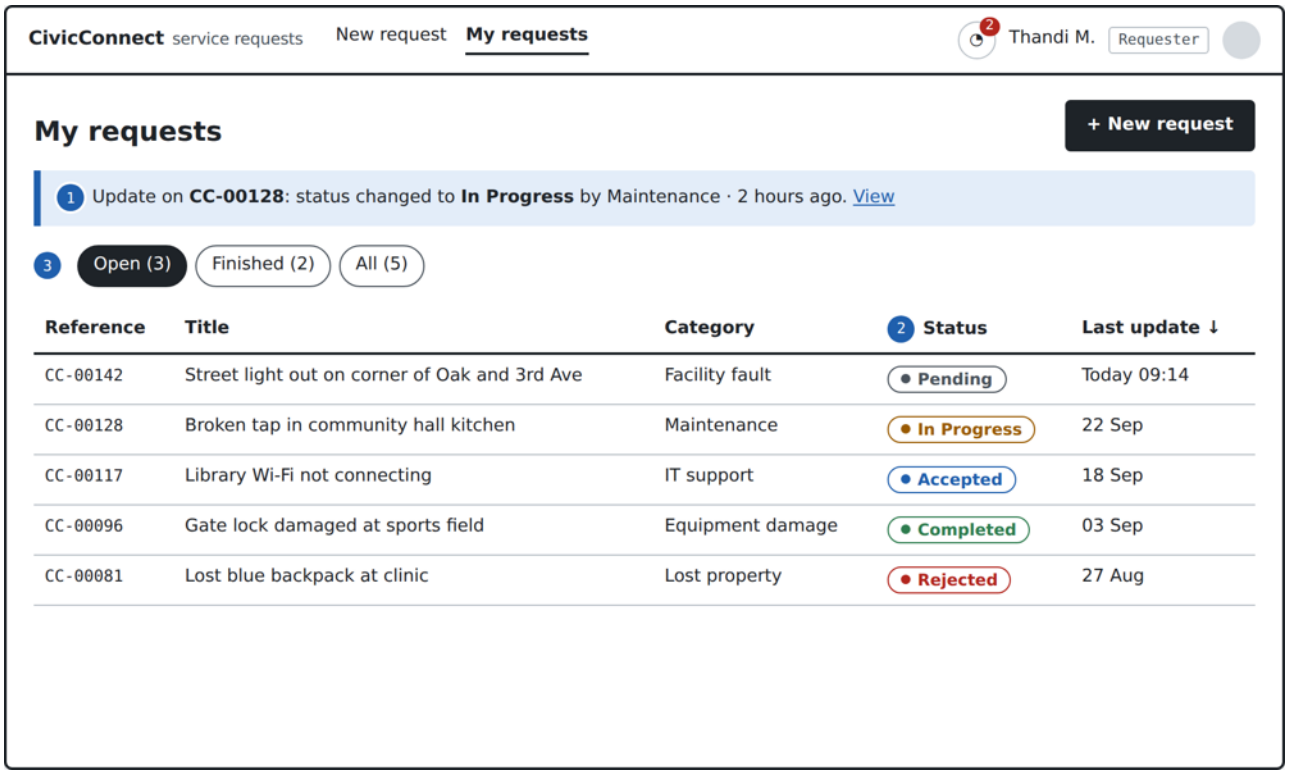


Figure 10-5. *Requester: my requests, with the latest feedback above the list (REQ-004 to REQ-006).*

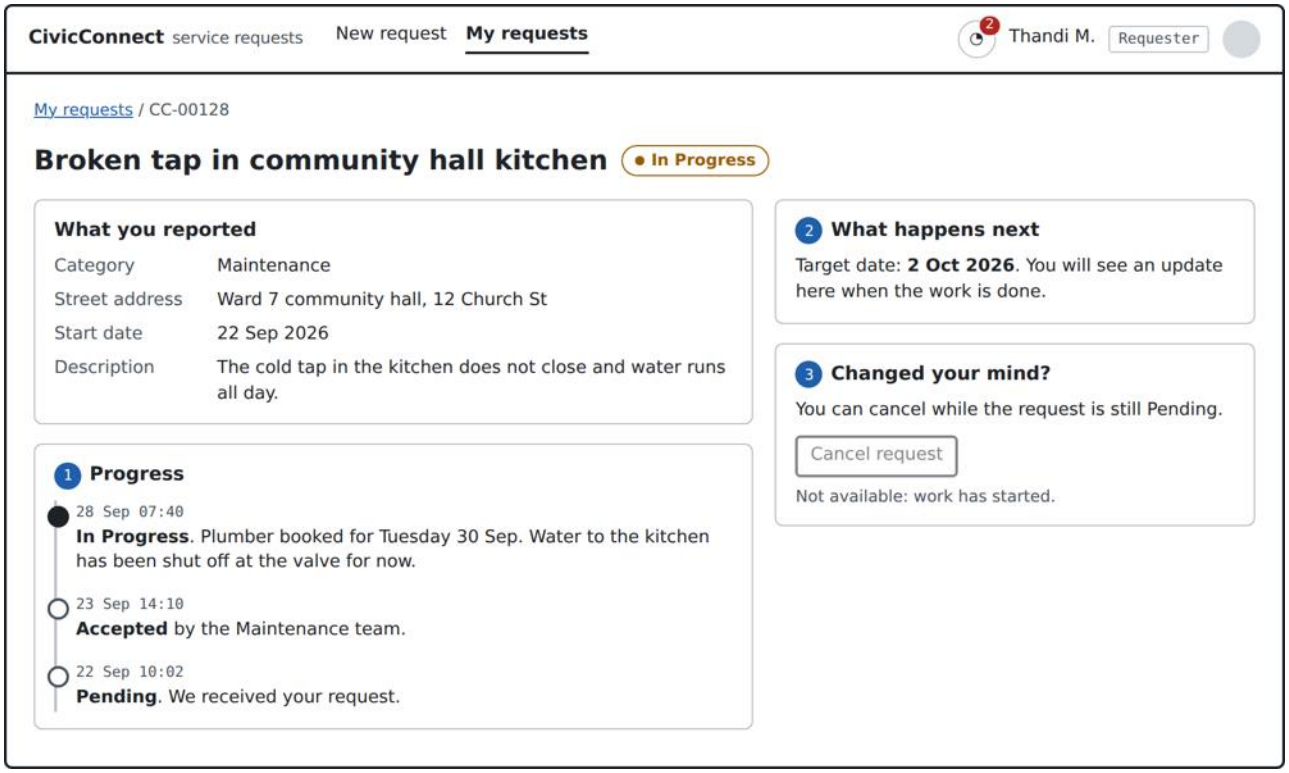


Figure 10-6. *Requester: request detail and plain-language progress timeline read from the action table (REQ-004, REQ-006, S-IN-010).*

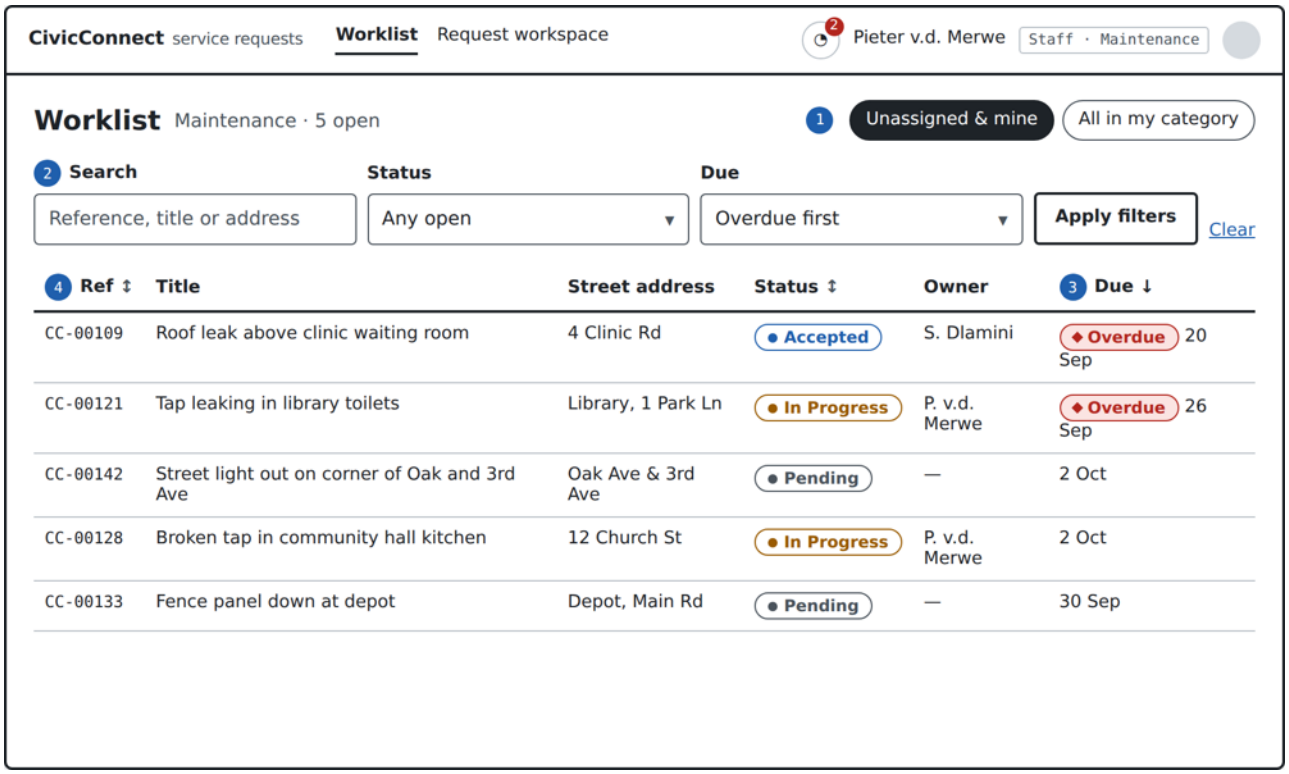


Figure 10-7. *Staff: worklist scoped to the staff member's category, with search, filter and sort (REQ-007, REQ-008, REQ-022).*

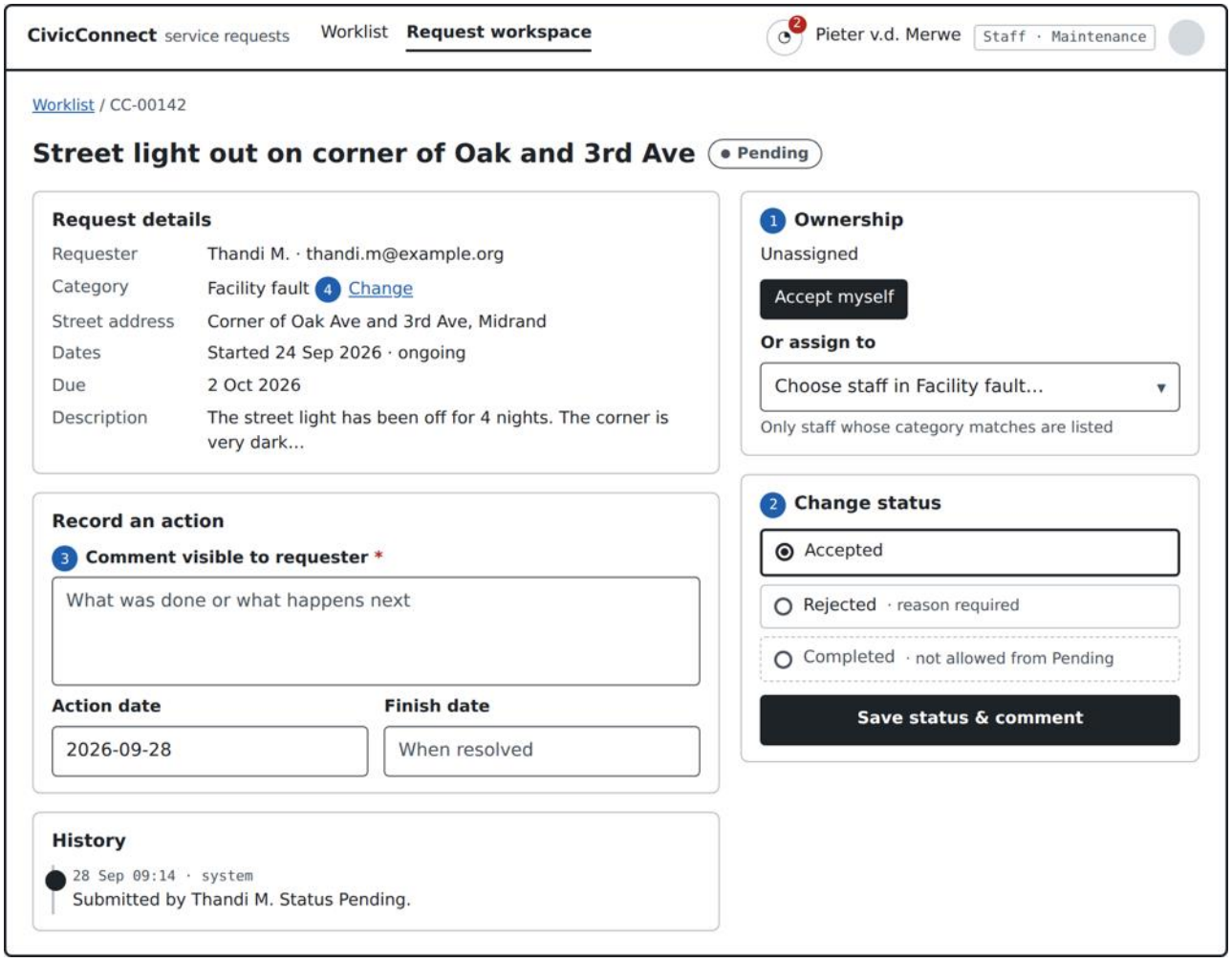


Figure 10-8. *Staff: request workspace for ownership, valid status transitions and actions (REQ-009 to REQ-017, REQ-024).*

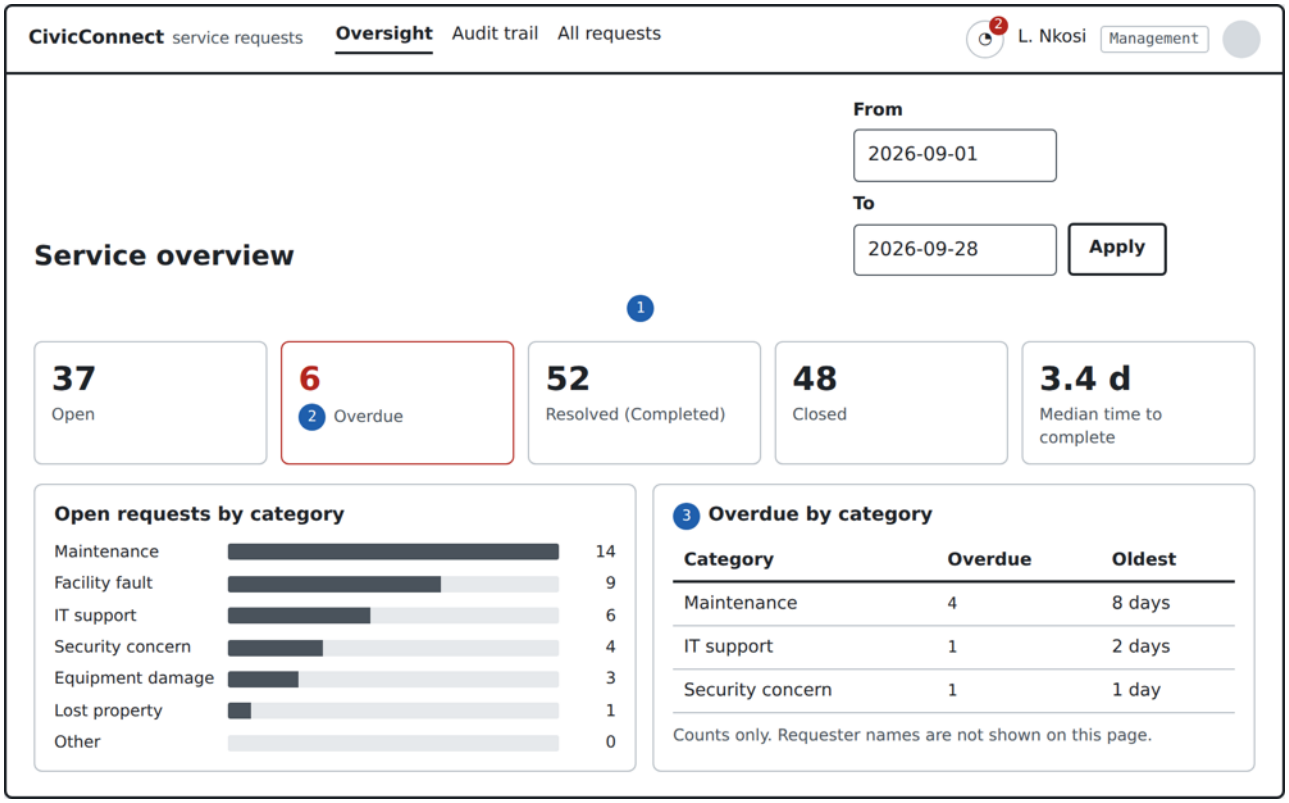


Figure 10-9. *Management: oversight dashboard with open, overdue, resolved and closed counts (REQ-018 to REQ-020, REQ-022).*

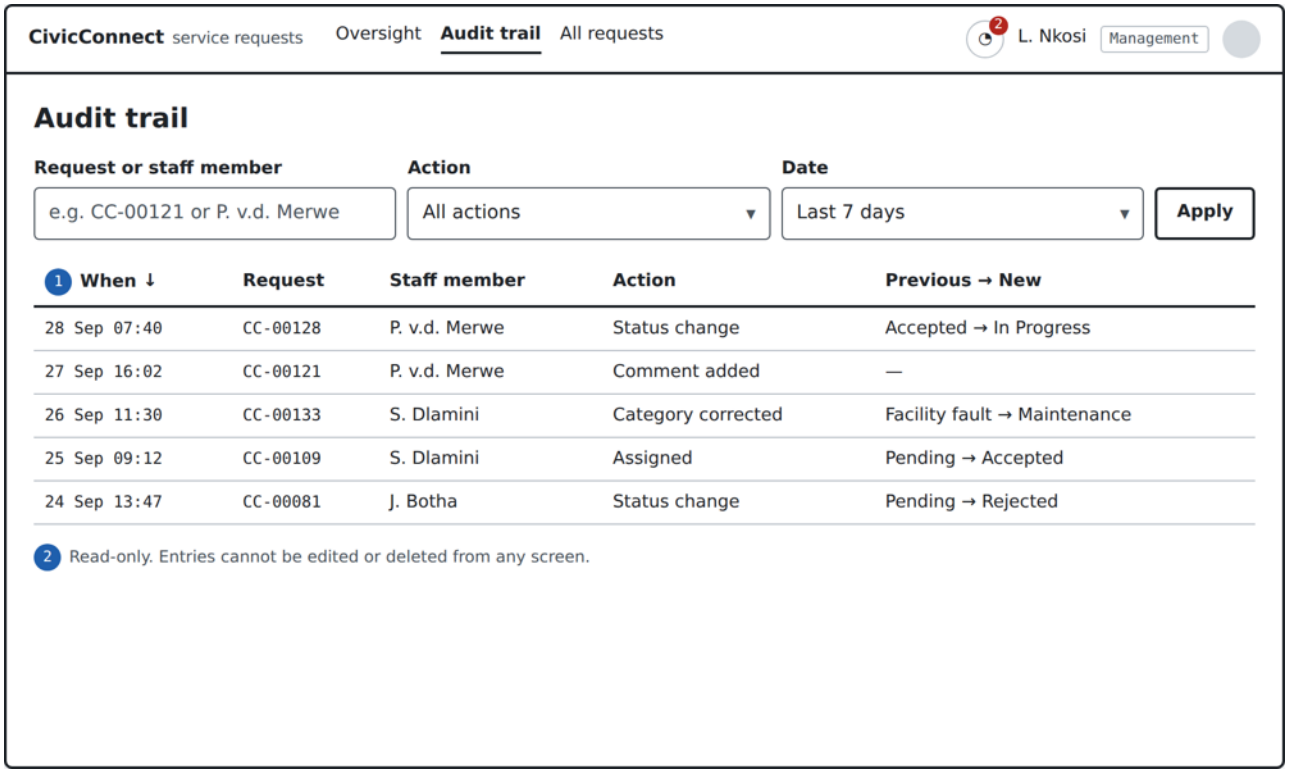


Figure 10-10. *Management: read-only audit trail built from action rows (REQ-021, ASR-009).*

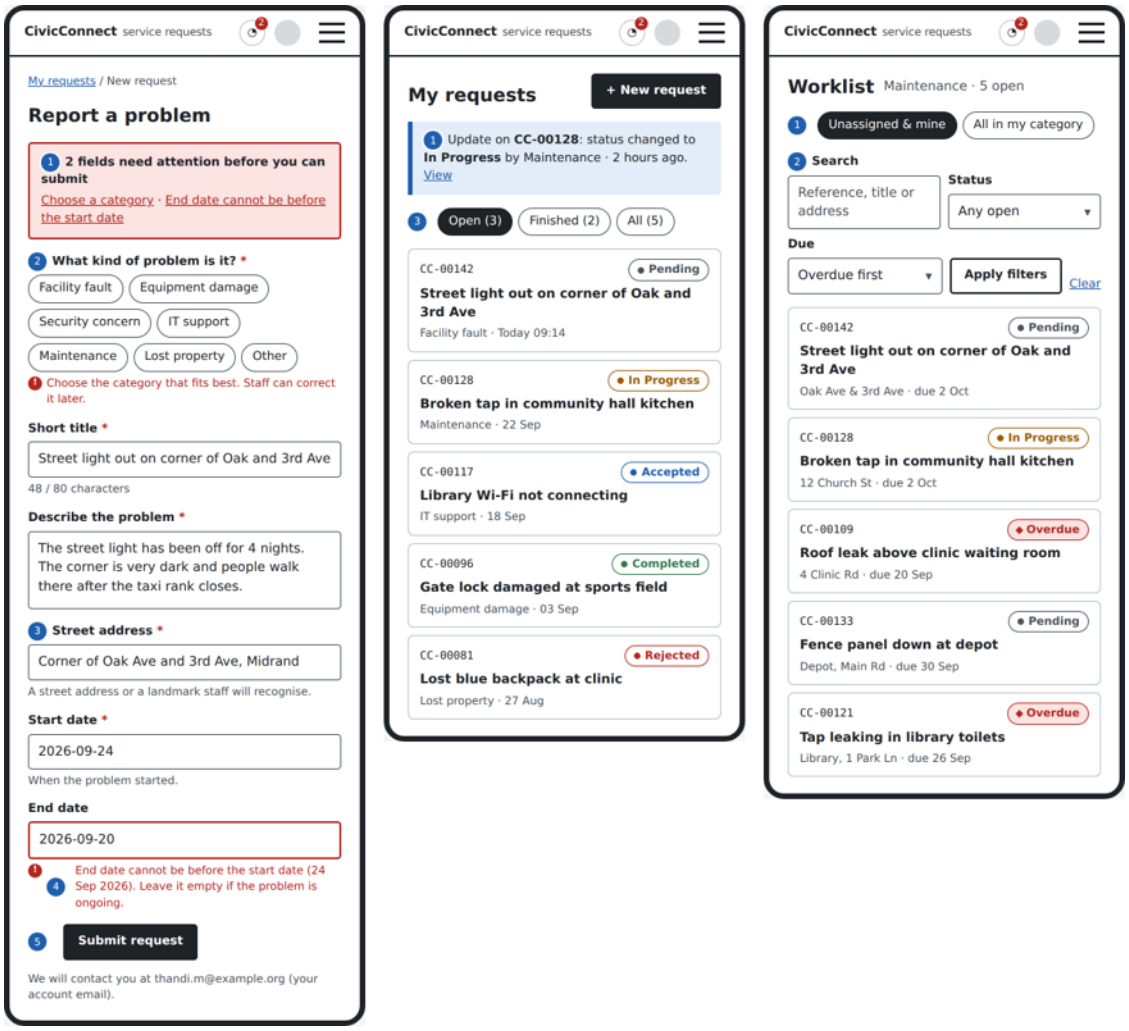


Figure 10-11. *Phone width (390 px): tables collapse to stacked cards and navigation to a menu button (REQ-032, ASR-010).*

10.3 Usability rationale

Each significant layout choice is tied to an established usability heuristic (Nielsen, 2024) and to the stakeholder need it serves.

Table 10-2. *Usability heuristics applied*

Heuristic			Design response	Screen	Serves
Visibility of system status	Status pill on every request; latest-update banner; plain-language timeline; target date shown; confirmation page states the link expiry	My requests, Request detail, Check your email	REQ-004, REQ-006, STK-001 "trust the request is not lost"		
Match with the real world	Categories use the brief's own words; "Report a problem" instead of "Create service request"; one street-address field in local terms	Submit request	REQ-003, REQ-027, CR-001		

Error prevention	Only valid transitions offered; submit disabled after the first click; staff list filtered to the request's category; end date checked against start date	Submit, Request workspace	REQ-011, REQ-013, REQ-031
Recognition rather than recall	All seven categories visible as chips; filter values as dropdowns; breadcrumbs	Submit, Worklist	REQ-003, REQ-008
Help users recover from errors	Error summary linking to each field; each message says how to fix it; "Resend confirmation email" on the check-email page and on the sign-in error	Submit, Check your email	REQ-002, D13
Consistency and standards	Same D10 status labels and pill style on every screen	All	REQ-004, REQ-019, REQ-022
Aesthetic and minimalist design	Requester form limited to the fields in REQ-001 after CR-001/CR-002; registration asks for name, email and password only	Submit, Register	PED §2.4 simplicity vs reporting depth
Flexibility and efficiency	Default staff view is "unassigned and mine"; overdue sorts to the top; the whole row is clickable; the confirmation link works on any device	Worklist, Check your email	REQ-007, REQ-008, STK-002

10.4 Accessibility rationale

The target is **WCAG 2.2 Level AA** (W3C, 2023), which gives the Section 5.9 quality boundary a testable standard. The criteria below shaped the wireframes; the right-hand column is the M3 verification.

Table 10-3. *WCAG 2.2 AA criteria applied to the design*

Success criterion		How the design meets it	Verification (M3)
1.4.1 Use of colour	Status pills combine colour, dot and word; overdue uses a border and label as well as red; errors use an icon and text	Greyscale screenshot review	
1.4.3 Contrast (minimum)	Body text and controls at least 4.5:1 against the background; checked again when the final palette is set	Automated check (axe DevTools or Lighthouse)	
1.4.10 Reflow	Single column at 320 CSS px; tables become cards; no horizontal page scroll	Browser at 320 px and 400% zoom	
2.1.1 Keyboard, 2.4.7 Focus visible	All actions are buttons or links with a visible focus outline; clickable rows are also focusable links	Keyboard-only walkthrough of each core task	

2.5.8 Target size (minimum)	Buttons and chips at least 24 × 24 CSS px (most are 38 px high)	Measure in browser dev tools
3.3.1 Error identification, 3.3.3 Error suggestion	Field message names the problem and the fix; summary at the top	Submit empty and invalid forms
3.3.2 Labels or instructions	Every field has a visible label, required marker and format hint	Automated check for unlabelled inputs
3.3.8 Accessible authentication (minimum)	Password fields allow paste and password managers (autocomplete attributes) and have a show-password toggle, so no memorised or retyped password is required; email confirmation is a single link	Register and sign in using a password manager and a screen reader
4.1.2 Name, role, value	Native HTML controls; error summary uses role="alert"; confirmations use role="status"	Screen reader spot check (NVDA)

10.5 Proposed measurable UI acceptance

REQ-027 ("intuitive to use") and REQ-035 (submit within 30 seconds) are not yet testable as written. The wireframes make them measurable, so **CR-007** proposes:

- **REQ-027:** at least 4 of 5 first-time requesters complete "report a problem" and "check its status" without help in a moderated walkthrough of the M3 build.
- **REQ-035:** the median time from opening the form to confirmed submission is 60 seconds or less for a typed description of about 20 words. Thirty seconds is not realistic once a meaningful description is typed; the walkthrough will provide evidence either way. Registration time is measured separately because it depends on email delivery.

10.6 Deliberately deferred UI decisions

- Visual style, colour palette and typography: M3, once the component approach in the React build is settled.
- Administrator screens (staff accounts, categories): M3 design, after the administrator role question in Section 8.1 is resolved.
- Photo upload on the submit form: deferred scope S-DEF-004.
- Evidence still needed: one walkthrough with a person outside the team playing a requester, to check the category chips, error messages and the confirmation email before these screens are built.

11. Initial Design Decisions

Assignment 2 Task 1 researched two CivicConnect design problems and recommended an approach for each. This section records the **final project decision** for each problem, says where it follows or departs from the A2 recommendation and why, and names the code that carries it. The research is referenced by A2 section rather than repeated. A third decision, email verification at registration, was not part of A2; it is recorded in 11.3 with its own evidence. Decisions are logged in Section 17 as **D11**, **D12** and **D13**.

Each decision names the design patterns it uses. A design pattern is a named, reusable way of arranging classes to solve a recurring problem; naming it tells a reader what to expect and what the known trade-offs are. To make the patterns easy to find, every decision below has a "Patterns used" table that gives, for each pattern, what it is in one sentence, which CivicConnect class plays which part, and the file it lives in. Section 11.5 gathers all of them into one register (Table 11-8).

Decision	Problem	Patterns used
11.1 DP-01 (D11)	Notify requesters through channels that can change	Strategy, Adapter, Dependency Injection
11.2 DP-02 (D12)	A status change with mandatory and optional side effects	Observer for optional reactions; explicit orchestration, Repository and the D10 state machine for mandatory work
11.3 (D13)	Email verification at registration	No new pattern. Reuses the Adapter from D11 and the Repository

11.1 DP-01: requester notifications and channel volatility (D11)

Problem in CivicConnect terms

When a request's status changes, the requester must get meaningful feedback (REQ-006). ASR-008 and REQ-026 ask for at least two channels and require that channels can be added or replaced without touching core logic. Scope item S-IN-003, however, commits only in-app feedback and defers email/SMS (S-DEF-002). With email verification at registration (D13), the platform must send email anyway, so an email provider becomes a required dependency rather than an optional one.

A2 evidence and alternatives

- **Candidate A, direct service with conditional selection** (A2 §2.2): simplest, but each new channel edits the same class and leaks provider details into tests.
- **Candidate B, Strategy contract with provider adapters** (A2 §2.3). Two patterns working together. *Strategy*: a narrow `NotificationChannel` interface with one implementation per channel, so the caller can use any channel without knowing which one it is. *Adapter*: one small class per external provider that translates the provider's own API into an interface CivicConnect owns. Which implementations are used is decided once, at the composition root.
- A2 recommended Candidate B, but set a condition (A2 §4): if only one channel is committed and provider volatility is not credible, keep the direct service and defer the Strategy.

Final decision

Follow the A2 recommendation (Candidate B). The A2 condition for Strategy is now met: ASR-008 names more than one channel, and the email provider is shared with registration confirmation, so it will change for reasons outside notifications (free-tier quotas, provider choice in M3). The design is:

Table 11-1a. *Patterns used in DP-01 and where each part lives*

Pattern	What it is	Who plays which part in CivicConnect	File
Strategy (behavioural)	Several interchangeable ways of doing one job sit behind one interface. The caller picks none of them and treats them all the same.	Strategy interface: NotificationChannel.send(recipient, message). Concrete strategies: EmailNotificationChannel (built), SmsNotificationChannel (deferred, S-DEF-002), FakeNotificationChannel (tests). Context (the caller): NotifyRequesterHandler, which loops over its list of channels.	modules/notification/
Adapter (structural)	A wrapper that makes a class with an unsuitable or third-party interface look like the interface the application wants.	Target interface: MailSender.sendMail(to, subject, text, html). Adapter: NodemailerMailSender. Adaptee: the Nodemailer library and the SMTP provider. Clients: EmailNotificationChannel and the User module's email verification.	infrastructure/mail/NodemailerMailSender.js
Dependency Injection	An object is handed the things it depends on instead of creating them itself.	app.js (the composition root) creates the mail sender and the channels once and passes them into the handler and services through their constructors.	app.js

Why two patterns and not one: Strategy answers "which *channel* do we notify through?" (email now, SMS later). Adapter answers "which *provider* carries the email?" (Ethereal in development, a real SMTP provider in staging). They change for different reasons, so they are kept apart. That is also why registration confirmation (D13) uses the Adapter but not the Strategy: it always sends email, so it has no channel to choose.

1. NotificationChannel interface with EmailNotificationChannel (built) and SmsNotificationChannel (deferred, S-DEF-002).
2. Both registration confirmation and the email channel depend on one MailSender interface, implemented by a Nodemailer SMTP adapter (Figure 11-1). Changing provider changes one adapter and its settings.
3. In-app feedback is **not** a channel: the requester's timeline reads the action table directly, so it cannot fail separately from the status change (S-IN-003 is met even if email fails).
4. Whether status emails are actually sent depends on **CR-005**. If it is rejected, the handler is registered with no channels and in-app feedback remains; no code changes.

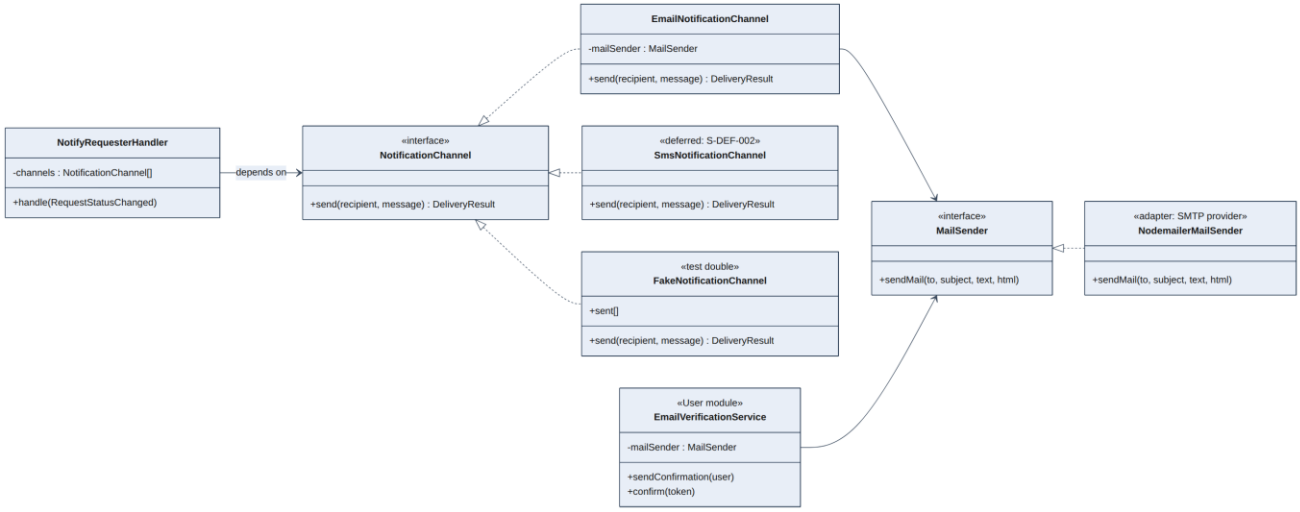


Figure 11-1. *DP-01 class design. One mail adapter serves registration confirmation and notifications. SMS is shown only to mark where a future channel plugs in.*

Table 11-1. *DP-01 expected benefit, cost and verification*

Aspect	Detail
Expected benefit	From Strategy: the Request module has no knowledge of how messages are delivered (Dependency Inversion), and adding SMS later is one new class plus one registration line (Open-Closed). From Adapter: changing email provider changes one file and its settings. From both: email failures cannot roll back core writes (Section 12.1, R07).
Complexity introduced	Two small interfaces (NotificationChannel for the Strategy, MailSender for the Adapter), one result type, and registration in the composition root. Every channel returns the same DeliveryResult meaning (Liskov Substitution).
Misuse to avoid	Returning provider objects from send(); adding templating, scheduling or receipts to the interface (Interface Segregation).
Affected code	Strategy: program/backend/src/modules/notification/ (NotificationChannel, EmailNotificationChannel, FakeNotificationChannel, NotifyRequesterHandler). Adapter: program/backend/src/infrastructure/mail/NodemailerMailSender.js. Dependency Injection: composition root program/backend/src/app.js.
Acceptance	No module's domain/ folder or service.js imports Nodemailer or an SMTP setting; only infrastructure/mail/ does.
Tests (planned)	Unit: handler sends one message per status change using the fake channel; a failing channel is logged and does not throw. Integration: email reaches the Ethereal test inbox.
Traceability	REQ-006, REQ-026 → ASR-008 → D11 → Notification module → tests above. Scope conflict → CR-005.

11.2 DP-02: status changes and their side effects (D12)

Problem in CivicConnect terms

A status change (REQ-013, REQ-016, REQ-017) must validate the transition, update the request and write an attributable action row (REQ-021, ASR-009). It should also notify the requester (REQ-006) and, later, may trigger

SLA escalation (S-DEF-001). If the service calls every reaction directly it grows with each new one; if everything becomes an event, the audit guarantee becomes hard to see and test.

A2 evidence and alternatives

- **Candidate A, explicit application-service orchestration** (A2 §3.2): visible order and one transaction, but fan-out coupling as reactions grow.
- **Candidate B, in-process Observer through domain events** (A2 §3.3). In the Observer pattern a publisher announces that something happened and any number of subscribers react, without the publisher knowing who they are. Here the announcement is a *domain event*, a small data object that records a business fact such as "request 42 moved from Pending to Accepted". The publisher stays independent of its subscribers, but ordering, failure handling and transaction timing must be defined.
- A2 recommended a hybrid: orchestrate the mandatory, transactional work explicitly; publish one in-process event for optional reactions (A2 §3.5).

Final decision

Follow the A2 recommendation: a hybrid. The decision deliberately uses the Observer pattern for only half of the work.

Mandatory work is explicit orchestration, not a pattern-driven event. The status service locks the service_request row, asks the ServiceRequest domain object to apply the D10 transition (the state machine in Section 7.5), and writes the request update and the action row through the repositories in **one transaction**. A reader can see the whole audit guarantee in one method.

Optional reactions use the Observer pattern. Only after COMMIT does the service publish a RequestStatusChanged event to an in-process EventDispatcher. The dispatcher calls every handler registered for that event. Handlers are not awaited by the HTTP response, so a slow SMTP server cannot push a status update past the 10-second boundary (REQ-025, ASR-005). Assignment, category correction and completion follow the same orchestrate-then-publish structure.

CivicConnect meets the A2 condition for adding events: two reactions that change for different reasons (notification now, escalation later) and one that must never be optional (the action row).

Table 11-2a. *Patterns used in DP-02 and where each part lives*

Pattern	What it is	Who plays which part in CivicConnect	File
Observer (behavioural), in-process	A subject keeps a list of observers and notifies all of them when something happens, without knowing what they do.	Subject: EventDispatcher (register(eventName, handler), publish(event)). Event: RequestStatusChanged (request ID, previous status, new status, staff ID, comment, timestamp). Publisher: changeStatus in the Request service, after COMMIT. Observer: NotifyRequesterHandler.handle(event). A future EscalationHandler (S-DEF-001) would be a second observer. Subscription: one register line per handler in app.js.	shared/events/EventDispatcher.js modules/notification/NotifyRequesterHandler.js modules/request/service.js
Repository	An object that hides SQL behind methods	The request repository (lock, update status) and the action repository (insert audit row), both called inside the service's transaction.	modules/request/repository.js

	named after what the application needs, so services never build queries.		
State machine (not the GoF State pattern)	A fixed table of which moves between states are allowed.	ServiceRequest.changeStatus(toStatus); see Section 7.5 for why a table was chosen over State classes.	modules/request/domain/ServiceRequest.js

How the two decisions connect: D12 decides **when** the requester is notified (after commit, by an observer). D11 decides **how** (through whichever channels are registered). NotifyRequesterHandler is the one class where they meet: it is the observer in D12 and the Strategy context in D11.

Table 11-2. *Classification of every reaction to a status change*

Reaction			Classification	Where it runs	If it fails
Validate transition (D10)	Mandatory	Domain object, before any write	Nothing written; 409 with the §12.4 error envelope		
Update service_request status and owner	Mandatory, transactional	Status service, in the transaction	ROLLBACK; nothing written		
Insert action row (StaffID, ActionType, PreviousStatus, NewStatus, Comment, Timestamp)	Mandatory, transactional	Status service, same transaction	ROLLBACK; status never changes without its audit row		
logging row	Automatic	Database trigger (Section 8)	Diagnostic only; no effect on the command		
Requester notification	Optional	NotifyRequesterHandler, after commit, not awaited	Logged with request ID; status change stands; timeline still shows it		
Oversight counts	None needed	SQL at read time (Admin module)	Not applicable		
SLA escalation	Optional, deferred (S-DEF-001)	Future handler	Not built in M2		

Rules the team commits to

- Handlers run after commit, in registration order; a handler error is caught and logged and never rolls back the command.
- Handlers must not change the same request again (no event chains). The event carries data, not a repository.
- The dispatcher is a small in-house class of roughly 20 lines: a map from event name to a list of handlers. It is the Observer pattern inside one process, not a message queue. There is no message broker, retry queue or outbox because there is no distributed boundary (A2 §3.3, D06, D09). The accepted consequence is that an email whose handler fails is logged and not retried; the requester still sees the change in the in-app timeline.

Table 11-3. *DP-02 expected benefit, cost and verification*

Aspect	Detail
Expected benefit	The audit guarantee is visible in one method and one transaction (ASR-007, ASR-009). New optional reactions are added as handlers without editing the transition use case (ASR-008).
Complexity introduced	The usual cost of Observer: the publisher no longer shows who reacts, so tracing a status change means checking the handler registrations in `app.js`. To offset this, the dispatcher logs each handler it calls. Added parts are one dispatcher, one event type and handler registration.
Affected code	Orchestration and publishing: program/backend/src/modules/request/service.js (changeStatus). State machine: modules/request/domain/ServiceRequest.js. Observer subject: shared/events/EventDispatcher.js. Observer: modules/notification/NotifyRequesterHandler.js. Subscription: app.js.
Acceptance	ServiceRequest has no infrastructure imports. A forced action-insert failure leaves the status unchanged. A second handler can be registered without editing the request service.
Tests (planned)	Unit: every allowed and forbidden transition. Integration: status and action row always agree; forced failure rolls back; the dispatcher calls exactly the registered handlers.
Traceability	REQ-013, REQ-016, REQ-017, REQ-021, REQ-028 → ASR-007, ASR-009 → D10, D12 → code above → tests above.

11.3 Additional decision: email verification at registration (D13)

This decision was **not** part of the A2 research. In M2 the lecturer confirmed that new accounts must prove they own their email address. Users still register and sign in with **email and password** as baselined (REQ-023, REQ-030, ASR-003, bcrypt in §9). The only addition is a one-time confirmation email at registration: an account cannot sign in until its email address is confirmed. Because this adds acceptance criteria to REQ-023 and a small data change, it is raised as **CR-004** (Section 18). Supporting evidence comes from the OWASP Cheat Sheet Series and NIST SP 800-63B-4.

Why verify the email

- The email address is the account's identity and, from CR-005, the channel for status notifications. A mistyped or someone else's address would send request updates to the wrong person.
- It stops accounts being created in another person's name and limits throw-away or automated sign-ups, which protects the staff worklist from junk requests.
- It gives a verified address for a password-reset flow later (M3).

Options considered

Table 11-4. *Account verification options*

Option		For	Against
A. No verification (current baseline)	Nothing to build; no email dependency	Any address can be used, including someone else's; notifications may go to the wrong person	
B. Confirmation link emailed at registration (selected)	One email per account; familiar pattern; password sign-in unchanged; NIST treats email	Depends on email delivery at sign-up; unverified accounts need a resend and clean-up rule	

	confirmation codes as outside its ban on email authentication	
C. Email code at every sign-in (email as second factor)	Adds a second step to every login	NIST SP 800-63B-4 §3.1.3.1: email "SHALL NOT be used for out-of-band authentication"; one extra email per sign-in; slower for staff who sign in many times a day
D. Authenticator-app or passkey second factor	Strongest protection	More build and test effort than M2/M3 allow; candidate for staff and management accounts later

Final decision

Option B. Registration takes name, email and password (hashed with bcrypt as in §9). The account is saved with email_verified = no and the server emails a single-use confirmation link. Opening the link sets email_verified = yes and clears the token; from then on the user signs in with email and password as normal and receives the signed JWT from §12. A sign-in attempt on an unconfirmed account is refused with a clear message and a "resend confirmation email" button. Figure 11-2 shows the flow.

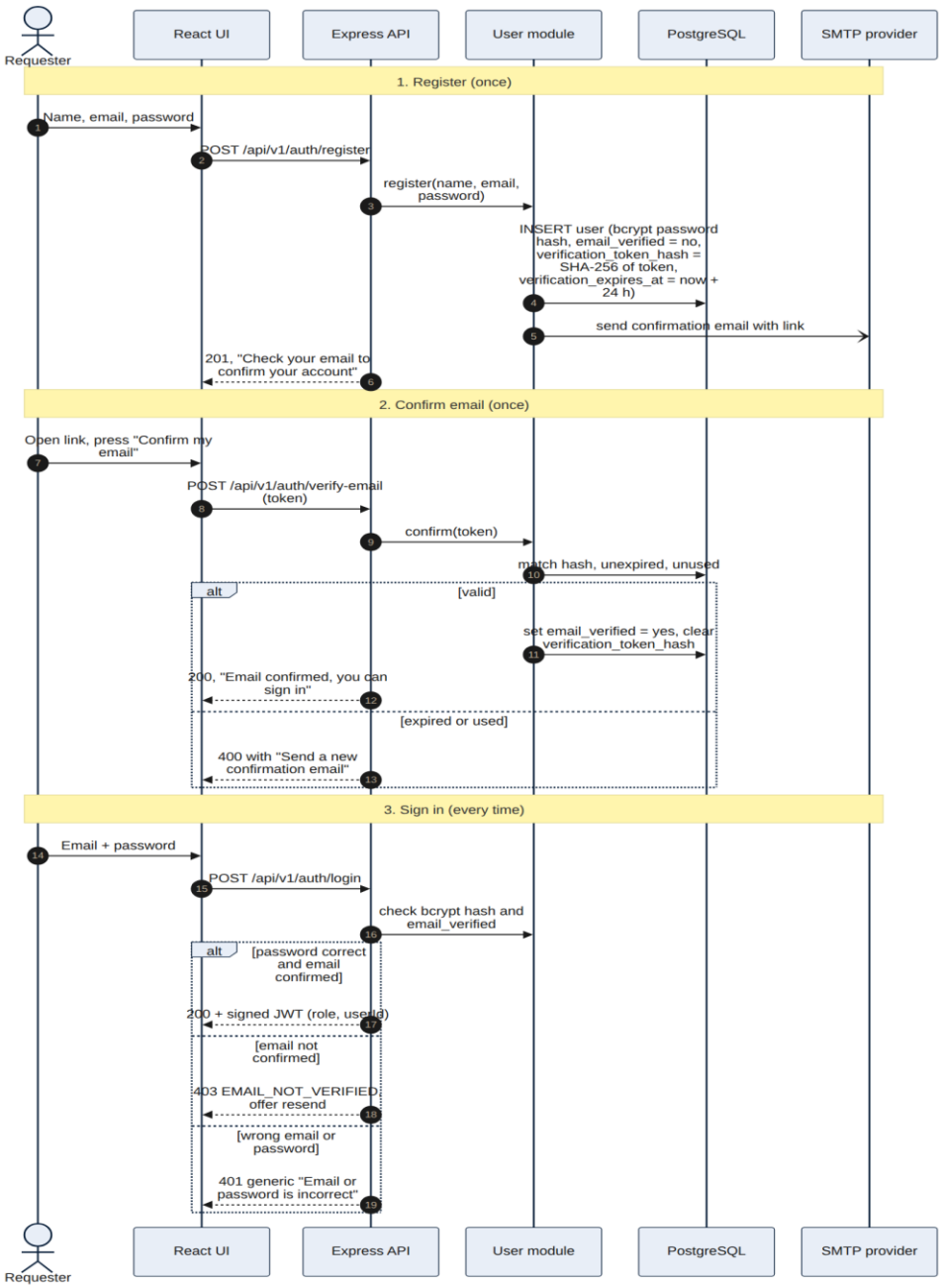


Figure 11-2. *Registration with email confirmation, followed by normal email + password sign-in.*

Table 11-5. *Security controls for the confirmation token*

Control		Design	Source
Unguessable token	32 random bytes from Node's crypto.randomBytes	OWASP Forgot Password Cheat Sheet: tokens "generated using a cryptographically secure random number generator"	
Nothing usable stored	Only a SHA-256 hash of the token is stored, in user.verification_token_hash, with user.verification_expires_at. No separate table is needed	OWASP: tokens "stored in a secure manner"	

Single use, limited life	Valid for 24 hours; the hash is cleared on first use, so the link works once; "resend" overwrites the hash, which invalidates the old link	OWASP: tokens "invalidated after they have been used" and should expire
Rate limits	Registration and resend requests limited per email and per IP	OWASP: rate limiting
No account enumeration	Registering an address that already exists shows the same "check your email" page; the existing owner gets a "someone tried to register with your address" email instead	OWASP Authentication Cheat Sheet: "A link to activate your account has been emailed to the address provided"
Mail scanners	The link opens a page with a "Confirm my email" button that POSTs the token, so automatic link scanners cannot use it up; Referrer-Policy: no-referrer	OWASP Forgot Password Cheat Sheet, referrer leakage
Passwords	Unchanged: bcrypt hashing (ASR-003, REQ-030, D05)	Section 9

Trade-offs the team accepts

- **Email delivery matters at sign-up.** If the confirmation email does not arrive, the new user cannot start. The resend button and provider checks in M3 mitigate this (R14, R16).
- **Unconfirmed accounts build up.** Accounts not confirmed within 7 days are deleted by a scheduled clean-up (R15).
- **This is verification, not two-factor sign-in.** Every later sign-in is email and password only. A true second factor for staff and management is a candidate for M3 (option D).

Table 11-6. *D13 affected code and verification*

Aspect	Detail
Affected code	program/backend/src/modules/user/service.js (register, verifyEmail, login), modules/user/repository.js (three new user attributes), infrastructure/mail/NodemailerMailSender.js; routes POST /api/v1/auth/register, POST /api/v1/auth/verify-email, POST /api/v1/auth/resend-verification; POST /api/v1/auth/login returns 403 EMAIL_NOT_VERIFIED for unconfirmed accounts; frontend pages Register, CheckEmail, SignIn in program/frontend/src/pages/auth/.
Tests (planned)	Unconfirmed account cannot sign in; token works once; expired token rejected; resend invalidates the old token; duplicate registration shows the same page; stored value is a hash, never the raw token; password still stored as a bcrypt hash.
Traceability	REQ-023, REQ-033 → ASR-002, ASR-003 → D13, CR-004 → User module → tests above. Risks R14, R15, R16.
Patterns reused	D13 introduces no new pattern. The confirmation email is sent through the same `MailSender` Adapter as status emails (D11), which is why Figure 11-1 shows two clients of one adapter. Token and verification fields are read and written through the user Repository. The Strategy is not involved because confirmation is always sent by email.

11.4 Research-to-decision summary

Table 11-7. *How evidence informed the M2 design decisions*

Problem			Evidence and recommendation	Decision	Same or different, and why
DP-01 Notification channels	A2 Task 1 §2.5, §4: Strategy + provider adapters; keep a direct service if only one channel	D11: Strategy (NotificationChannel) for channels plus one shared Adapter (MailSender, implemented by NodemailerMailSender) for the provider; SMS deferred.	Same as A2. The A2 condition for Strategy is met (ASR-008, shared email provider).		
DP-02 Status side effects	A2 Task 1 §3.5: explicit orchestration for mandatory work + in-process event for optional reactions	D12: as recommended. Explicit orchestration in one transaction for the mandatory writes; Observer (EventDispatcher publishing RequestStatusChanged) for optional reactions, with handlers not awaited.	Same. Two independently changing reactions plus one mandatory audit write.		
Account verification	Not in A2. Lecturer requirement; OWASP Authentication and Forgot Password cheat sheets; NIST SP 800-63B-4	D13: confirmation email at registration, password sign-in unchanged	New decision; risks R14–R16.		
Submission integrity (supporting)	A2 Task 2 §5.6: UI/API checks + database constraints; no cache	D08 and §12.3 (owned by teammates)	Same. The team still has to choose between idempotency keys (§12.3) and pre-save checks (D08).		
Client–API boundary (supporting)	A2 Task 3 §6.3: synchronous REST	D06 (§12)	Same.		

12. API / Integration Decisions

12.1 Interaction Boundary and Integration Mechanism Selection

- **Established Interaction Boundary**: The main network boundary connects the React client application to the Node.js/Express backend API across four core logical modules defined in Section 8.1 (User module, request module, Staff module and Admin module and Notification module)
- **Selected Mechanism (informed by Assignment 2 research):** A Synchronous HTTP/REST API exchanging JSON payloads over the internal client-server boundary is being adopted by CivicConnect. REST APIs were evaluated against asynchronous message queues (Task 3 of assignment 2). While asynchronous queues decouple producers from consumers, peer-reviewed microservice research shows they introduce eventual consistency and complex distributed tracing overhead (Nookala, 2023). Conversely, synchronous request-response interactions provide a deterministic execution model and immediate transactional confirmation required by interactive user interfaces, provided downstream latency is controlled (De Iasio and Zimeo, 2021). Due to the fact that ASR-005 (REQ-035) and ASR-007(REQ-028) require immediate confirmation that a submitted service request is persisted without corruption, synchronous REST is the most proportionate mechanism for a team of our size.
- **External Notification Boundary (ASR-008 / REQ-026):** While core client-server operations are synchronous, external notification dispatch (email on status update per D11 / D12) is isolated behind the application-owned NotificationChannel Strategy and MailSender Adapter contract (informed by Assignment 2 Task 1 research on pattern selection; Bijlsma et al., 2022; Naghdipour, Hasheminejad and Barmaki, 2023) and triggered *after* transaction COMMIT so third-party SMTP delays or failures (R07) never block the HTTP response or roll back core database writes..

12.2 Initial REST API Endpoint Contract Baseline

Endpoint & Method	Module & Role Boundary	Traced Requirements	Request Payload / Query Contract	Success Response & Persistence Side Effect
POST /api/v1/auth/register POST /api/v1/auth/login	User module (Public → Authenticated)	REQ-023, REQ-029, REQ-030, REQ-033, CR-002, CR-004	{ name, email, password } (Baseline Register/Login) or { name, email } + { token / code } (D13 email verification)	201 Created / 200 OK with signed JWT (userId, user_type, category). Hashes secret (ASR-003) before persisting to user (user_type = 'requester').
POST /api/v1/requests	Request module (user_type = 'requester' only)	REQ-001, REQ-002, REQ-003, REQ-028, REQ-031, REQ-035, CR-001, CR-002	Header: Authorization: Bearer <JWT>. Body: { title, description, street_address, start_date, end_date, category }.	201 Created returning persisted service_request_id, created_at, and initial status: "Pending" (REQ-004, D10) inside an atomic PostgreSQL transaction.
GET /api/v1/requests/my	Request module (user_type = 'requester' only)	REQ-004, REQ-005, REQ-006, REQ-029	Header: Authorization: Bearer <JWT>. Scoped strictly by authenticated requester_id = user.user_id	200 OK returning requester's own service_request records and associated Action timeline history (new_status, date, comment, action_type).
GET /api/v1/staff/requests GET /api/v1/staff/members	Staff module (user_type = 'staff' only)	REQ-007, REQ-008, REQ-009, REQ-010, REQ-029	Query params: `?category=&status=&search=&sort=asc	descscoped to staff member'suser.category`.

PATCH /api/v1/requests/:id/assign PATCH /api/v1/requests/:id/category	Staff module (user_type = 'staff' only)	REQ-011, REQ-012, REQ-024	{ staff_id } (Assign/Accept responsibility) or { category } (Triage correction)	200 OK. Verifies active staff_id (user_type = 'staff', REQ-011), updates service_request, and inserts an action audit row in one transaction.
PATCH /api/v1/requests/:id/status	Request module (staff or requester per D10 rules)	REQ-013–REQ-017, REQ-021, REQ-026, REQ-028, CR-003	{ new_status, comment, action_type }	200 OK. Atomically updates service_request.status across the 7 D10 states (Pending, Accepted, In Progress, Completed, Closed, Rejected, Cancelled) and inserts action record; dispatches RequestStatusChanged event after commit (D12).
GET /api/v1/admin/overview GET /api/v1/admin/audit	Admin module (user_type = 'management' only)	REQ-018–REQ-022, REQ-029	Query params: ?status=&category=&start_date=&end_date=.	200 OK returning aggregated SQL counts (open, closed, resolved, overdue derived per D10) and chronological action audit logs (ASR-009).

Note: In alignment with approved Change Requests CR-001 and CR-002 (Section 18) and the consolidated Section 8.2 schema (D08), country, province, city, and phone number/contact have been removed in favor of street_address on service_request and email on the unified user table.

12.3 Layered Validation, Integrity and Caching Exclusion

Following the research in assignment 2 CivicConnect enforces a hybrid responsibility allocation across the UI, Application (Express) and the Database(PostgreSQL) layers:

1. **UI & Express Application Layer:** Before opening a database transaction, Express middleware verifies the JWT signature, enforces role and jurisdiction permissions via user_type and user.category (ASR-002 / CON-SEC-03), checks that mandatory fields (title, description, street_address, category, start_date, end_date) are non-empty, applies regex sanitization against disallowed characters, validates state transitions against the ServiceRequest domain rules (D10), and confirms start_date <= end_date.
2. **Database Persistence Layer (Authoritative ACID Enforcement):** Relying solely on application-layer validation is insufficient when handling concurrent requests across multiple processes (Hellerstein, Stonebraker and Hamilton, 2007). Therefore, PostgreSQL enforces NOT NULL, CHECK (start_date <= end_date), valid status and user_type CHECK constraints, UNIQUE (email), PRIMARY KEY, and FOREIGN KEY constraints (requester_id and staff_id referencing user.user_id; service_request_id referencing service_request.service_request_id) inside explicit BEGIN ... COMMIT/ROLLBACK transactions (ASR-007, REQ-028, R06), backed by automated AFTER triggers writing diagnostic snapshots to the logging table.
3. **Duplicate Prevention & Caching Exclusion (REQ-031 / NFR-007**, D08**):** In accordance with Decision D08, Milestone 2 prevents duplicate submissions and concurrent staff claim collisions using transactional pre-save checks and row-level locking (SELECT ... FOR UPDATE, Figure 7-5) rather than client-generated idempotency keys (which are deferred for reassessment in M3). In accordance with Bailis et al. (2012), in-memory read caching is **deliberately excluded** from the request submission and status-transition validation path to eliminate the probabilistic window of data staleness that could permit duplicate submissions or double-claims by staff.

12.4 Error Semantics, Security Boundary and Versioning

- **Standardized Error Responses:** All failed API calls return a uniform JSON error envelope { "error": { "code": "STRING", "message": "Human-readable message", "details": [] } } without exposing internal SQL queries, logging table internals, or Node.js stack traces (CON-SEC-04):
    - 400 Bad Request: Validation failure (empty fields, end_date < start_date, invalid category).
    - 401 Unauthorized / 403 Forbidden: Unauthenticated caller or user_type role violation (e.g., a requester attempting to call /api/v1/admin/audit or update a staff-only status, violating REQ-029 / CON-SEC-03).
    - 409 Conflict: Duplicate submission detected by pre-save check (REQ-031, D08) or invalid lifecycle state transition rejected by ServiceRequest domain rules (D10).
    - 500 Internal Server Error: Unhandled persistence failure; executes automatic transaction ROLLBACK (REQ-028, R06).
- **Version Control:** All endpoints are namespaced under /api/v1/ so future contract modifications require explicit version increments or Change Requests (CON-GOV-02) rather than silently breaking frontend integration.

13. Deployment Compatibility

13.1 Target Deployment Direction and Environment Strategy

In accordance with CON-TEC-03, CivicConnect is architected for a three-tier cloud compatible deployment topology while remaining completely operable on local developer machines at no cost (ASR-001).

1. **Development Environment (Active State):** Local developer workstations running the Vite React dev server (:5173), Node.js/Express API (:3000), a fake SMTP test inbox (Ethereal), and a local development PostgreSQL 16 instance (civicconnect_dev) populated with non-sensitive seed data (CON-TEC-03, CON-SEC-04).
2. **Test/ Continuous Integration Environment**: Ephemeral GitHub Actions runner executing npm ci, build verification, linting, and automated unit tests against an ephemeral PostgreSQL 16 service container and FakeNotificationChannel on every Pull Request targeting main (informed by Assignment 2 Task 4 research; Santos et al., 2022; Saidani, Ouni and Mkaouer, 2022).
3. **Anticipated Staging/ Production Direction**: A static web host/CDN serving the compiled React production bundle (npm run build), a stateless Linux PaaS container running the Express API, a free-tier SMTP provider, and a managed PostgreSQL instance with automated standby failover (ASR-004).

13.2 State, Persistence and Networking Implications

- **Stateless Application Tier (ASR-006):** Because authentication is carried via signed JWTs (ASR-002) and all domain and audit state resides in PostgreSQL (user, service_request, action, logging, and login_token under CR-004), the Node.js/Express backend stores zero in-memory session state. This avoids sticky-session routing issues and allows the API tier to restart or scale horizontally to handle 210 concurrent users (REQ-037) without dropping active sessions.
- **Database Connection Pooling & Failover (ASR-004, ASR-007):** Express connects to PostgreSQL using a shared connection pool (pg.Pool) configured with explicit connection timeouts and SSL support, compatible with managed cloud databases and the primary-replica failover architecture defined in Section 8.5.1 (REQ-034, REQ-036).
- **Network Boundary & CORS Security:** Cross-Origin Resource Sharing (cors) middleware restricts API access strictly to the configured CORS_ORIGIN frontend domain, and all external traffic in staging/production is assumed to terminate over HTTPS/TLS (CON-SEC-01, ASR-003).

13.3 Configuration Parity and Secrets Control

Research demonstrates that configuration drift between development and production environments is a primary cause of deployment failures (Nazário, Bonifácio and Pinto, 2025). To satisfy CON-SEC-02, CON-TEC-03, and CON-TEC-04:

- **Strict Secret Externalization**: Database connection strings (DATABASE_URL), JWT signing keys (JWT_SECRET), server ports (PORT), and external notification credentials (SMTP_PASS / API keys) are strictly excluded from source code and injected at runtime via environment variables.
- **Repository Protection:** .env files are blocked via .gitignore (CON-SEC-02, Section 18.1), while a sanitized .env.example file is committed to document required configuration keys for teammates and assessors.
- **Deterministic Dependency Resolution:** All environments install dependencies exclusively via npm ci against the committed package-lock.json file to prevent version drift (Gamage et al., 2026).

Deployment Risks and Deliberately Deferred Decisions

- **Identified Compatibility & Operational Risks**:
    - ***Free-Tier Quota & Cold-Start Constraints (CON-CST-01, ASR-001, ASR-005):*** As noted in Section 15 (Forward Engineering Considerations), relying on free-tier cloud PaaS/DB providers introduces

risks of vendor plan removal (Heroku, n.d.), connection limits under 210 concurrent users (R04), or container sleep/cold-start delays exceeding the 10-second processing boundary (REQ-025, R03).

- ***Over-Fetching & Unindexed Query Degradation**:* Microservice and database decomposition studies highlight that poorly bounded queries across network boundaries degrade rapidly as tables grow (Çetin and Eldenk, 2025), reinforcing the Section 8.5.2 requirement to index RequesterID, StaffID, Status, and Action_Date before production load testing.

- **Deliberately Deferred Deployment Decisions (Reserved for Milestone 3):** In strict compliance with Milestone 2 Section 5.8 and Section 11 boundaries, the following decisions are **deliberately deferred to Milestone 3**:
  - Final selection of the commercial/free-tier cloud hosting provider for production deployment (requires M3 operational cost and quota verification per CON-CST-03)
  - Automated live standby database failover execution and automated backup restoration testing (REQ-036, R05, Section 8.5.4).
  - Production observability pipelines, centralized log aggregation, and automated alerting thresholds (CON-TEC-04).

14. Requirements Traceability Matrix

Req ID	Source	Priority	Acceptance criteria	ASR link	Module · data · technology	Implementation	Test case(s)	Expected result	Execution evidence	Defect / status	ADR / CR / risk
REQ-001	**STK-001, FR-001** The system should allow the requester to enter and submit service request information. *Project Master Brief, Min Business Capabilities, p.7*	Medium	Meaningful messages guide user input. Able to enter title, description, and street address. Validation check ensures no empty fields. Information must be submitted to the database.	ASR-005, ASR-007	Request · service_request · React + Express + PostgreSQL	SubmitRequest.jsx	TC-001, TC-016	Title, description, street address left blank → Each detail will result a proper error recovery message Register with valid details → receive 6-digit code → confirm → login → submit request → request appears in "My Requests" with status Pending	Console output showing PASS frontend/tests/unit/empty_fields.test.js TC-016.test.js todo entry; current scope-deferral record (PASS)	Approved	D06, D08, CR-001, CR-002, R06
REQ-002	**STK-003, FR-002** The system should allow the requester to select a predefined category for a service request. *Project Master Brief, Min Business Capabilities, p.7*	High	Predefined categories can be selected via dropdown. Validation check ensures that a category was selected.	ASR-007	Request · service_request.category · React + Express	SubmitRequest.jsx	TC-004, TC-012, TC-016	Title: "Malfunctional robot", Description: "Our friendly cafeteria robot started attacking our students with coffee mugs!", Street Address: "10 IT Street", Empty or unknown category → clear error, no request created Register with valid details → receive 6-digit code → confirm → login → submit request → request appears in "My Requests" with status Pending	Console output showing PASS backend/tests/unselected_category.test.js `milestone3-results.json`; `TC-012.test.js`; (FAIL) TC-016.test.js todo entry; current scope-deferral record (PASS)	Approved	D06, R06
REQ-003	**STK-001, FR-003** The system should allow the requester to see the status of a service request. *Project Master Brief, Min Business Capabilities, p.7*	Medium	Message "pending" displays if request not assigned. Message "accepted" displays if request was assigned. Message "in progress" displays if request is underway. Message "completed" displays if request is complete. Message "rejected" displays if request was rejected. Message "updated" displays if request was updated. Message "cancelled" displays if request was cancelled.	ASR-002	Request · service_request.status · React + Express	RequestDetail.jsx	TC-003, TC-016	Pending → Completed (skip) → throws InvalidTransitionError (mapped to 409) Register with valid details → receive 6-digit code → confirm → login → submit request → request appears in "My Requests" with status Pending	Console output showing PASS backend/tests/unit/TC-003.test.js and the trapped [Status Update Error]. TC-016.test.js todo entry; current scope-deferral record (PASS)	Approved	D06, D10, CR-003, R11

REQ-004	**STK-001, FR-004** The system should allow the requester to view the history of her/his service requests. *Project Master Brief, Min Business Capabilities, p.7*	Medium	All submitted service requests are visible in history.	ASR-002	Request · service_request · Express + PostgreSQL	MyRequests.jsx	TC-016	Register with valid details → receive 6-digit code → confirm → login → submit request → request appears in "My Requests" with status Pending	TC-016.test.js todo entry; current scope-deferral record (PASS)	Approved	D06
REQ-005	**STK-001, FR-005** The system should allow the requester to view meaningful feedback for a request when it is accepted, rejected, updated, or completed. *Project Master Brief, Min Business Capabilities, p.7*	Medium	Meaningful feedback is provided for a service request whenever it is accepted, rejected, updated, or completed.	ASR-008	Request / Notification · action · Express + Nodemailer	RequestDetail.jsx	TBD	TBD	—	No verification yet	D11, CR-005, R07
REQ-006	**STK-002, FR-006** The system should allow the staff to view a list of service requests according to their category. *Project Master Brief, Min Business Capabilities, p.7*	High	Visible service requests appear on the list subject to staff category.	ASR-002	Staff · service_request · Express + PostgreSQL	Worklist.jsx	TC-017	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	D06
REQ-007	**STK-002, FR-007** The system should allow the staff to search, filter, and sort service request using criteria information. *Project Master Brief, Min Business Capabilities, p.7*	High	Service requests can be searched, filtered, and sorted according to the provided criteria.	—	Staff · service_request · Express + PostgreSQL	Worklist.jsx	TC-017	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	—
REQ-008	**STK-002, FR-008** The system should allow the staff to view the full details of a service request. *Project Master Brief, Min Business Capabilities, p.7*	High	Full details of request displays whenever pressed.	ASR-002	Staff · service_request + action · Express	RequestWorkspace.jsx	TC-017	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	D06
REQ-009	**STK-002, FR-009**	High	Display authorized staff for	ASR-002	Staff · service_request.staff_id,	RequestWorkspace.jsx	TC-017	Staff login → worklist shows only own-category requests →	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	D06

	The system should allow the staff to assign service request to another staff or accept it themselves. *Project Master Brief, Min Business Capabilities, p.7*		service request. Staff can assign service request to authorized staff. Staff can take ownership of service request.		user.user_type = 'staff' · Express + PostgreSQL			accept or assign → update status through allowed transitions → resolution comment visible to requester			
REQ-010	**STK-002, FR-010** The system should allow the staff to select a request status option. *Project Master Brief, Min Business Capabilities, p.7*	High	Staff can update request statuses using the following vocabulary: pending, in progress, accepted, completed, and rejected.	—	Request / Staff · service_request.status · Express	RequestWorkspace.jsx	TC-017	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	D10, CR-003, R11
REQ-011	**STK-002, FR-011** The system should allow the staff to enter and submit service request feedback. *Project Master Brief, Min Business Capabilities, p.7*	High	Able to submit actions, comments, date, and resolution information. Validation check ensures no empty fields. Submitted feedback must be displayed on requester's side.	ASR-009	Staff · action · Express + PostgreSQL	RequestWorkspace.jsx	TC-017	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	D06
REQ-012	**STK-002, FR-012** The system should allow the staff to resolve or close a service request. *Project Master Brief, Min Business Capabilities, p.7*	High	Staff can resolve a service request and status is changed to "completed". Staff can close a service request and status is changed to "rejected".	—	Staff · service_request.status · Express	RequestWorkspace.jsx	TC-017	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	D10, CR-003
REQ-013	**STK-005, FR-013** The system should allow management to view overall statistics of service requests. *Project Master Brief, Min Business Capabilities, p.7*	High	Visible display of total summary statistics of service requests.	ASR-009	Admin · read-only aggregate · Express + PostgreSQL	Oversight.jsx	TBD	TBD	—	No verification yet	D12, R01
REQ-014	**STK-005, FR-014** The system should allow management to view service requests with work status. *Project Master Brief, Min Business*	High	Displays a visible list of all service requests. Message "open" displays if request is not taken yet. Message "closed" displays if	ASR-009	Admin · service_request.status · Express	AllRequests.jsx	TBD	TBD	—	No verification yet	D10, R11

	Capabilities, p.7		request was rejected. Message "resolved" displays if request is completed. Message "overdue" displays if request is past its due date.								
REQ-015	**STK-005, FR-015** The system should allow management to filter and sort request information by status, category, start date, and end date. *Project Master Brief, Min Business Capabilities, p.7*	High	Display filtered and sorted service request when criteria information such as work status, category, start/end dates are provided.	—	Admin · service_request · Express + PostgreSQL	AllRequests.jsx	TBD	TBD	—	No verification yet	—
REQ-016	**STK-003, FR-016** The system should allow management to view audit of staff actions. *Project Master Brief, Min Business Capabilities, p.7*	High	Display visible audit of staff with the actions for a service request. Visible audit of staff updating status requests.	ASR-009	Admin · action · Express + PostgreSQL	AuditTrail.jsx	TC-017	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Manual execution by Benno Weideman on the 16th of October 2026.	Approved	D12
REQ-017	**STK-006, FR-017** The system should allow users to register an account by entering details and submit.	High	Users can enter their name, email, and password to sign an account. New account is created when registered. Passwords between 8 and 64 characters are accepted (Cheat Sheets Series Team, 2026). Account is only created when the 6-digit token verification is confirmed via email by the user; otherwise, registration is at halt.	ASR-003, ASR-008,	User · user · Express + PostgreSQL + email confirmation	AuthPage.jsx	TC-005, TC-006, TC-015, TC-016	Password "abc1234" → rejected; valid registration stores only hashes Valid name/email/password → 201; token hashed; login returns 403 EMAIL_NOT_VERIFIED until code confirmed Code < 6 or > 6 digits rejected; used code rejected; correct code sets email_verified = true Register with valid details → receive 6-digit code → confirm → login → submit request → request appears in "My Requests" with status Pending	Console output showing PASS backend/tests/unit/TC-005.test.js Console output showing PASS backend/tests/integration/TC-006.test.js (5.632 s) `milestone3-results.json; TC-015.test.js;` (PARTIALLY PASS) TC-016.test.js todo entry; current scope-deferral record (PASS)	Approved	D13, CR-004, CR-008
REQ-018	**STK-003, FR-018** The system should allow staff to correct category for a request. *Project Master Brief, Project Scenario: CivicConnect, p.6*	High	Staff can select the correct category for a service request. Staff can see the updated category.	—	Staff · service_request.category · Express	RequestWorkspace.jsx	TBD	TBD	—	No verification yet	—

REQ-019	**STK-008, NFR-001** The system should not delay processing by more than 10 seconds.	Medium	When a button is pressed for any system related functionality, the loading should not take more than 10 seconds.	ASR-005	Architecture-wide · n/a · Stateless Express + PostgreSQL pool	Not yet implemented	TBD	TBD	—	No verification yet	D12
REQ-020	**STK-008, NFR-002** The system should support Email to allow notifications to be sent regarding status updates. (Changed requirement to exclude SMS)	Medium	Status update notification is sent to Email. (Removed SMS)	ASR-008	Notification · n/a · Nodemailer + SMS provider	Not yet implemented	TBD	TBD	—	No verification yet	D11, CR-005, S-DEF-002, R07
REQ-021	**STK-001, NFR-003** The system should be intuitive for requesters to use and should receive meaningful feedback. *Project Master Brief, Project Scenario: CivicConnect, p.6*	Medium	At least 80% of beginner requesters should be able to submit a service request and check its status without assistance in a moderated walkthrough of the M3 build.	ASR-010 (Added ASR-010)	Client · n/a · React	_	TBD	TBD	—	No verification yet	CR-007, R13
REQ-022	**STK-001, NFR-004** The system should ensure that service requests do not become corrupted or duplicated during submission. *Project Master Brief, Project Scenario: CivicConnect, p.6*	Medium	Service request is submitted and stored in database without corruption. Rollback occurs for corrupted data and recovery message is displayed. Requester cannot submit duplicate service requests.	ASR-007	Request / Data layer · service_request · PostgreSQL transaction, validation + constraints	SubmitRequest.jsx	TC-004	Title: “Malfunctional robot”, Description: “Our friendly cafeteria robot started attacking our students with coffee mugs!”, Street Address: “10 IT Street”, Category: “” → “You must select a category”	Console output showing PASS backend/tests/unselected_category.test.js	Approved	D06, D08, R06
REQ-023	**STK-006, NFR-005** The system should protect user data and enforce role-based access. *Project Master Brief, Project Scenario: CivicConnect, p.6*	High	A requester can only see their requests. A staff can only see requests inside their jurisdiction. Senior management has access to oversight information. No one can gain access to another user’s data unless under strict, authorized reasons.	ASR-002	User (cross-cutting) · user.user_type, user, service_request · Express middleware, JWT + RBAC	AuthPage.jsx, authMiddleware.js	TC-002, TC-007, TC-014	Requester JWT on /api/v1/staff/requests → 403 Forbidden IT-Support staff JWT → only IT-Support requests returned Each forbidden combination → 403 or UI hides action	Console output showing PASS backend/tests/unit/TC-002.test.js Console output showing PASS tests/integration/TC-007.test.js `milestone3-results.json`; `TC-014.test.js`; (FAIL)	Approved	D06, D08, D13, R01
REQ-024	**STK-006, STK-008, NFR-006**	High	Hashing should convert	ASR-003	User · user · bcrypt (passwords), SHA-256 (tokens)	Backend/modules/user/service.js	TBD	TBD	_	No verification yet	D13, R02, CR-

	The system should encrypt passwords and verification tokens once a user creates a new account.		passwords and tokens to hashed values. Hashed passwords and tokens must be stored in the database.								004, CR-008
REQ-025	**STK-001, NFR-007** The system should allow users to use the platform via any device.	Medium	The platform can be used with a phone, tablet, laptop, and desktop computer.	ASR-010	Client · n/a · Responsive React	_	TBD	TBD	—	No verification yet	R08
REQ-026	**STK-001, STK-002, STK-003, STK-004, STK-005, STK-006, NFR-008** The system should ensure that it is available 99.9995% of the time.	High	The platform should have a maximum RTO of 4 hours (City and County of SAN FRANCISCO, 2023).	ASR-004	Architecture-wide / Deployment · n/a · Stateless API, host chosen in M3	_	TBD	TBD	—	No verification yet	D07, R05, CR-006
REQ-027	**STK-001, NFR-009** The system should allow the requester to submit service requests quickly.	Medium	The median time from opening the form to confirmed submission is 60 seconds or less for a typed description of about 20 words.	ASR-005	Request · n/a · Single-transaction insert	_	TBD	TBD	—	No verification yet	D06, CR-007
REQ-028	**STK-006, NFR-010** The system should ensure that a backup database is ready if the main database fails.	High	Standby database takes main database's place during database failure with a RPO of less than 15 minutes (City and County of SAN FRANCISCO, 2023).	ASR-004	Deployment · n/a · PostgreSQL standby / replica	Not yet implemented	TBD	TBD	—	No verification yet	D07, R05, CR-006
REQ-029	**NFR-011** The system should ensure that 210 concurrent users can use the system. *Past similar system (Airport Company South Africa, n.d.), Project Master Brief, 2. Project Scenario: CivicConnect, p.6*	Medium	If 210 users use the system at the same time, the system itself should not crash.	ASR-006	Architecture-wide · n/a · Stateless auth + PostgreSQL pool	_	TBD	TBD	—	No verification yet	D05, R04
REQ-030	**STK-008, NFR-012** The system should ensure that a single-time email notification is sent to the user to	High	A user receives 6-digit code notification via email and can confirm registration and land to home page.	ASR-003, ASR-008	User / Notification · user.email_verified, verification_token_hash, verification_expires_at · Nodemailer (MailSender)	Not yet implemented	TC-006	Valid name/email/password → 201; token hashed; login returns 403 EMAIL_NOT_VERIFIED until code confirmed	Console output showing PASS backend/tests/integration/TC-006.test.js (5.632 s)	Approved	D13, CR-004

	confirm successful registration.										

15. Risk Register

Risk ID	Description	Cause	Probability	Impact	Priority	Mitigation Strategy	Contingency Plan	Owner	Status
R01	Failure to enforce role-based access, allowing requesters or staff to view unauthorized oversight information.	NFR-005 mandates strict jurisdiction visibility, which is complex to secure during early architectural phases.	Medium	High	Critical	Define strict authorization boundaries and route protection in Milestone 2 architecture planning.	Restrict system access entirely until authorization boundaries are verified.	Mohau Drew Modiselle (Backend)	Active
R02	Password hashing fails or user credentials become exposed.	NFR-006 and NFR-009 require secure credential handling and encryption upon account creation.	Low	High	Critical	Implement industry-standard encryption and security protocols before user registration is built.	Force a global password reset and patch the hashing vulnerability.	Mohau Drew Modiselle (Backend)	Active
R03	System processing delays exceed the 10-second threshold.	Heavy filtering, sorting (FR-020), and management statistics (FR-018) calculations executed concurrently against the database.	Medium	Medium	High	Optimize database queries and indexing specifically for heavy filtering/sorting tasks.	Defer live statistical rendering in favor of asynchronous reporting.	Benno Weideman (Database/Lead)	Active
R04	System crashes or degrades under load.	NFR-013 mandates support for 210 concurrent users, which strains unoptimized server resources.	Medium	High	High	Design scalable server architecture and include load testing in the testing strategy.	Temporarily queue incoming user requests until capacity frees up.	Mohau Drew Modiselle (Backend)	Active
R05	The backup database fails to execute a successful failover.	NFR-012 requires redundant database architecture, introducing complex synchronization dependencies.	Low	High	High	Define rigorous database redundancy and failover protocols in the Forward Engineering Considerations in M2 once tech stack is chosen	Initiate manual database restoration from the most recent safe snapshot.	Benno Weideman (Database/Lead)	Active

R06	Submitted service request information becomes corrupted and the system fails to execute a rollback.	NFR-004 requires corruption-free storage and rollback mechanisms during transaction failures.	Low	High	High	Enforce strict transactional database rules and atomic commits for all data ingestion.	Isolate the corrupted tables and restore the missing records from local logs.	Benno Weideman (Database/Lead)	Active
R07	External API failures prevent the system from sending required status notifications via Gmail or SMS.	NFR-002 requires external platform integration, making the system reliant on third-party uptime.	Medium	Medium	Medium	Map out API rate limits and build robust error handling for external communication	Default to displaying in-app status updates (FR-004) until API connectivity is restored.	Tammy Fourie (Frontend/Design)	Active
R08	Platform interface fails to function or render correctly across phone, tablet, laptop, and desktop devices.	NFR-008 requires compatibility across phone, tablet, laptop, and desktop interfaces.	High	Medium	High	Commit to a highly responsive UI framework in the Milestone 2 technology selection.	Enforce a desktop-first deployment until mobile layouts are stabilized.	Tammy Fourie (Frontend/Design)	Active
R09	Scope creep driven by the high volume of complex "High" priority features (e.g., auditing staff actions, FR-021) leads to premature M1 coding.	High volume of complex "High" priority features (e.g., FR-021 auditing) tempting developers to build before baselining.	High	Medium	High	Strictly bind these complex requirements to future milestones and lock the PED v1.0 scope baseline.	Revert any premature implementation branches and refocus on documentation.	Tammy Fourie (or Benno Weideman)	Active
R10	Layer and module boundaries erode; controllers start to contain SQL or business rules.	Deadline pressure; layering is new to the team (D09).	Medium	Medium	Medium	Dependency rules in the PR checklist now; ESLint import rule in M3.	Refactor the offending module before the M3 merge freeze.	Mohau Drew Modiselle (Backend)	Active
R11	Status names differ between screens, API, database and tests.	CR-003 not approved or not applied everywhere (REQ-004, REQ-013, REQ-017, REQ-022, §8.1, §12.2).	Medium	High	High	One status enum in the domain layer (D10); UI reads labels from the API; CHECK constraint in PostgreSQL.	Map old labels at the API until CR-003 is closed.	Mohau Drew Modiselle or Benno Weideman	Active

R12	Wireframes do not match what requesters understand (category names, error wording, confirmation email).	No one outside the team has tried the design yet.	Medium	Medium	Medium	One walkthrough with a non-team participant before the forms are built (§10.6).	Change labels and wording; cheap before build.	Tammy Fourie	Active
R13	Accessibility defects are found late, when styling is applied.	Wireframes are greyscale; contrast and keyboard use untested.	Medium	Medium	Medium	Automated contrast and keyboard checks in the M3 quality gates (§10.4).	Fix before release; record residual issues.	Tammy Fourie	Active
R14	Confirmation emails are delayed, blocked or land in spam, so new users cannot activate their account.	Email verification at registration (D13) depends on email delivery.	Medium	High	High	Provider with SPF/DKIM set up; "resend confirmation email" on the check-email page and sign-in error; delivery tested in M3.	Administrator can mark an account as verified after checking identity; switch provider through the MailSender adapter (D11).	Mohau Drew Modiselle	Active
R15	Unconfirmed accounts build up (typos, abandoned or automated sign-ups).	Accounts stay unconfirmed when the link is never opened (D13).	Medium	Low	Low	Rate limits on registration; unconfirmed accounts deleted after 7 days by a scheduled clean-up.	Manual clean-up query by the administrator.	Mohau Drew Modiselle & Tammy Fourie	Active
R16	Free-tier email quota is exceeded, so confirmation and notification emails stop.	ASR-001 (R0 budget) limits provider choice; every registration and (if CR-005) status change sends an email.	Low	Medium	Medium	Rate limits per email and IP; check provider quotas when choosing in M3 (CON-CST-03).	Pause status emails first (in-app feedback continues), keep confirmation emails.	Benno Weideman & Mohau Drew Modiselle	Active
R17	Additional entities for the ERD would require major restructuring of existing database if already fully set up for old ERD, causing extra time loss and technical debt.	CR-009, which aks for change of ERD to include 5 additional entities to fit better normalization standards.	Low	Medium	Critical	Enforce understanding for all team members to fully understand what does the CR-009 entails and appropriately adapt to changes made to ERD.	Use previous option for an alternative ERD which only included 6 entities (user, service_request, action, logging, category, verification_token).	Benno Weideman	Active

16. Forward Engineering Considerations

Concern	Why does it matter now?	What later decision/activity may it influence?	What information is still missing?	What is the risk of ignoring it?	Status
Native vs web application	This affects the non-functional requirements such as portability, intuitive ease of use, visibility of platform, and 10 second processing delay.	It will influence the UX design, implementation, and testing phases.	None.	If the wrong architecture is chosen, it can have an impact on the performance of the platform since there is a difference in energy consumption between native and web applications (Horn et al., 2023).	Resolved — React web application selected (D05).
Database failover operation	This affects the non-functional requirements such as availability, uncorrupted data, and the actual database failover operation must work. Department supervisors and senior management depend on the saved audit trail of staff to ensure accountability.	It will influence implementation, testing, deployment, and maintenance.	Which deployment platform will host PostgreSQL, and what specific failover/replication mechanism it supports. The level of expected community outages is still unknown.	If a real database failure occurs and the system does not apply the failover operation correctly, users could lose significant front-end facing functionality, i.e. data on the pages will not display (Datadog, 2025).	In progress
Free tier development	This affects the level of data insights that management can use for the system specified in the stakeholder analysis. It also affects non-functional requirements such as the speed of processing service requests and the user experience of a requester.	It will influence the UX design, implementation, testing, deployment, and maintenance.	IDE, programming frameworks and database are selected which are Visual Studio Code, React, and PostgreSQL. However, the free hosting platform still needs to be selected.	If the team relies solely on a single vendor for free-tier development tools, it could face a challenge where the vendor forces all non-enterprise users to pay from a specific date (Heroku, n.d.).	In progress — tooling confirmed; hosting platform still open.
Scalability	This affects the non-functional requirements such as the concurrent number of users, processing time of 10 seconds, as well as the audit trail functional requirement.	It will influence design, implementation, testing, deployment, and maintenance.	Only the community's growth rate beyond the 210-concurrent-user target (NFR-013) is not known; architecture style is resolved, which is layered, references D05.	The database can be designed of poor quality where it tries to query large amounts of requesters' service requests, causing over-fetching if there are not many requesters at all (Çetin & Eldenk, 2025).	In progress
Security	This will affect the non-functional requirements such as role-based access, user credentials and history protection, and password/authentication handling. Supervisors and management require oversight data but must not have request information exposed.	It will influence the implementation, testing, deployment, and maintenance phases, and the RTM status of REQ-023, REQ-030 and REQ-033.	If the email provider will expect costs whenever notification tokens are sent via email to establish user identity for first-time user registration.	Surprise costs of email provider for tokens will force the team to move to an approach where the notification token should be simulated instead, degrading real notification quality.	In progress
External communication	Status updates must be communicated to requesters via notifications (NFR-002); email is the most likely first channel given the R0 budget constraint (ASR-001).	Will have an impact on the Notification module's provider adapters (D11) as well as the AI Usage Register if AI compares between providers.	What free email provider and second communication channel will be used and if the second channel should be added or not.	Notification boundary, which is designed in the D11, does not have an adapter, having NFR-002 unimplemented, since the provider was not selected yet.	In progress
Audit/diagnostic log retention and archiving	The logging table is meant for diagnostic purposes and records any rows which has CRUD implemented on it.	Have an impact on deployment platform decision, the backup method, and R11/R12.	What is the time that logging records can be kept before it enters retention or deletion, and what hosting platform can support that amount.	If left without management, *logging* can cause more overloaded writes than operational tables, other entities such as *user*, *service_request*, and *action* can be degraded without realization why.	Not yet started

17. Engineering Decision Log

Decision ID	Context	Constraints	Alternatives	Decision	Rationale	Trade-offs	Risks	Evidence	Later Consequence
D01	Repository setup and team collaboration workflow.	Mandatory 3-student team size and strict GitHub assessment criteria.	Unprotected main branch or single-reviewer sign-off.	Adopt a protected main branch with a mandatory two-reviewer approval model.	Enforces peer accountability and prevents uncontrolled code/documentation from entering the baseline.	Slower initial integration speed due to review bottlenecks.	PRs sitting unreviewed, risking schedule delays.	Branch protection rules applied in GitHub settings.	**Active & Extended in M2:** Continues to govern all M2 code and PED branches (CON-RES-04, CON-GOV-01). Extended in M2 using Assignment 2 Task 4 research by introducing a structured peer-review checklist and automated GitHub Actions CI quality gates (npm ci, build, lint, unit tests) on Pull Requests (D07, Section 7.4).
D02	Technology stack and database selection.	Milestone 1 boundaries explicitly prohibit final technology selection.	Select a framework immediately to begin coding.	Defer the final technology stack selection to Milestone 2.	Prevents premature technical commitment before requirements and operational constraints are baselined	Delays technical experimentation.	Teams may face a steeper learning curve in M2 once the stack is locked.	Recorded as a deferred decision in PED v1.0.	**Superseded in M2 by D05 and D08:** Resolved once M2 Architecturally Significant Requirements (ASR-001–ASR-010) and Assignment 2 persistence/integration research were completed, resulting in the baselined PERN stack (D05, Section 9) and consolidated 4-table PostgreSQL schema (D08, Section 8)
D03	Scope of detailed UI wireframes, DB schemas, and APIs.	Milestone 1 is restricted to an engineering and requirements baseline.	Creating mockups and schemas to get a head start.	Exclude UI, DB schemas, and APIs from Milestone 1 deliverables	Keeps the team focused entirely on defining requirements, risks, and constraints.	Less visual context for stakeholders during M1.	Misalignment on UI expectations in M2.	Scope baseline boundaries in PED v1.0	Superseded/ Fulfilled in M2: Resolved in PED v2.0 through the baseline ERD and persistence model, Information Architecture and clickable wireframes as well as the REST API Endpoint Contract Baseline (Section 12, D06).
D04	Standardization of AI usage and verification.	AI output is not authoritative evidence and requires human verification.	Ad-hoc tracking or total prohibition of AI tools.	Mandate the AI Usage Register with specific naming conventions (e.g., AI-RES).	Maintains academic integrity, transparency, and ensures human accountability for all AI-assisted work.	Administrative overhead in logging queries.	Team members forget to log minor AI queries.	AI Usage Register document.	**Active & Expanded in M2:** Continued in Section 21 (AI-016 to AI-023) to govern and independently verify M2 code generation, architecture diagrams, database connection setup, and peer-reviewed academic citations (CON-SEC-06, CON-GOV-03)
D05	Milestone 2 Technology Stack Commitment (supersedes M1 deferred decision D02).	ASR-001 (R0 budget), CON-RES-01 (3-person team), ASR-006 (210 users), ASR-007 (ACID integrity), ASR-010 (cross-device UI)	Native mobile apps + NoSQL (MongoDB) or enterprise Java/.NET stack.	Baseline the **PERN stack** (PostgreSQL 16, Express 4, React 18, Node.js 20 LTS) with pg, bcrypt/crypto, jsonwebtoken, nodemailer, and npm ci lockfiles	Open-source (R0 cost), unified JavaScript across client/server, native ACID transaction support in PostgreSQL, and responsive web reach across all devices (Horn et al., 2023; Hellerstein, Stonebraker and Hamilton, 2007)	Less schema flexibility than NoSQL; requires strict npm supply-chain auditing (npm audit)	Transitive npm package vulnerabilities (CON-SEC-05); free-tier hosting limits (R04)	PED v2.0 Section 9; package.json, package-lock.json, and initial backend/frontend bootstrap in GitHub repository.	All M2/M3 modules (User, Request, Staff, Admin) and CI checks must conform to the PERN baseline
D06	Client-Server Integration Mechanism & Data Validation Boundary for Request Submission and Status Updates.	ASR-005 (<10s/30s response), ASR-007 (REQ-028 zero corruption), REQ-031 (NFR-007 no duplicates),	Asynchronous message broker (RabbitMQ/Kafka) or single-layer validation with read-caching	Adopt a **Synchronous HTTP/REST JSON API** (/api/v1/) with **hybrid validation** (Express fail-fast + PostgreSQL ACID constraints) and **exclude read-caching** on write validation paths	Provides immediate transactional confirmation to the React UI without message-broker complexity (De Iasio and Zimeo, 2021; Nookala,	Tight temporal coupling between React client and Express server during submission.	Downstream DB slowness directly impacts API latency (R03, R06)	PED v2.0 Section 12; A2 Task 2 & Task 3 research; Express route/controller artefacts.	External notifications (ASR-008) are isolated behind the D11 Strategy (`NotificationChannel`) and Adapter (`MailSender`) and are dispatched after commit by the D12

		CON-SEC-03 (RBAC)			2023) and prevents stale-cache duplicate bugs (Bailis et al., 2012).				Observer (`EventDispatcher`), so they never block synchronous REST responses.
D07	Milestone 2 Deployment Compatibility & Environment Configuration Strategy	CON-TEC-03 (environment separation), CON-SEC-02 (no committed secrets), ASR-001 (R0 budget), M2 Section 5.8 boundary.	Hardcoded local config or premature full cloud/container production orchestration in M2.	Enforce **stateless API design**, runtime .env secret injection (.gitignore + .env.example), and npm ci parity while **deferring final cloud hosting/failover automation to M3**	Prevents credential leaks (CON-SEC-02) and environment drift (Nazário, Bonifácio and Pinto, 2025; Gamage et al., 2026) without wasting M2 coding capacity on premature infrastructure	Local development does not yet simulate real cloud network latency or live replica failover	Free-tier cloud limits or cold starts discovered during M3 deployment (R03, R05).	PED v2.0 Section 13; .gitignore and environment config structure in repository	Milestone 3 must select the final hosting platform and verify backup/failover recovery (REQ-036, CR-006).
DO8	Requester, Staff, and Management are merged into a single entity called "user".	R0 budget (ASR-001), RBAC (ASR-002), password security (ASR-003), no corrupted requests (ASR-007), small team.	1 user table vs 3 user tables; Idempotency keys against pre-save checks (A2 §5.4.1). Caching against no caching for duplicates (A2 §5.5).	Single "user" table with "user_type", caching duplicates not allowed, and database has the authoritative say.	Merging the tables is simpler without weakening RBAC. A2 §5.2 requires all-or-nothing saves. A2 §5.3 gave the validation layers. A2 §5.4.1 favoured pre-save checks over idempotency keys as proportionate for M2. A2 §5.5 warned against caching due to staleness. A2 §5.6 recommended the Database+App hybrid, which we followed.	The "user_type" must be a filter usage for users (R10). Duplicates still possible due to no idempotency (R13).	R01, R02, R06, R10, R13	SEN381 Assignment 2, Task 2, §5.1–5.6; PED v2.0 §8.3 and §8.5	Make update to this when submission endpoint is done. Make visit to idempotency keys during M3.
D09	M2 must fix the system structure before construction; ten ASRs (§6).	ASR-001, ASR-002, ASR-007, ASR-008, ASR-009; CON-RES-01; CON-SCH-01.	Microservices; serverless functions + hosted backend; unstructured monolith (§7.2).	Layered modular monolith: one React client, one Express API with five modules (User, Request, Staff, Admin, Notification) and five logical layers, one PostgreSQL database.	Highest weighted score (89/100). One transaction boundary for ASR-007/009 and one enforcement point for ASR-002, with module boundaries for ASR-008 (Fowler, 2015; Richards and Ford, 2020).	More folders and interfaces than a flat app; the API scales as one unit; one crash stops all modules.	R10 (boundary erosion); R05 (single database).	PED v2.0 §7, Figures 7-1 to 7-3; repository folder structure (README).	Review in M3; a module could be extracted later without rewriting domain code.
D10	Status vocabulary differs across REQ-004, REQ-013, REQ-017, REQ-022, §8.1 and §12.2; S-IN-007 needs reject, cancel and reopen.	ASR-007, ASR-009; S-IN-007.	Keep per-requirement wording and translate in the UI; free-text status.	Seven stored states: Pending, Accepted, In Progress, Completed, Closed, Rejected, Cancelled. Open and Overdue are derived.	Screens, audit and reports read the same values; invalid moves become a tested domain rule (409).	Needs CR-003 to reword baselined criteria.	R11.	PED v2.0 §7.5, Figure 7-4, Table 7-3.	Section 8 CHECK constraint and §12.2 status values follow D10.
D11	Requester notifications (REQ-006, REQ-026, ASR-008) with channels that can be added or replaced; email provider now also needed for registration confirmation.	ASR-008; S-IN-003 and S-DEF-002 (scope); ASR-001.	Direct notification service (A2 Task 1 §2.2); Strategy + adapters (A2 §2.3).	Strategy pattern for channels: NotificationChannel interface, EmailNotificationChannel built, SMS deferred. Adapter pattern for the provider: one MailSender interface implemented by NodemailerMailSender (Nodemailer, MIT-0), shared with registration confirmation. In-app feedback is not a	A2 condition for Strategy is met (more than one channel named, shared provider). Adding a channel is one new class; changing provider touches one adapter; email failure never rolls back core writes (§12.1, R07).	Two small interfaces to maintain.	R07, R14, R16.	PED v2.0 §11.1, Figure 11-1; A2 Task 1.	Status emails switched on only if CR-005 is approved; SMS added later as one class.

				channel; it reads the action table.					
D12	A status change must update the request and its audit row together and may trigger notifications and future escalation.	ASR-005, ASR-007, ASR-009.	Pure orchestration, with the service calling every reaction directly (A2 Task 1 §3.2); pure Observer, with every reaction including the audit row driven by events (A2 §3.3).	Hybrid (A2 §3.5). Mandatory work is explicit orchestration: request update and action row in one transaction. Optional reactions use the Observer pattern: `RequestStatusChanged` is published to an in-process `EventDispatcher` after COMMIT and handlers are not awaited.	Audit guarantee visible in one method; new reactions added without editing the use case; slow email cannot delay the response.	Observer adds indirection: the publisher does not show who reacts, so handler registration in `app.js` must be checked. A failed email handler is logged, not retried.	R06, R10.	PED v2.0 §7.6, §11.2, Figure 7-5.	Escalation (S-DEF-001) can be added as a handler.
D13	Lecturer requirement confirmed in M2: new accounts must confirm their email address before they can sign in.	ASR-002, ASR-003, ASR-001; REQ-023, REQ-033.	No verification (baseline); email code at every sign-in; authenticator-app or passkey second factor.	Register with name, email and password (bcrypt, unchanged); a single-use confirmation link (stored only as a SHA-256 hash on the user row, 24-hour expiry) is emailed; sign-in is refused until the email is confirmed.	Proves the requester owns the address used for notifications; blocks sign-ups in someone else's name; password sign-in unchanged (OWASP Authentication and Forgot Password cheat sheets; NIST SP 800-63B-4 treats email confirmation as outside its ban on email authentication).	Email delivery needed at sign-up; unconfirmed accounts need clean-up; not a second factor.	R14, R15, R16.	PED v2.0 §11.3, Figure 11-2; CR-004.	Adds three user attributes: email_verified, verification_token_hash, verification_expires_at (§8); three auth endpoints in §12.2; Nodemailer added to §9; a true second factor for staff is an M3 candidate.
D14	RTM simplification.	All functional and non-functional requirements, and ASRs.	Leave requirements unchecked and do not simplify.	Review functional and non-functional requirements and simplify and merge similar requirements together and fix unambiguities.	Testing and verification workload will be reduced significantly.	Additional review time allocated against a tight deadline.	Reviewer loses time and impacts other content that requires link.	Pull request created with committ “8000f4c”.	Extra reviews and updates to dependent documentation content.
D15	ERD changes to include additional 5 entities to normalize ERD.	The 5 existing entities such as user, service_reuqest, action, and log (was logging). The PostgreSQL ERD must be adapted.	Keep the same outdated ERD excluding the new 5 entities. Only include 2 additional entities for category and verification_token.	Choose the AI generated ERD suggestion including 5 additional new entities with more relationships.	Entities do not contain unnecessary attributes, e.g. user with user_type of requester will not benefit containing the category attribute if only meant for staff, separate category in a separate category table.	More entities to manage and more relationships making it needed to involve more joins to perform queries.	R17	8. Data and Persistence Baseline, updated entities overview, new genrated ERD image, and justification of data model.	For CR-009, start to develop the database schema using the new additional 5 entities and include the necessary relationships and indexes.
D16	Removal of user input for start and end dates for a service request.	REQ-001	Rejected decision and allow requesters to enter start and end dates. Allow the requester to only submit start date and end date is calculated.	Deliberately remove the start and end date input option for requesters.	Requesters can provide unrealistic start and end dates since they are not knowledgeable of the duration of need for a specific service request. Start date is calculated using the current date, and the end date is calculated based on a predefined number of days added to the current date. The days are determined based on the service category and the days are based on general	Predefined durations based on a category remove realistic durations; emphasizes focus on a predefined metric.	Service request duration may not reflect real activity taking place to resolve request.	PR 61#	Real end dates need to be calculated based on the actual specfic duration of a real service request; this is not suitable for a real environment.

					durations of a category request.				

18. Change Impact Analysis

Change ID Requested by Date	Requested change	Reason / Expected value	Requirements affected	Architecture / Design affected	UI/API/data affected	Security/privacy impact	Quality/ testing impact	Scope impact	Schedule/resource impact	Cost impact	Risk impact	Recommendation	Approval / rationale
CR-001 Benno Weideman 26/09/2026	Remove country, city, and province from service request and replace with street address.	Less complexity and unnecessary information.	FR-001, REQ-001	Service request entity will have less fields.	Less textboxes for new service request to be entered.	None	Service request receives less details and specifics. Request location can now only be mentioned by street address.	None	Small	None	None	ACCEPT	The CivicConnect platform is for a community, not national usage.
CR-002 Benno Weideman 29/09/2026	Remove number attribute from service request	Unnecessary contact details that are replaced by email for users.	FR-001, FR-002, REQ-001, REQ-002	None	Less textboxes for service request to be entered, one attribute reduced from service request.	Sensitive contact information is reduced and only contains email.	None	None	Small	None	None	ACCEPT	The CivicConnect already has users having their email, number is unnecessary contact details.
CR-003 Tammy Fourie 29/09/2026	Adopt the D10 state model; reword status criteria of REQ-004, REQ-013, REQ-016, REQ-017, REQ-022; add cancel and reopen.	One status vocabulary for screens, API, audit and reports; meets S-IN-007.	REQ-004, REQ-013, REQ-016, REQ-017, REQ-022	§7.5 (D10), §8.1 lifecycle column, §11.2	Status pills and filters; §12.2 status values; CHECK constraint on service_request.status	None	Transition unit tests for every allowed and forbidden move	Adds cancel and reopen (already required by S-IN-007)	Small	None	Reduces R11	ACCEPT	Pending team approval
CR-004 Tammy Fourie 29/09/2026	Add email verification at registration: a confirmation link is emailed and the account cannot sign in until it is confirmed. Email + password sign-in is unchanged.	Confirms the requester owns the email used for notifications; prevents sign-ups in someone else's name.	REQ-023 (add criteria: confirmation email sent; sign-in refused until confirmed), REQ-033	§11.3 (D13); §8.1 user attributes email_verified, verification_token_hash, verification_expires_at; §9 add Nodemailer; §12.2 auth endpoints	Register, Check-email and Sign-in screens; POST /api/v1/auth/verify-email and /auth/resend-verification; login returns 403 EMAIL_NOT_VERIFIED	Token stored only as a hash; no account enumeration	Token expiry, single use, resend and unconfirmed-login tests	Email provider moves from deferred to required for registration	Small: one table, three endpoints, two screens	Free-tier SMTP; quota risk R16	Adds R14, R15, R16	ACCEPT	Pending team approval
CR-005 Tammy Fourie 29/09/2026	Resolve REQ-026/ASR-008 vs S-IN-003/S-DEF-002: commit two channels (in-app + email) and keep SMS deferred.	Meets "at least two channels" at R0 cost using the email provider already needed by CR-004 (registration	REQ-026 (reword), REQ-006	§11.1 (D11); ASR-008 wording ("2 or less" vs "at least two")	Email template for status changes; opt-out link	Email contains request title and status only, not the description	Email channel tests in Ethereal	Moves email notifications from S-DEF-002 into S-IN-003; SMS stays deferred	Small once CR-004 is in place	None beyond CR-004	R07, R16	ACCEPT	Pending team approval

		confirmation).											
CR-006 Tammy Fourie 29/09/2026	Restate REQ-034 (99.9%) and REQ-036 (failover) as measurable RPO/RTO targets set when the M3 host is chosen.	Makes ASR-004 testable on the hosting the team can afford.	REQ-034, REQ-036; ASR-004	§7.7; §8.5.3 and §8.5.4 (weekly backups vs failover); §13	None	None	Backup-restore test replaces an untestable uptime claim	None	Decided in M3	Depends on host	R05	MODIFY: set values in M3	Pending team approval
CR-007 Tammy Fouri 29/09/2026	Make REQ-027 and REQ-035 measurable as in §10.5.	Turns "intuitive" and "30 seconds" into walkthrough measures.	REQ-027, REQ-035	§10.5	None	None	Moderated walkthrough with 5 first-time users in M3	None	About 2 hours in M3	None	R12	ACCEPT	Pending team approval
CR-008 Benno Weideman 30/09/2026	Add token hashing in ASR-003.	Tokens require hashing to prevent exploiters from using tokens to exploit newly registered users receiving tokens via email.	NFR-006, REQ-030	The "user" entity contains an additional attribute called "*verification_token_hash*".	Additional API for taking a user to their email to confirm 6-digit "token" to confirm first registration.	Tokens can be safely stored in the PostgreSQL database without plain text exposure for exploit abuse.	Improved user verification via tokens.	Adds additional hashing scope for credentials.	None	None	R02, R14	ACCEPT	Token hashing is vital for security; protecting against exploits, not just hashing passwords.
CR-009 Benno Weideman 03/10/2026	Include additional 5 entities for the ERD.	The ERD requires normalization; there are entities that contain irrelevant attributes and need to be added to suitable entities instead.	ASR-003, ASR-009, REQ-007, REQ-015	Four new entities (verification_token, category, staff_category, action_status, action_type). All baselined changes that affect the data is stated in 8.1.1, p.31	None	Require that verification_token should never be recorded in the log entity.	Additional attributes to include for testing queries and joins.	Database administrator need to incorporate additional 5 entities, however, database development has not started yet, so little impact for ERD taht was generated by AI.	Small, database development has not formally started	None	Reduces R01 and R02	ACCEPT	Team has approved based on the assessor feedback; normalization was crucial to better the ERD.
CR-010 Tammy Fourie 05/10/2026	Name and explain the design patterns consistently across the PED. Add "Patterns used" tables to §11.1 and §11.2 and a new §11.5 design pattern register (Tables 11-	PED v2.0 used the patterns without always naming them or saying where they sit in the code. Clearer naming lets the assessor	None. No requirement or acceptance criterion changes. Traces for REQ-006, REQ-026, REQ-029, ASR-	Wording only in §6 (ASR-002, ASR-005, ASR-008), §7.1, §7.3, §7.5, §7.6, §9.2, §9.3, §11 intro, §11.1 to §11.4, §12.1, §16, §17 (D06, D11, D12). New §11.5. Decisions D09 to D13 are unchanged.	None	None	None to planned tests. The pattern status lines must match the release candidate, checked in §23.2.	None	Small: documentation only	None	Reduces R10 (boundary erosion), because the intended structure is now explicit.	ACCEPT	[Pending team approval / Approved by team on __]. Clarification only; no baselined decision changes.

	8 and 11-9). Correct the Nodemailer /Adapter wording in §9.2 and §9.3. Add an M3 status line for each pattern.	and every team member trace a pattern from ASR to decision to file, and supports M2 baseline conformance in §23.	002, ASR-005 and ASR-008 are clarified only.										
CR-011 Benno Weideman & Mohau Drew Modiselle 10/10/2026	Remove start and end date user input so that the system automatically calculates them both.	Requesters may provide unrealistic end dates for a specific request category, and providing input for start date does not confirm that the actual start date is the begin of the progress.	FR-001, REQ-001	Wording only for functional requirements and requirements.	Check constraint must be removed from the service request entity that prevents start and end date contradiction. The input fields for start and end dates must be removed.	Improved security; requesters do not have access to real start and end dates for a request; management has more realistic control of when a request is completed.	A suitable test case must be included to test the effects of two contradicting dates regardless of if the system automatically creates the dates in a suitable method.	Additional methods and code logic need to be created to cater for start and end date.	None	None	None	None	ACCEPT	Allowing user input for start and end dates will cause unrealistic request completion times; remove input ability and allow the system to calculate suitable dates.

19. GitHub Controls

This document defines the governance rules, branch strategy, and pull request workflow for the CivicConnect-Application repository. All team members are required to follow these controls to maintain code quality, accountability, and a stable main branch.

19.1 Governance Rules

Control	Rule
Repository	There will only be a single repository used on GitHub, called CivicConnect-Application.
Main Branch	This branch represents the protected, stable state of the project — it must always be in a working condition.
Direct Development on Main	Under no circumstances is any person allowed to commit directly to main.
Branches	All work must be committed to a dedicated branch before it can be merged into main (e.g. docs/requirement for the requirements document).
Pull Request	Before work is merged into main, a pull request must be opened. A minimum of **two** team members must review and approve the work.
Approval	Work must be approved by a minimum of **two** reviewers before it can be merged into main.
Self-Approval	Self-approval is **not permitted**. A person may believe their work is correct, but it must still be independently validated against sound software engineering principles.
Review Quality	Reviewers must provide meaningful, actionable feedback. Comments such as *"Code is good"* are not acceptable, as they do not meet project review standards.
Secrets	Any sensitive and confidential information such as API keys, passwords, private keys, and tokens may NEVER be committed on the repository.
History	Every commit, pull request, merge, and change must show progression and not bulk uploads

19.2 Branch Strategy (M1)

The following branches will be used to commit work before it progresses to the main.

Branch	Purpose
docs/problem-business-need	Project charter and business value
docs/stakeholder-analysis	Analysis of key stakeholders
docs/scope-baseline	In-scope, out-of-scope, and deferred scope items
docs/requirement	Requirements document with acceptance criteria
docs/constraints	Analysis and record of project constraints
docs/rtm	Requirements Traceability Matrix
docs/risk-register	Identification and analysis of project risks
docs/forward-engineering	Identification and explanation of early lifecycle concerns
docs/engineering-decision-log	Decision log for recorded engineering decisions
docs/github-governance	GitHub governance and team controls (this document)
docs/ai-usage-register	Live register tracking AI usage and assistance
docs/PED	Fully working PED containing all of the above project elements

19.3 Pull Request Workflow

Follow these steps whenever creating and submitting a pull request.

Step	Explanation

1. Create a Branch	Navigate to the repository on your computer, then create a new branch:
	`cd path/to/repository`
	`git checkout -b docs/requirements`

2. Make Your Changes	Edit the relevant part of the project.
3. Commit Your Changes	`git add .` `git commit -m "Update requirements"`
4. Verify the Commit	`git status`
5. Push to GitHub	`git push origin docs/requirements` (GeeksforGeeks, 2026)
6. Open the Repository on GitHub	Navigate to the project repository on the GitHub website.
7. Start the Pull Request	Click the green **Compare & pull request** button.
8. Set the Branch Comparison	Base branch: main, compare branch: the branch containing your changes
9. Add Details	Provide a clear, descriptive title and description of the changes.
10. Create the Pull Request	Click the green **Create pull request** button.
11. Request Reviews	A minimum of **two** team members must review the pull request.
12. Provide Feedback	Reviewers give meaningful feedback aligned with project standards.
13. Merge	Once feedback has been addressed and both approvals are given, merge the pull request.
14. Done	The change is now committed to the main branch (GitHub Docs, 2026).

20. Team Working Agreement

Project Name: CivicConnect	Module / Course: SEN381
Date Created: 9 September 2026	Repository: https://github.com/Stelselopsiener/CivicConnect-Application.git

20.1 Team Members & Roles

Student Name	Primary Role & Focus	Contact / Email
Benno Weideman	Team Leader	602904@student.belgiumcampus.ac.za
Mohau Drew Modiselle	Team member 1	602295@student.belgiumcampus.ac.za
Tammy Fourie	Team member 2	601264@student.belgiumcampus.ac.za

20.2 Primary Goals & Standards

Target Grade: Aiming for 80%.
Code Quality: All code must be checked and human verified by all team members before approval.
Workload Commitment: Every team member must contribute fairly to project and understand everything.

20.3 Communication & Meetings Protocol

Primary Channel: Teams for daily meetings, WhatsApp for quick check-ins.
Response Expectations: Messages should be acknowledged mostly during week, only on Saturdays should some messages be seen, Sunday is off.
Synchronous & Async Standups: Daily meeting at 17:00 (depends on the schedule of every person). If something occurs, meeting is moved to the next day.

20.4 Git & Workflow Standards

Branching Strategy: Team members create own branches but direct pushes to main branch are not allowed.
Code Reviews: If a pull request must be made, at least 2 reviewers must review the work and approve.
Commit Convention: Ensure that meaningful description of work is given for the commit.

20.5 Deadlines & Conflict Resolution

Internal Deadlines: Every project milestone has the following submission dates:

- M1: 9 September 2026
- M2: 30 September 2026
- M3: 16 October 2026
- Final Project: 18 October 2026

24-Hour Advance Notice: In the event if something happens, a team member must inform his/her team members within 24 hours.

20.6 Signatures & Agreement

By signing below, all team members commit to adhering to the terms outlined in this agreement.

Student Name	Signature	Date
Benno Weideman	_Benno Weideman_____	2026-09-09
Mohau Drew Modiselle	_Mohau Drew Modiselle____	2026-09-09
Tammy Fourie	_Tammy Fourie__	2026-09-09

21. AI Usage Register

In the event if any team member has asked the assistance of AI to analyse, research, compare architecture, design, code, test, debug, review documentation and operational analysis, the following table underneath provide all the information that each AI usage must require. The following key points of the table will assist in completing each AI usage within the table:

Features

1. ID: This is the ID of an AI usage. The naming convention of AI IDs are “AI-001”.
2. Team member: This is the full name of the team member who used AI.
3. Date: This is the date on which the team member acquired the usage of AI. For conciseness, keep the date format such as “25/06/2026”.
4. Tool: This is the tool that a team member used, in other words, name the AI tool. E.g. “Gemini”.
5. Engineering Task: Team member must explain why they used the AI tool. How did it assist for the project?
6. AI contribution: Write down what the AI contributed for the engineering task. Do not copy and paste the actual response from AI, simply summarize what the AI contributed for the task.
7. Verification: How did a team member check that the AI’s claim or assistance is correct? E.g. If a team member asked AI to guide research, verify the AI’s claim by seeing if the resources recommended has not been fabricated and exists on Google Scholar.
8. Decision: This is where a team member decides if they will accept the AI’s assistance or if they reject it. Simply write what was accepted, changed, or rejected from AI.
9. Issues Found: If AI did not contribute well and issues were found when a team member asked AI for assistance, write down what went wrong.

ID	Team member	Date	Tool	Engineering task	AI contribution	Verification	Decision	Issues Found
AI-001	Benno Weideman	04/09/2026	Gemini	Edit initial commit of README to have a clear description	Create fresh main branch. Create initial commit with clear description. Delete original main branch and rename fresh main to main. Force overwrites the new main to GitHub.	Manually checked the repository and locally on machine, the README is there with a clear description other than "Initial commit".	Entire suggestion was accepted and applied to fix commit.	None
AI-002	Benno Weideman	05/09/2026	Gemini	Idea generation of proper ID naming conventions for AI usage.	Suggested naming convention for IDs such as ai-"pmpt" and "resp".	Visually checked the naming convention for ID and it is appropriate and concise.	Accepted the naming convention but did not accept the word usage. Own words tailored to project was used such as DEBUG, RES, and REV.	None
AI-003	Benno Weideman	05/09/2026	Claude	Apply styling to GOVERNANCE.MD file.	AI applied styling to GOVERNANCE.MD file making it more appropriate and user friendly.	File looks visually appealing. Content was not changed.	Accept the entire change of styling.	None
AI-004	Benno Weideman	07/09/2026	Gemini	Provide links to the academic/standard/professional resources, not older than 2022, also make sure that it is free of cost and instantly readable.	Systems & Software Quality Standards ISO/IEC 25010 Software Quality Model is the title with link provided and AI explains what it is and why it helps for this research (Pacific Certifications, 2026).	Link is provided and access works, Pacific Certifications is a real company.	Accept the pdf, used it for usability, availability, portability, error prevention, and security for non-functional requirements.	Struggled to gain access to specific resources that was suggested by my prompt.
AI-005	Benno Weideman	07/09/2026	Gemini	Provide me academic/professional/standard resources from 2022 that is free that explicitly talks about their nonfunctional requirements in depth and is a similar system to civicconnect.	OpenLMIS Technical Documentation — Non-Functional Requirements Specification, ISO/IEC 25010:2023 Product Quality Model & Industry Standards, Open311 GeoReport API Specification (Official Standard), OSDM Non-Functional Requirements Specification, and Airports Company South Africa (ACSA) — Customer Relationship Management & Service Request Solution Scope (October 2022).	Link provided to ACSA CRM & Service Request Solution Scope works, company exists.	Accept the ACSA resource, especially the sum of all full functional rights that are customer query management, key account management, and employee query management. This had users of 100, 40 and 70 respectively and combined them to 210 concurrent users for non-functional requirements (Airport Company South Africa, n.d.).	Struggled to find resources that are close to the CivicConnect project that details non-functional requirements.
AI-006	Benno Weideman	08/09/2026	Gemini	Provide a solid explanation of what is expected to write for the forward engineering consideration with regards to the CivicConnect project.	Explained in depth what is a forward engineering consideration, what is required for why it matters now, what later activity may it influence, what information is still missing, and what is the risk of ignoring it.	Provided a full breakdown of each element in the forward engineering consideration table.	Accepted the format on how to write and structure the answers	Struggled to understand what was expected to write for forward engineering consideration with AI, this solution helped.
AI-007	Benno Weideman	08/09/2026	Claude	Provide 5 valid resources that are free, professional/academic/standard and are not older than 2022. These 5 resources should be aligned with the 5 forward	Provided 5 resources regarding the 5 forward engineering considerations that will help find evidence of risks of ignoring them.	All 5 resources have valid DOIs, they are searchable in Google Scholar, authors are real, links to website for some of the resources are real companies.	Accepted only 5 resources.	Struggled to find resources that provide evidence for risks and had to prompt AI to give proper resources.

				engineering considerations that our team chose such as native vs web, free tier development, database failover operation, scalability, and security				
AI-008	Benno Weideman	08/09/2026	Claude	Prompted AI to explain where to find answers of evidence for each resource and explain why the native vs web app, free tier development, and database failover operation risks resources provide evidence.	Analysed the sections for each resource and show where to find answers. Also explained how they can help to provide evidence.	Read through the resources and the summary of AI is aligned with what the actual resources say	Accept sections that has answers and used own words to explain the risks of ignoring the forward engineering considerations.	Struggled to find sections that can aid research in the documents.
AI-009	Mohau Drew Modisel le	08/09/2026	Gemini	Structuring the Initial Risk Register and Engineering Decision Log for PED v1.0.	Formatted project-specific risks and architectural decisions into Markdown tables containing all mandatory columns (Cause, Trade-offs, Evidence).	Cross-referenced the generated table headers directly against Appendix A of the SEN381 Master Project Brief.	Accepted the table structure and incorporated the formatted matrices into the baseline document.	None
AI-010	Mohau Drew Modisel le	08/09/2026	Gemini	Synthesizing academic literature to draft the Problem Statement and Business Value.	Integrated specific academic authors (Netshirando & Ngoepe, Hofisi & Chigova, Mashau) with CivicConnect stakeholder requirements to demonstrate civic and commercial value.	Verified that the text accurately reflected the provided authors and explicitly mapped to Requesters, Staff, and Management needs. Check source's credibility.	Accepted some of the synthesized text and appended the academic reference list to the document.	Not all sources were free, so I had to pick free sources only.
AI-011	Tammy Fourie	08/09/2026	ChatGPT	Review and refine the CivicConnect stakeholder analysis for Milestone 1.	AI was used to review the clarity and structure of the stakeholder analysis and to identify areas where stakeholder needs, influence, interest and competing expectations required clearer explanation.	I compared the recommendations with the CivicConnect scenario and the stakeholder-analysis expectations in the Master Project Brief and Milestone 1 brief. I confirmed that each retained point was relevant to an identified stakeholder and supported by the project context.	I retained the stakeholder identification and analysis I had developed, accepted selected wording and structural improvements, and rewrote the final content in my own words.	None
AI-012	Tammy Fourie	09/09/2026	ChatGPT	Review the CivicConnect project constraints and their engineering implications.	AI assisted in checking whether the analysis addressed the required scope, schedule, cost and resources, quality, and security constraints. It also suggested areas where the interactions and possible ripple effects between constraints could be explained more clearly.	I checked every proposed interaction against the constraints stated in the Master Project Brief. I reviewed whether the consequences were realistic for a three-person team working across four milestones and removed or revised unsupported statements.	I accepted selected suggestions that clarified valid constraint interactions. I determined the final implications and controls based on the project brief and the team's actual circumstances.	None
AI-013	Tammy Fourie	09/09/2026	ChatGPT	Review and refine the CivicConnect Scope Baseline for PED v1.0.	AI was used to compare the draft with the Master Project Brief and Milestone 1 expectations. It helped identify unclear boundaries and suggested wording for the in-scope, out-of-scope and deferred items, the deliberate exclusion d	I manually checked every scope item against the minimum business capabilities in the Master Project Brief. I confirmed that the revised scope did not introduce a final architecture or technology-stack decision and that excluded or deferred items were	I retained the scope decisions made by the team, accepted selected clarity and formatting improvements, and revised the final wording myself. I also confirmed that any future additions would require formal change control .	None

					efence, and scope-control statement.	not presented as committed requirements.		
AI-014	Benno Weideman	09/09/2026	Gemini	Asked for assistance on how to set up two-reviewer settings on GitHub.	Provided instructions on how to set up two reviewer model.	Manually checked and it does ensure that no push allow for pull request unless two people reviewed and approved it.	Accept all feedback and incorporated into GitHub repository settings.	None
AI-015	Benno Weideman	09/09/2026	Gemini	Asked for assistance in providing a solid-proof template for a Team Working Agreement template.	Provided a template for the Team Working Agreement.	Visually checked and contains all necessary information such as roles, team members, agreements, and signatures.	Accepted entire template	None
AI-016	Benno Weideman	14/09/2026	Claude	Asked for generating code on how to set up the PostgreSQL database connection to the initial front-end and back-end	Provided database connection code and fixes and missing implementations in the backend.	Despite the website that works, there must still be testing, verification, and changes done to ensure it is aligned with the project.	Make changes to the code and website structure given by AI as milestone 2 progress but do not accept all work without verification.	None
AI-017	Tammy Fourie	12/09/2026	Claude	Frontend development and interface implementation	Claude was used to assist with generating an initial implementation of the frontend. The generated code served as a starting point and was reviewed and evaluated before being incorporated into the project.	The frontend was manually reviewed and tested to confirm that the implemented functionality operates as expected and aligns with the relevant rubric requirements. The generated code was also inspected to identify areas requiring refinement or modification.	Partially accepted. The implementation provides a suitable foundation and meets the current functional requirements, but it has not been accepted as the final version. Further manual changes, refinements, and integration work will still be completed by the team.	Some areas still require refinement, including code cleanup, interface improvements, consistency with the wider application, and any adjustments identified during continued testing and integration.
AI-018	Benno Weideman	26/09/2026	Claude	Provide draft for data persistence baseline, particularly entity overview, justification of persistence model, and SPOF, availability, scalability implications	Gave a draft based on the project for data persistence.	Verified via AI-021	Accept the content and apply further fixes that lead to AI-021 usage and verification.	None
AI-019	Benno Weideman	27/09/2026	Claude	Provide a fixed draft for ASR.	Simplified ASR into 10 ASRs and populated columns with referenced project information.	Manually verified the ASRs, sources, measurables, and influences against the PED v1.0 to ensure the content is not fabricated but links to the work.	Accepted the 10 suggestions and paraphrased all the work and made necessary word changes to suit the CivicConnect platform.	None
AI-021	Benno Weideman	29/09/2026 / 30/09/2026	Claude	Apply the data persistence baseline to fit with new updated entities and make a link to assignment 2 for informed decision in the decision log.	Provided a fixed draft for data persistence baseline, with a new ERD image and decision log update.	Manually verified the draft against the requirements, ASRs, stakeholders, and other important source links to ensure content is not fabricated.	Accepted the new draft and thoroughly paraphrased the work and changed/removed content that did not align with the project.	None
AI-022	Tammy Fourie	28/09/2026 – 30/09/2026	Claude	Draft architecture (§7), wireframes and usability/accessibility rationale (§10), initial design decisions (§11), README; align them with the updated ASRs and add email verification at registration.	Drafted text, diagrams, clickable wireframes, D09–D13, CR-003 to CR-007, R10–R16 and RTM links; flagged inconsistencies as comments.	Reviewed all sections against PED v1.0, the team's ASRs, data model, API contract and A2 Task 1, and had them realigned to the team's IDs. Confirmed the authentication requirement with the lecturer (email + password with email confirmation) and agreed the three user attributes with the database owner. Checked the README against the actual GitHub repo and tested the	Accepted with changes. Directed the major revisions (authentication design, database attributes, risk numbering, repository structure and folder layout) and rejected the passwordless design after the lecturer's clarification. Paraphrased and integrated the content into the team PED; teammates' sections were not edited, and inconsistencies were raised with owners as	Passwordless sign-in was drafted first and dropped after the lecturer clarified the requirement (email verification at registration only).

						wireframes in a browser.	comments instead. CR-003 to CR-007 remain pending team approval.	
AI-023	Mohau Drew Modiselle	28/09/2026 – 29/09/2026	Gemini	Drafting and aligning Technology Stack , API/Integration Decisions , Deployment Compatibility, and Decision Log entries (D05–D07) with the updated ERD and M2 architecture.	Structured comparative technology matrices, REST endpoint contracts, deployment parity controls, and D05–D07 records linked to A2 peer-reviewed sources and the consolidated 4-entity schema (user, service_request, action, logging).	Cross-checked all requirements, ASRs, CRs (CR-001–CR-004), and entity/column names against PED v2.0 Sections 6–8 and verified all academic DOIs/citations in Google Scholar.	Accepted the matrices and endpoint specifications after updating table/attribute names (user_type, street_address) and aligning duplicate prevention with D08.	Initial draft referenced the pre-consolidated 5-table schema and idempotency headers; manually reconciled with D08 and CR-002.
AI-024	Benno Weideman	29/09/2026	Claude	Asked AI to provide a draft for the RTM containing existing data with additional column data such as ASR link, data persistence impact, architecture, technology decision and ADR, change, or risk reference.	Provided a word document containing drafted RTM.	Verified requirements against the previous RTM, ASRs, decisions from EDL, technology stack, data entities and risks.	Accepted draft and made necessary changes to the RTM where errors were found, such as the data persistence impact column.	None
AI-025	Benno Weideman	29/09/2026 / 30/09/2026	Claude	Provide fixed draft for Forward Engineering Considerations and provide at least 3 suggested new considerations	Receive fixed draft for Forward Engineering Considerations with 3 suggestions.	Verified considerations against original work, and PED, and make checks to which ones are redundant.	Accepted draft, removed 2 suggestions, and made edits to all considerations and paraphrased.	AI limit reached 29 Sep to 100%, had to wait till the next day to use it.
AI-026	Benno Weideman	30/09/2026	Claude / Gemini	Provide guidance on how to build tests for CivicConnect frontend and ask for fixing the test.	Provided clear guidance for using .github folder with workflows, and .yml file to have functioanl build dependency test. Fixes was applied to broken version.	Verified against the actual front-end and tested the Github CI pipeline to make it work.	Accepted the guidance and fixed code and incorporated into repository.	Claude gave clear guidance and fixed code but had limit on 30 Sep; Gemini did not give better guidance.
AI-027	Benno Weideman	02/10/2026	Claude	Provide resources that help find RPOs and RTOs for a similar project of the CivicConnect platform, in this case, a service request management system.	Provided resource links to pdf standards.	Verified resource by seeing if the company is real that provides the resource, if the resource can be found if searched in the company's search bar, and it was found.	Accepted the resource and used available information of RTO and RPO for default estimates for non-functional requirements.	None.
AI-028	Benno Weideman	03/10/2026	Claude	Provide draft template for submission requirements, milestone 3 overview, and milestone 2 baseline conformance.	Ready-to-use templates for the 3 mentioned sections provided.	Verified draft against what the real M3 needs.	Accepted the draft and start to incorporate work as development progresses.	None
AI-029	Benno Weideman	03/10/2026	Claude	Fix and update the current RTM based on updated requirements, change impact analysis, and decision log.	Fixed RTM has been provided with additional columns for M3.	Verified the RTM against the updated functional and non-functional requirements and the ASRs, change requests, risks, and decision log.	Accepted draft and corrected IDs and requirement descriptions.	None
AI-030	Benno Weideman	03/10/2026	Claude	Fix and update ASR to retain the same requirements but change to fit proper ASRs.	Provided a fixed draft for current ASRs.	Verified ASRs against previous ASRs and the real PED to ensure no subtle fabricated changes.	Accepted draft, paraphrased work and add correct fixes to requirement IDs mentioned in source column.	None
AI-031	Benno Weideman	02/10/2026, 03/10/2026, 04/10/2026	Claude	Provide an updated draft for the data and persistence baseline with the suggested ERD image.	Update data and persistence baseline draft with AI ERD image generated.	Verified ERD and data and persistence baseline against PED content.	Accepted draft, paraphrased work, and kept ERD.	None
AI-032	Benno Weideman	05/10/2026	Gemini	Generate a SQL script based on the milestone 3 ERD for data and persistence	Provided a generated SQL script for the entire database.	Verified against the data and persistence baseline, looked at what exists in the	Accepted the script, but changed the entity names from plural to singular, removed	AI suggested using UUID which are hash-type IDs but told AI to make that it

				baseline for tables in PostgreSQL.		ERD, and where constraints, triggers, and indexes was used for the real database.	creation date columns from specific entities, unnecessary triggers and indexes.	only caters for integer IDs, had to generate again.
AI-033	Benno Weideman	05/10/2026	Gemini	Asked AI to provide a repository structure on GitHub based on master project brief recommendation.	Folder structure template provided to follow.	Structure is aligned with the visual recommendation from the master brief.	Accepted the structure.	None
AI-034	Benno Weideman	05/10/2026	Claude	Provided draft for milestone 3	Template for milestone 3	Verified template against the real M3 instructions.	Accepted template but removed unnecessary content that is not required and paraphrased sentences that were kept.	None
AI-035	Benno Weideman	06/10/2026	Gemini	Asked AI to provide code to fix connect between repository.js which is for service request module to the supabase postgresql database. Also asked AI to provide a test.repo.js file to test the connection.	Provided fix code and a new script for testing.	Verified the code against the database structure, entities, and triggers. Also verified the code against the records in the database and if records does not already have IDs that the code want to use for new inserts.	Made changes to the code since entity names used was different, code violated the triggers for actions, because it tried to insert an action where in the database a trigger does that.	Asked AI several times to ask what is the problem because entity names was different, foreign keys and triggers was not in agreement with AI code, had to prompt many times before a fix was given.
AI-036	Benno Weideman	09/10/2026	Gemini	Asked AI to explain why action insert queries in the backend "staff" module does not work.	Provided a clear explanation on what to fix.	Verified by executing the test.js file for connecting to the PostgreSQL database and ensuring that entities, foreign keys, and relationships match with the backend code..	Accept code.	None
AI-037	Benno Weideman	09/10/2026	Grok	Asked AI to provide repository.js, service.js, and controller.js files for the admin module based on the code for the project and the database schema.	Provided the three script files.	Verified if the column names used in the sql code was correct against actual database,	Accept code, but made changes to it to fit the app and database logic.	AI did not provide working code at start, had to show the database to make it fit to the database needs.
AI-038	Benno Weideman	10/10/2026	ChatGPT, Claude, Gemini, and Grok	Asked AI to provide assistance on how to fix start and end date logic in the backend and frontend and how to modify the database to fit it by adding a target_date column in the category table.	Provided SQL scripts and backend and frontend code.	Verified changes against the database, made sure that the code does not contradict database, removed unnecessary code.	Accept code, made changes to fit the code correctly.	Struggled to find a suitable method quikcly on how to store and display start, end, and target date in the UI.
AI-039	Benno Weideman	11/10/2026	Gemini and Grok	Asked AI to provide a fixed integration between the applied fixes mentioned in AI-039 with the backend branch. In addition to that, using the work done in AI-038, the admin module was also integrated with the backend branch.	Provided connection between the modules.	Verified the code against the database, made sure that it does not cause conflict between other modules.	Accepted the code and made changes where unncessary complexity was introduced.	Currently, still searching for a way to make search bar for audit trail work, the filter and search bar was difficult to try and fix with AI.
AI-040	Benno Weideman	11/10/2026	Grok	Provide a draft for the quality strategy applied.	Provided a template draft for the quality strategy using the current risks.	Verified the strategy against the real risks and their controls and test case IDs.	Accepted draft but modified content to fit real PED.	None
AI-041	Benno Weideman	12/10/2206	Gemini	Ask AI to explain why there is a mismatch between category names and IDs for requests, since a request such as Lost property was assigned to category Faculty fault.	Provided an explanation of how to fix issue and explained that the database values for the categories were case sensitive to the ones used in the server.js file of the request module.	Queried the results of the category to verify that the values are the same as the values of categories used in the app itself.	Accepted changed code,	None

AI-042	Benno Weideman	12/10/2026	Claude	Provide a draft for the TC-001 unit test, provide a draft for the pipeline following milestone 3 requirements.	Provide a complete code draft for the unit test and the CI pipeline.	Verified that the unit test assertion was meaningful by looking at the original methods used, the pipeline needed to be checked to ensure it scans throughout both frontend and backend.	Accepted code, made changes to fit the project.	None
AI-043	Benno Weideman	13/10/2026	Gemini	Provide a correct code draft for the TC	Code provided for the TC-008, TC-009, TC-011, and TC-017	Verified that the code assertion were correct and are aligned with the real methods and logic code used by the backend.	Accepted but modified paths for methods to be used and modified columns used as queries.	None
AI-044	Benno Weideman	14/10/2026	Gemini	Asked for instructions on how to set up the staging environment using Supabase, Render, and Vercel.	Instructions are provided.	Followed the instructions and manually set up Render, Vercel, and Supabase, confirming what was told works.	Accept guidance	None
AI-045	Tammy Fourie	13/10/2026	ChatGPT / Codex	Reconcile registration with PED and briefs.	Inspected uploaded code/documents; identified long-token flow, token in API response, missing email sender and unverified login; proposed six-digit implementation.	Assistant compared source with PED §8/§11/§24 and M3 requirements. Tammy reviewed the proposed setup; final team baseline review not evidenced.	Used as an implementation starting point; keep final acceptance subject to peer review.	PED contained D13/D16 and requirement-ID conflicts; early wording treated draft D16 too definitively. Approval must be recorded separately.
AI-046	Tammy Fourie	13/10/2026	ChatGPT / Codex	Implement verification.	Generated user-service/repository/controller/routes, MailSender, migration, configuration templates, screens and integration instructions.	Assistant tested code with fake DB/mail dependencies. I reported successful database migration and later real signup/email/sign-in. Live constraints/triggers and concurrency not fully verified.	Applied and subsequently revised through cleanup and manual UX changes.	Implementation added ten-minute expiry, HMAC and schema fields beyond old wording. Those decisions require PED updates; no guarantee of whole-app compatibility.
AI-047	Tammy Fourie	13/10/2026	ChatGPT / Codex	Create and interpret regression tests.	API/component and repository unit tests; auth gateway/routing checks; build and lint execution; test-evidence explanations.	I supplied output showing 26 backend tests passed. Assistant later ran 28 backend/10 focused frontend cases and build. Full frontend earlier showed nine request-related failures.	Retain meaningful tests, but rerun on final candidate and update duplicate-response expectation.	Fake dependencies do not prove actual SQL/SMTP behaviour. Counts changed between snapshots; latest manual changes are outside those passing results.
AI-048	Tammy Fourie	13/10/2026	ChatGPT / Codex	Local installation and real integrations.	Diagnosed commented JSON lockfile; guided private .env setup, Gmail App Password, SMTP check/test email, Supabase connection and migration.	I reported successful installation/tests, SMTP authentication, received test email, SELECT 1 connection and migration success. Credentials were kept out of chat.	Used local guidance; retain placeholders in .env.example and exclude actual .env files.	Connection/SMTP acceptance alone does not prove account persistence/delivery. Earlier reassurance about compatibility needed qualification; shared migration required owner review and recovery planning.
AI-049	Tammy Fourie	13/10/2026	ChatGPT / Codex	Investigate missing signup email and remove auth demo behaviour.	Reviewed updated ZIP; routed live auth to HTTP, removed demo shortcuts/inbox, cleared demo sessions, narrowed duplicate conflicts and added safe logs.	Assistant ran 28 backend/10 focused frontend cases and build. I subsequently reported the flow working. Exact original failure cause was not isolated with runtime evidence.	Applied cleanup; keep unrelated mock fixtures/tests and teammate business logic.	Do not claim demo mode was conclusively the original root cause. Shared real-API default affects data source selection; private env/runtime could not be inspected.
AI-050	Mohau Drew Modiselle	13/10/2026	Gemini	Used to draft some of the test cases	Reviewed requirement and made test case draft	I reviewed outputted code made changes as required and submitted for peer review	Kept adapted test cases	Irrelevant test cases were given at times but were just rejected.
AI-051	Mohau Drew Modiselle	11/10/2026	Claude	Backend file structure	Ai gave a recommended folder structure to map out the backend before coding started.	Made sure the structure matched out chosen architecture.	Used the recommended structure but changed file names to match project.	none

AI-052	Mohau Drew Modiselle	11/10/2026	Gemini	Assisted in some of the code	Ai contributed in some of the backend development.	Made sure to review the code and understand it so that nothing irrelevant was accepted. Adding some of my own code as well.	Used correct and reviewed code after verification.	None
AI-053	Mohau Drew Modiselle	14/10/2026	Gemini	Debugging	Ai was used for debugging purposes	Geminin helped identify causes of defects in the system	Corrected the defects once causes were revealed	None

22. Milestone 3 Overview

22.1 Purpose

This section onwards covers the content of milestone 3. The goal of this section onwards is to serve that the evidence index exists for the release of the milestone 3 CivicConnect platform.

22.2 Release candidate

The baseline will be assessed using commit f313e28 on the main branch, built by CI run. Whenever a pull request has been successfully merged into the main, the corresponding backend and frontend deployment platforms update the platform automatically.

22.3 Implemented scope

	Capability	Target Boundary	Status
S-IN-001	Responsive web submission	A requester can submit a service request with validated essential information and a controlled category: facility fault, equipment damage, security concern, IT support, maintenance, lost property or other.	Implemented
S-IN-002	Requester status and history	An authenticated requester can view the current status and lifecycle history of an individual request and a list of their previously submitted requests.	Implemented
S-IN-003	Meaningful requester feedback	The platform provides in app feedback when a request is accepted, rejected, materially updated, resolved or closed. External email/SMS delivery is not committed, however, a requester will receive notification via email for 6-digit code to confirm user registration once.	Partially implemented; email verification is deferred to milestone 4; email verification is deferred to milestone 4.
S-IN-004	Staff worklist	Authorized staff can view full details of relevant requests and search, filter and sort them using agreed useful criteria.	Implemented
S-IN-005	Triage and ownership	Authorized staff can correct categorization where permitted, assign a request or accept responsibility and view current ownership and ownership history.	Partially implemented; category correction is not implemented.
S-IN-006	Controlled request lifecycle	Authorized staff can update status through valid transitions and record actions, comments and resolution information. Actor and timestamp information is retained for auditability.	Implemented
S-IN-007	Resolution and closure	Authorized staff can resolve or close requests and provide resolution information. The detailed state model must address rejection, cancellation and reopening before requirements sign-off.	Mostly implemented; the platform does not support reopening.
S-IN-008	Role-based access	Requester, staff, supervisor/management and administrator capabilities are separated using role-based access and least privilege.	Implemented

S-IN-009	Oversight views	Authorized oversight users can identify open, overdue, resolved and closed requests and filter information by category and status.	Implemented
S-IN-010	Due-date/overdue basis	The system stores or derives a target resolution date using an approved rule so that overdue work can be identified. Automatic escalation is excluded.	Implemented
S-DEF-006	Registration Email verification	The system should send email verification to newly registered users to confirm account creation using a 6-digit token to their email and once confirmed through the app, the user may proceed to enter the landing page.	Email verification was implemented but will not be included in milestone 3; deferred to milestone 4.

22.4 Current status

Requesters can fill in and submit requests, see statuses, cancel requests, and receive feedback from staff. Staff in specified jurisdiction can receive category-related requests and provide feedback on progress and change statuses. Management can view overall requests, statistics, the audit trail, and all requests to support reporting. All users can create accounts; however, due to integration issues, email verification was deferred to M4. The database failover operation, category correction, and diagnostics logging was not implemented. The readiness position would be a CONDITIONAL GO, as not all features were tested sufficiently, and some requirements were not implemented.

22.5 Where to find things

RTM: p.63 - 65 · Risk Register: p.67- 69 · Decision log: p.71- 73 · Technical Debt Register: p.101· Defect Register: p.99 · AI Usage Register: p.80 - 86.

23. Milestone 2 Baseline Conformance

23.1 Approach

The M2 decisions recorded in PED v2.0 §§7.3, 7.5, 11.1 and 11.2 were compared with the latest supplied implementation snapshot, identified by archive commit reference `69a3d6c12b38bfed226aa3c1ef691dc836971943`. The review considered the backend structure, notification adapter, request-status handling, audit records and authentication/authorization controls.

Local test results recorded were considered as additional evidence.

Items are classified as implemented, partially implemented, not evidenced or requiring a documented decision. A deviation is not treated as approved merely because it appears in the implementation.

23.2 Conformance table

M2 decision / component	M2 baseline	Implemented?	Implementation evidence	Status
D11 Strategy: notification channels	NotificationChannel with email and fake channels (§11.1).	Partly; full strategy not evidenced	The user service accepts an injected mailSender in src/backend/modules/user/service.js. Tests substitute a fake sender. However, the reviewed backend does not demonstrate the planned NotificationChannel abstraction or a general channel-selection strategy.	Partial alignment. A fake test dependency does not by itself establish the full Strategy implementation. See BC-M3-01.
D11 Adapter: mail provider	MailSender implemented by NodemailerMailSender (§11.1).	Yes, at code level	src/backend/infrastructure/mail/NodemailerMailSender.js encapsulates Nodemailer transport and sending. src/backend/package.json includes Nodemailer, and src/backend/server.js injects the adapter into the user service.	Adapter present. Current deployed email delivery is not established; signup verification has been deferred. See BC-M3-02.
D12 Observer: post-commit events	EventDispatcher, RequestStatusChanged and NotifyRequesterHandler (§11.2).	Not evidenced	These components were not found in the reviewed backend. The request-status service calls repository operations directly; no post-commit notification dispatcher was identified.	Open implementation gap. The team must implement the mechanism or record an agreed scope/design revision. See BC-M3-03.
D12 transactional audit	Status update and action row written in one transaction (§11.2).	Alternative implementation; conformance requires review	In src/backend/modules/request/repository.js, addTimelineAction() inserts an action containing the status. Request queries derive the current status from the latest action. This differs from updating a separate current-status value and inserting an audit row in one explicit transaction.	An action-based status representation is present. Its equivalence to the M2 intent, including concurrent updates and failure behaviour, has not been demonstrated. See BC-M3-04.
D10 state machine	Seven states and a defined transition table in ServiceRequest (§7.5).	Partly; not conforming to the complete baseline	The reviewed src/backend/modules/request/service.js contains transition rules for Pending, In Progress, Completed and Cancelled. It rejects the required Accepted transitions and reopening. The later local TC-	Baseline mismatch and unresolved test failures. Reconcile the

			013 run failed all eight checks and requires comparison with the reviewed version.	agreed lifecycle across service logic, persistence and UI. See BC-M3-05.
D09 Repository and layers	Controllers contain no SQL; responsibilities separated into modules/layers (§7.3).	Yes for the reviewed separation of responsibilities	The user, request, staff and admin modules under src/backend/modules/ contain separate controller, service and repository files. Reviewed controllers delegate work rather than querying the database directly; SQL is located in repositories.	The main separation-of-responsibilities intent is implemented. The feature-based folder layout should be reflected in the implementation description.
D09 middleware chain	Verify JWT, authorise and validate (§7.3, Figure 7-2).	Partly	src/backend/middleware/authMiddleware.js provides JWT verification and role checking. Reviewed staff routes use role checks, but admin and request routes do not consistently enforce the required role restrictions. Validation also occurs in services. The later local TC-014 run recorded three permission-check failures; TC-012 recorded two category-validation failures.	Authentication is present, but complete authorisation and validation conformance has not been demonstrated. Investigate version differences and resolve confirmed defects. See BC-M3-06.

23.3 Significant deviations

Ref	Original decision (M2)	Problem found in implementation	Revised decision	Consequence	Control record
BC-M3-01	D11: Notification channels use the Strategy design, including email and fake channels.	The reviewed implementation injects a mail sender into the user service and uses fake senders in tests. However, the complete planned notification-channel strategy was not identified.	No replacement design has been agreed. The current implementation provides a mail-sender abstraction for provider isolation and testing. The broader notification-channel strategy remains an incomplete part of the M2 design.	Tests can replace the mail sender without sending real email. This supports test isolation but does not demonstrate the full planned channel-selection behaviour.	D11; §23.2 notification-channel conformance entry; src/backend/modules/user/service.js. Status: partial implementation. No completed approval record for a reduced Strategy design was supplied.
BC-M3-02	Email confirmation forms part of account activation, supported by the D11 mail-provider adapter.	Local email delivery previously worked, but the team encountered an SMTP deployment blocker. The current signup flow skips verification, permits immediate sign-in and, in the reviewed repository, creates users with email_verified=true.	The team has deferred SMTP-based email verification to the next milestone. The existing Nodemailer implementation is retained in the reviewed code but is not used to confirm new	Current registration does not establish email ownership. Accounts created during the bypass must not be treated as having completed verification solely because	D11; REQ-030 and the related REQ-017 registration flow; TC-015, TC-016 and TC-016-R; §31 release limitations; Technical Debt Register verification-deferral entry. Status: team-confirmed deferral. Replacement-provider selection, account-transition handling and formal scope-acceptance evidence remain outstanding.

			accounts during signup. Brevo is under investigation as an alternative; no replacement provider has been selected or verified.	the flag is true. Original TC-015 and TC-016 remain incomplete; the successful journey without verification is recorded separately as TC-016-R. Retained verification endpoints and mail configuration also require reconciliation with the deferral.	
BC-M3-03	D12: Request-status changes produce post-commit events handled by an observer-based notification mechanism.	EventDispatcher, RequestStatusChanged and NotifyRequesterHandler were not identified in the reviewed backend. Request-status changes call repository operations directly.	The M2 decision remains unchanged. The observer-based notification mechanism is recorded as an implementation gap. Deferring signup verification does not, by itself, change the separate requirement for request-status notifications.	The current evidence does not establish that request-status changes trigger notifications after a successful commit or that notification failures are handled independently of the status change.	D12; PED §11.2; §23.2 observer conformance entry; src/backend/modules/request/service.js. Status: implementation not evidenced. No separate decision to remove or defer request-status notifications was supplied.
BC-M3-04	D12: A status change and its audit action are committed together in a transaction.	The reviewed repository inserts an action containing the status and derives the current request status from the latest action. It does not use the originally described combination of a separate status update and audit insert in one explicit transaction.	The observed implementation uses action history as the source of current status. This is recorded as a design difference, not yet as a verified equivalent of the complete M2 transactional requirement.	An inserted action associates the recorded status with its audit information. However, the correctness of transition validation, concurrent updates and failure handling has not been established by the available tests.	D12; PED §11.2; src/backend/modules/request/repository.js, including addTimelineAction() and latest-action queries. Status: implementation difference recorded; acceptance of equivalence and database/concurrency evidence remain outstanding.
BC-M3-05	D10: The seven-state lifecycle and agreed transition table govern request-status changes.	The reviewed service contains a narrower transition table. The later local TC-013 run failed all eight checks: allowed-transition checks returned the previous status, while forbidden-transition checks did not reject the operation. These results differ from the earlier snapshot run.	Retain the documented M2 lifecycle as the expected behaviour. No reduced lifecycle has been confirmed as an approved replacement. Record the test failures as unresolved findings until the	The evidence does not demonstrate the complete Accepted, completion, cancellation and reopening workflow. The successful request-creation journey only confirms that a newly created	D10, S-IN-007 and R11; TC-013; local milestone3-results.json, 15 October 2026; §31 release blockers. Status: unresolved baseline mismatch and failed checks. No corrective retest or approved replacement lifecycle has been supplied.

			local implementation and test setup have been reconciled.	request appeared as Pending; it does not resolve lifecycle failures.	
BC-M3-06	D09: API requests pass through authentication, authorisation and validation controls.	JWT checking is present, but the local tests did not demonstrate all required restrictions. TC-014 returned success for requester assignment, requester role modification and management status modification. TC-012 accepted whitespace and unknown categories.	Retain the M2 access-control and validation requirements without relaxation. Treat the observed failures as findings requiring investigation and correction where confirmed. Neither the verification deferral nor the successful API journey changes these requirements.	Unauthorised actions and incorrectly categorised requests may be possible. Mocked tests do not establish the complete live impact, but the failures prevent a claim that the required controls have been verified.	D09; REQ-002, REQ-023 and R01; TC-012 and TC-014; local milestone3-results.json, 15 October 2026; §31 release blockers. Status: unresolved validation and permission findings. Real database category-isolation checks and corrective retests remain outstanding.

23.4 Summary

The implementation shows the mail-provider adapter and controller/service/repository separation, but full M2 conformance remains unproven. Notification strategy and observer behaviour, transactional-audit equivalence, lifecycle rules and middleware enforcement remain incomplete or insufficiently verified. Acceptance of the team-reported decision to defer email verification must be recorded.

The revised local API journey confirms registration, sign-in, request submission and retrieval, but does not close these gaps. Findings and evidence must be reconciled with the exact M3 submission commit before judging conformance and release readiness.

24. Controlled Changes

24.1 Change summary

Ref	What changed	Reason (implementation evidence)	Requirements / tests / data affected	Record	Evidence
CC-001	Data model expanded from 4 to 9 entities	Normalization was required to simplify entites that contained unrelated attributes.	The "user", "service_request", "action" and "log" (logging) entites.	D15, CR-009	p. 31 - 35
CC-002	Registration confirmation link replaced by 6-digit code	6-digit token is only sent to their user registered.	REQ-017, REQ-030	CR-004 (Affect)	Deferred to milestone 3
CC-003	Start and end date is not provided by the requester but automatically created	Lowers risk of requester entering inaccurate date, such as past date or unrealistic date.	REQ-001	D16, CR-011	PR #61

24.2 Decision record: D15

Aspect	Detail
Decision	D15: ERD changes to include additional 5 entities to normalize ERD.
Original decision (M2)	None
Implementation evidence / problem	Database entities were not normalized, and entities contained too many unrelated attributes.
Revised decision	No decision recorded to revise, but the original 4 entities will be added together with the 5 extra entities.
Alternatives considered	Keep the same 4-entity relationship in the database.
Consequence	More entities to manage and more relationships making it needed to involve more joins to perform queries.
Status / approval	Approved by Benno Weideman
Evidence	Data & Persistence Baseline, p. 33

24.3 Decision record: D17

Aspect	Detail
Decision	D17: Defer email verification
Original decision (M2)	Email verification to authenticate users in scope for M3
Implementation evidence / problem	Chosen provider was not supported by our backend deployment environment (with paying), after deferring email verification backend deployment is back up and working.
Revised decision	Email verification provider will be changed and email verification will be deferred due to time and cost constraint.
Alternatives considered	Switching email verification providers
Consequence	Technical debt, documentation changes, expected feature deferred
Status / approval	Approved by Mohau Drew Modiselle
Evidence	PR #74

25. Construction/Configuration

25.1 Repository and change control

Governance rules are stipulated in GitHub Controls, p.78, and decision D01, p.71.

Control	Configured as	Evidence
Protected main	A pull request is required before it can be approved; a minimum of two approvals from two team members required before the pull request is merged into the main branch.	https://github.com/Stelselopsiener/CivicConnect-Application/settings/rules/22651548
Direct push blocked	Any attempts of team members to push commits to main are rejected.	https://github.com/Stelselopsiener/CivicConnect-Application/settings/rules/22651548
Merge blocked without 2 approvals	Merge pull requests are only allowed if at least two team members review and approve the requests with meaningful feedback.	#74, #65, #60
Merge blocked by failing CI	Pull request failed due to a test failed with check, corrected measure taken.	#69, #70
Progressive history	Commits and pull requests demonstrate progressive history with meaningful development.	https://github.com/Stelselopsiener/CivicConnect-Application/pulls

25.2 Configuration and secrets

Secrets are added during CI pipeline through environment variables, added both in the GitHub secrets and variables, actions and inside the .env files for both frontend and backend that are also named in the .gitignore to prevent commits to include secrets.

Environment variables	Approach
NODE_ENV, PORT, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DATABASE_URL, JWT_SECRET, SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY	These backend environment variables are listed with their secret values added in the .env file located in the backend folder of the project repository; they are also listed in the root ".gitignore" file to prevent commits to contain them being pushed to GitHub branches.
DATABASE_URL, JWT_SECRET	These variables are used during runtime of the CI pipeline; in the secrets and variables, in the actions section, all secrets required to run in the CI pipeline will be added there and placeholders are used in the CI pipeline .yml file to prevent secrets from being found.
VITE_USE_MOCK_API, VITE_API_BASE_URL, VITE_API_PROXY_TARGET, VITE_RELEASE	Like the backend variables, these frontend environment variables are listed in the .env file located in the frontend folder of the project repository and will not be committed to GitHub due to them being added in the .gitignore.

25.3 Staging environment

The CivicConnect platform will be hosted in a simulated staging environment using Render as the backend deployment platform and Vercel as the frontend deployment platform. The following link can be used to access the live application: https://civic-connect-staging-application.vercel.app/sign-in and to demonstrate that the correct release candidate is deployed, in Render, the latest commit must be deployed and for Vercel, automatic redeployment takes place based on last merge pull request to main; visible in the deployments tab. The database utilized for the staging environment is Supabase (PostgreSQL) with the project containing the database called "CivicConnect (Staging)". The test case "TC-017", which entails the staff journey of signing in, viewing category worklist, accepting/assigning request, and providing progressive feedback, will be demonstrated via the real staging environment and is recorded in 28.2 Test evidence records.

Initial development required PostgreSQL locally installed on the computer but at later stage, Supabase was acquired to ensure entire team has access to same database for staging, secrets are recorded in .env files for development but staging requires both .env files and the secrets added in the deployment platform settings. The local environment uses http://localhost:5173/ to test website, where the real staging environment requires a URL link as mentioned above. The local environment made use of the SMTP provider to test email verifications via an account; staging defers the email verification, since Render no longer supports SMTP ports (Render, 2025); a different provider must be selected for email verification.

In the event of a necessary rollback, the following can be followed: When entering Render, select the web service connected to the CivicConnect, and redeploy the backend to the last committed deliverable for milestone 3 release candidate. In the GitHub CI pipeline, automatic redeployment takes place and confirms from Vercel that it is successful, with the latest commit. For milestone 3, the latest commit to be preserved is.

26. CI & Automated Verification

26.1 Pipeline summary

Workflow file: *.github/workflows/ci.yml*. Triggers: push to main and pull requests to main. There are currently three jobs that run parallel which makes use of an Ubuntu runner.

Job	Steps	What it protects
frontend	Checkout code > Set up Node.js > Frontend Lint & Audit	Code issues and secret vulnerabilities in the frontend.
backend	Checkout code > Set up Node.js > Backend Lint & Audit	Code issues and secret vulnerabilities in the backend.
test	Checkout code > Set up Node.js > Run Test Suite	Only execute the tests that are written in the tests folder for frontend and backend.

26.2 Quality gate rules

Gate	Rule	What it blocks	Evidence
G1 Frontend Lint & Audit	Fronted must be built and run; static analysis and secrets must be scanned and must function.	Code that fails to merge and build due to conflict, warnings shown by static analysis lint and secrets scanning.	PR #74
G2 Backend Lint & Audit	Backend must be built and run; static analysis and secrets must be scanned and must function.	Code that fails to merge and build due to conflict, warnings shown by static analysis lint and secrets scanning.	PR #74
G3 Test Suite	Tests must successfully be built, acquire dummy database variables, and execute tests that should pass.	Error in the usage of dummy variables and broken tests.	PR #74
G5 Two Reviewer	At least two reviewers must review and approve the pull request.	Unreviewed pull requests to be merged into the main branch.	PR #74

26.3 CI run evidence

Run	Commit / PR	Result	What it demonstrates	Link
Release candidate run	f313e28	Pass	All gates passed on the exact submitted baseline	https://github.com/Stelselopsiener/CivicConnect-Application/pull/79
Deliberate failure	PR #58	Fail	Quality gate prevented merge due to deliberate cancellation of a job in the CI pipeline.	https://github.com/Stelselopsiener/CivicConnect-Application/pull/58
Defect found by CI	PR #69	Fail then Pass	CI caught failing test case in backend. Test cases passed in isolated environment but had to be refactored so that they passed and were able to be run by GitHub actions	https://github.com/Stelselopsiener/CivicConnect-Application/pull/69

26.4 Static analysis and security checks

Tool	Scope	Result at RC	Handling of findings
ESLint	Frontend and backend	1 Error, 0 Warnings	Resolve test correctly next milestone
npm audit	Frontend and backend	19 Moderate, 3 High	Apply "npm" audit next milestone

27. Quality Strategy Applied

Risk	Requirements	Why it gets priority	Verification applied	Tests
R01 Role-based access failure	ASR-002, NFR-005, REQ-023	Platform requires roles; unauthorized users gaining access to sensitive data can cause broken trust boundaries.	JWT middleware and repository category filters; negative tests guarded points.	TC-002, TC-007, TC-008, TC-014
R02 Password / token hashing fails	ASR-003, NFR-006, REQ-024, REQ-030	Secrets that are not hashed will cause instant security risks.	bcrypt and SHA-256 asserted; secrets never appear in responses or log	TC-005, TC-006
R06 Request corruption	ASR-007, NFR-004, REQ-001, REQ-022	Requests that are corrupted breaks requester trust.	Single transaction; forced mid-transaction failure	TC-001, TC-009, TC-011
R11 Status vocabulary inconsistency	D10, FR-003, FR-010, S-IN-007	Different combinations of status names will cause confusion and inconvenience for users of the system.	Domain transition table with CHECK constraint; only D10 values returned	TC-003, TC-013
R03 Processing > 10 s	NFR-001, REQ-019, ASR-005	Staff must be able to utilize the daily worklist.	Indexed queries; p95 under modest load	PT-01
R14 / R16 Email delivery	D13, D11, REQ-017, ASR-008	Registration cannot proceed if a 6-digit token is not confirmed by the new user.	Fake channel in CI; status change commits even if mail throws	TC-006
R04 Crash under load	NFR-011, REQ-029, ASR-006	210-user target is a necessity to ensure that the system does not crash.	Stateless JWT and pg.Pool; limited concurrent ramp only	PT-01
R05 Backup / failover	ASR-004, NFR-008/010	Standby database is not implemented	Manual procedure to restore database with script.	Not tested
R08 Responsive layout	NFR-007, ASR-010	User journey must be performed on any device.	Walkthrough with a phone, tablet, laptop, or desktop computer.	Not tested
R10 Layer boundary erosion	D09	If controllers contain SQL, it can have a negative effect on the layered modular monolith system.	ESLint static analysis and manual reviews.	Review static analysis
R12 / R13 Usability	NFR-003/009,	Final styling pass is the right place	Moderated walkthrough + contrast/keyboard checks	Not tested

28. Black-box/Integration/System Validation

28.1 Technique selection

Technique	Applied to	Why appropriate	Tests
Boundary value analysis	Enforce 6-digit token length and password between 8 and 64 characters.	Ranges must be restricted to prevent clusters.	TC-005, TC-015
State transition	Request lifecycle, seven states (D10)	Allowed action depends on current state	TC-003, TC-013, TC-020
Decision table	Category role enforcement with rejection of requester to staff pages. (REQ-023)	There is more than one condition that can be tested on.	TC-002, TC-014
Negative / unauthorized	Validation, authentication, permissions, dependency failure	Ensures that the system will refuse to accept invalid results/ unauthorized access.	TC-001, TC-002, TC-004, TC-007, TC-008, TC-009, TC-012, TC-014, TC-018, TC-019, TC-021, TC-022, TC-023

28.2 Test evidence records

Test ID	Basis	Owner	Test and why it matters	Technique/Level	Input > Expected result	Actual result	Status	Evidence	Interpretation
TC-001	ASR-007, REQ-001, R06	Benno Weidemann	This tests that no empty fields are submitted; all fields must contain details; this ensures no corrupted or empty half-filled data do not persist to the database.	Unit; negative	Title, description, street address left blank → Each detail will result a proper error recovery message	✓ rejects all empty fields - category, title, description, and street address must be filled in (3 ms)	Pass	Console output showing PASS frontend/tests/unit/empty_fields.test.js	This assertion passed for allowing zero fields to be empty for the service request submission; the error recovery message showed.
TC-002	ASR-002, REQ-023, R01	Mohau Drew Modiselle	Blocks a requester JWT from accessing staff/admin endpoints. Critical for role-based access: one broken boundary would let requesters see oversight or staff data.	Unit; decision table / negative	Requester JWT on /api/v1/staff/requests → 403 Forbidden	Returns 403 Forbidden.	Pass	Console output showing PASS backend/tests/unit/TC-002.test.js	The test confirms that the system correctly refuses unauthorized access, demonstrating that the role-based access boundary successfully protects staff and admin data from requesters.
TC-003	D10, R11	Mohau Drew Modiselle	Rejects forbidden status transitions (e.g. Pending → Completed). Ensures the single D10 lifecycle vocabulary is enforced so screens, API, audit and reports stay consistent.	Unit; state transition	Pending → Completed (skip) → throws InvalidTransitionError (mapped to 409)	Throws InvalidTransitionError (mapped to 409 Conflict).	Pass	Console output showing PASS backend/tests/unit/TC-003.test.js and the trapped [Status Update Error].	The test proves that the single D10 state machine strictly enforces allowed lifecycle moves, preventing arbitrary or forbidden status jumps.
TC-004	REQ-002, REQ-022	Benno Weidemann	Rejects a request that has no category selected. Category	Unit; negative	Title: "Malfunctional robot", Description: "Our friendly cafeteria robot started	ejects service requests without category selected (5 ms)	Pass	Console output showing PASS backend/tests/unselected_category.test.js frontend/tests/empty_fields.test.js	The test function successfully passes assertion if no category was selected,

			drives staff jurisdiction and management statistics; an uncategorised request breaks both.		attacking our students with coffee mugs!", Street Address: "10 IT Street", Category: "" → "You must select a category"				in other words, if category = "".
TC-005	ASR-003, REQ-017, R02	Mohau Drew Modisell e	Rejects passwords outside 8–64 characters and verifies that passwords and verification tokens are stored only as hashes. Prevents credential exposure and weak passwords.	Unit; BVA	Password "abc1234" → rejected; valid registration stores only hashes	Password "abc1234" is rejected; valid registrations successfully store passwords and tokens as hashes.	Pass	Console output showing PASS backend/tests/unit/TC-005.test.js	The result demonstrates that credential exposure is prevented by successfully enforcing minimum length boundaries and hashing secrets before persistence.
TC-006	REQ-017, REQ-030, R02, R14	Mohau Drew Modisell e	Registers a user, stores a hashed 6-digit token, and blocks sign-in until the email is confirmed. Protects against unowned email addresses and ensures the registration confirmation path works.	API; high-priority	Valid name/email/password → 201; token hashed; login returns 403 EMAIL_NOT_VERIFIED until code confirmed	Returns 201 Created; token is hashed; subsequent login attempt returns 403 EMAIL_NOT_VERIFIED.	Pass	Console output showing PASS backend/tests/integration/TC-006.test.js (5.632 s)	The test validates the end-to-end registration flow, proving that unowned email addresses are protected by halting account activation until the emailed token is confirmed.
TC-007	ASR-002, REQ-023, R01	Mohau Drew Modisell e	Returns only requests that match the staff member's assigned category. Verifies the core jurisdiction filter that keeps staff from seeing other departments' work.	API; negative / high-priority	IT-Support staff JWT → only IT-Support requests returned	Returns only requests matching the IT-Support category	Pass	Console output showing PASS tests/integration/TC-007.test.js	The test proves the jurisdiction filter functions correctly, confirming that the least-privilege model successfully isolates departmental data from unauthorized staff members.
TC-008	ASR-002, R01	Benno Weidem an	Refuses a requester JWT access to management statistics. Confirms least-privilege isolation between requester and management roles.	API; negative	Requester JWT on /api/v1/admin/stats → 403	Returns a status code of 200.	Fail	Console output showing FAIL backend/tests/integration/TC-008.test.js	The test shows that requesters have access to the admin dashboard, conveying unprotected API.
TC-009	ASR-007, R06	Benno Weidem an	Forces the action-row insert to fail and proves the entire transaction (request + action) is rolled back. Guarantees no partial/corrupted audit trail.	API; negative / high-priority	Force action-insert to fail → ROLLBACK; status unchanged; zero partial rows	Returns an array with a single element containing service request details	Fail	Console output showing FAIL backend/tests/integration/TC-009.test.js	The test shows that the service request that experience a

TC-011	REQ-001, NFR-004, R06	Benno Weidem an	Successful submission appears correctly in “My Requests” with status Pending. Confirms the happy-path integrity of the core requester capability.	Black-box; positive	Title “Computer froze”, description “Freezes after login “, street address “12 Civic Centre Road “, category “IT support “ → request visible with Pending status	Return an error message with status code 401 and the message stating "Protected Deployment"	Fail	Console output showing FAIL backend/tests/black-box/TC-011.test.js	The test shows that the deployment environment such as Vercel could not receive the request.
TC-012	REQ-002	Tammy Fourie	Rejects empty or unknown category values. Protects the category-driven routing and reporting logic from invalid data.	Black-box; negative	Empty or unknown category → clear error, no request created	Four checks passed and two failed. Missing, null and empty categories were rejected, and the valid-category control passed. Whitespace and unknown categories were accepted.	Failed in local component run	`milestone3-results.json`; `TC-012.test.js`;	Category validation did not meet the expected behaviour for whitespace and unknown values. Real database checks remain pending.
TC-013	D10, S-IN-007, R11	Tammy Fourie	Exercises the full allowed lifecycle (Pending → Accepted → In Progress → Completed, Cancel, Reope n) and rejects forbidden moves. Ensures the single status vocabulary works end-to-end.	Black-box; state transition	Pending → Accepted → In Progress → Completed; Pending → Cancelled; Completed → In Progress (reopen); forbidden moves rejected	All eight checks failed. Five allowed-transition checks returned the previous status. Three forbidden-transition checks resolved instead of rejecting the change..	Failed; cause under investigation	`milestone3-results.json`; `TC-013.test.js`;	Results differ from the earlier review run. Compare the local implementati on and test setup before classifying every failure as an application defect.
TC-014	REQ-023, R01	Tammy Fourie	Verifies the role × action matrix (requester cannot assign, staff cannot see other categories, management is read-only). Directly protects the least-privilege model.	Black-box; decision table / negative	Each forbidden combination → 403 or UI hides action	One check passed, three failed and one remains todo. Unauthenticated admin access was rejected. Requester assignment, requester role modification and management status modification returned 200 instead of 403.	Failed; category isolation pending	`milestone3-results.json`; `TC-014.test.js`;	The tested paths did not enforce the expected restrictions. Investigate the local implementati on and test setup, and complete real database category-isolation checks.
TC-015	REQ-017, NFR-012	Tammy Fourie	Enforces 6-digit code length and single-use rules. Prevents weak or replayed verification tokens from activating accounts.	Black-box; BVA	Code < 6 or > 6 digits rejected; used code rejected; correct code sets email_verified = true	Three current component checks passed: five-digit, seven-digit and nonnumeric codes were rejected. Successful database activation and code-reuse checks remain todo. Earlier verification results are retained separately as historical evidence.	Partial component pass; feature deferred	`milestone3-results.json`; `TC-015.test.js`;	The current results establish format validation only. They do not demonstrate account activation, single-use enforcement or deployed verification.
TC-016	REQ-017, REQ-001–004	Tammy Fourie	Full requester journey: register → confirm email → login → submit request → see it in “My Requests”. Proves the primary	System / E2E; high-priority	Register with valid details → receive 6-digit code → confirm → login → submit request → request appears in “My Requests” with status Pending	The original journey including email verification was not executed and remains todo.	Deferred / blocked by deferred verification	`Local JSON results report; TC-016.test.js`	The original E2E acceptance criteria have not been demonstrate d in the current version.

			citizen-facing path works across UI, API and database.						
TC-016-R	REQ-017, REQ-001–004	Tammy Fourie	Full requester journey: register → confirm email → login → submit request → see it in "My Requests". Proves the primary citizen-facing path works across UI, API and database.	System / E2E; high-priority	Register with valid details → receive 6-digit code → confirm → login → submit request → request appears in "My Requests" with status Pending	Two supporting component checks passed: registration sends no verification email, and duplicate registration is rejected. The revised system journey remains todo.	Partial; live API and browser execution pending	Local JSON results report; `TC-016.test.js`	These results cover supporting registration behaviour only. The revised API journey is the next activity; it does not replace the original verification journey without scope acceptance.
TC-017	REQ-006–012, REQ-016	Benno Weideman	Full staff journey: login → see own-category worklist → accept/assign → update status → resolution visible to requester. Proves the core operational workflow works end-to-end.	System / E2E; high-priority	Staff login → worklist shows only own-category requests → accept or assign → update status through allowed transitions → resolution comment visible to requester	Sign in as liammoose@gmail.com with password abcd1234, view worklist, select a service request with pending status, and accept request, status is updated, change status and add comment, it works.	Pass	Manual execution by Benno Weideman on the 16th of October 2026.	End-to-end test works perfectly for staff to sign in, view worklist, and accept requests and provide feedback functions as intended.
TC-018	REQ-017, REQ-030; D16 verification controls	Tammy Fourie	Rejects expired codes so an old email cannot activate an account.	API/component; expiry boundary; negative	Submit the issued code at the ten-minute expiry boundary → verification rejected.	The test advanced the controlled clock by ten minutes. Verification returned 400.	Passed on the earlier local implementation. Verification is deferred from the current release; this result does not demonstrate verification on Render.	`user.test.js`: "expired codes cannot activate an account".	Supports the expiry rule with a controlled clock and fake repository.
TC-019	REQ-017; ASR-003; R02	Tammy Fourie	Limits incorrect guesses to reduce code-guessing attacks.	API/component; state transition; negative	Submit five incorrect codes, then the correct code → all attempts rejected after the limit is reached.	All five incorrect attempts returned 400. The correct code was also rejected after the fifth failure.	Passed on the earlier local implementation. Verification is deferred from the current release; this result does not demonstrate verification on Render.	`user.test.js`: "five incorrect attempts disable the code, even when the next input is correct".	Supports enforcement of the failed-attempt limit in the tested flow.
TC-020	REQ-030; D16 resend controls	Tammy Fourie	Checks the resend cooldown and replacement of old codes without changing the password.	API/component; time-based state transition	Request an immediate resend → no additional email. Retry after 61 seconds → replacement sent; old code rejected and new code accepted.	The immediate resend sent no additional email. After 61 seconds, a replacement was sent. The old code failed, the new code succeeded and the password hash remained unchanged.	Passed on the earlier local implementation. Verification is deferred from the current release; this result does not demonstrate verification on Render.	`user.test.js`: "resend waits 60 seconds; replaces old code without replacing password".	Supports cooldown and replacement behaviour. This checks immediate and post-cooldown states, not every timing boundary.
TC-021	REQ-030; R14, R16	Tammy Fourie	Checks recovery when	API/component; injected failure	Make the mail sender fail → account remains	The simulated mail failure left the account pending. A later resend succeeded, and its	Passed on the earlier local implementation.	`user.test.js`: "mail failure preserves the pending account and resend can recover".	Supports the application's failure-handling

			registration email submission fails.		pending. After cooldown, resend and verify successfully.	code verified the account. The checked warning did not expose the injected secret error text.	Verification is deferred from the current release; this result does not demonstrate verification on Render.		path with a fake sender. It is not a real Gmail outage test.
TC-022	REQ-017; ASR-003; R02	Tammy Fourie	Ensures a code cannot verify a different email address.	API/component; negative	Issue a code for one address and submit it with another → reject verification.	Verification returned 400, and the original account remained unverified.	Passed on the earlier local implementation. Verification is deferred from the current release; this result does not demonstrate verification on Render.	`user.test.js`: "verification is scoped to email, so another address cannot redeem a code"	Supports binding verification to the intended email address.
TC-023	REQ-017, REQ-030; verification transaction integrity	Tammy Fourie	Checks rollback if verification fails between account activation and code consumption.	Repository unit test; injected failure	Simulate an error between the two writes → repository issues ROLLBACK instead of committing.	The repository test passed its rollback assertions after the simulated error.	Passed on the earlier local implementation. Verification is deferred from the current release; this result does not demonstrate verification on Render.	`verificationRepository.test.js`: "error between activation and consumption rolls back both".	Confirms the repository's transaction-control calls with a mocked PostgreSQL client. A real-database rollback test remains necessary to establish actual persistence behaviour.

29. Performance/Reliability

29.1 Test design

Aspect	Detail
Operation under test	GET /health (Readiness and Route Parsing)
Why performance matters here	Ensures the Express server event loop and routing layer do not crash or bottleneck when all users hit the platform simultaneously.
Target and source	210 concurrent users (REQ-029).
Tool and script	Artillery (load-test.yml).
Workload	210 Virtual Users executing a loop of 10 requests each (2,100 total requests) over a 10-second burst phase.
Environment	Local
Data assumptions	Seeded users and requests; authenticated token reuse

29.2 Results (PT-01)

Metric	Result	Target	Met?
Total requests / duration	2,100 / 11 seconds	2,100	Yes
Success rate and error types	100% (2,100 HTTP 200 OK)	[≥ 99%]	Yes
Average response time	0.2ms	–	–
p95 response time	1ms	≥ 50ms	Yes
Maximum response time	6ms	10 s limit	Yes
Throughput (requests per second)	214 req/sec	50 req/sec	Yes
CPU / memory / DB connections (optional)	_	–	–

29.3 Interpretation and limits

The result allows us to conclude	It does not prove
The Express API effortlessly meets the target of 210 concurrent users, processing 2,100 requests with a p95 latency of 1ms. The Node.js event loop and routing configuration are highly optimized and stable under burst load, with zero ECONNREFUSED errors or dropped connections.	Behaviour at real municipal scale or with production data volume. Real network conditions and mixed workloads

30. Defects & Quality Interpretation

30.1 Defect register

ID	Description	Found by	REQ · test · risk	Severity	Status	Fix and regression evidence
DEF-001	4 variables were declared but never used. The function was imported but never used. --ci was regarded as an unknown option added for the frontend test.	PR #65	TC-001	Low	Fixed	d0939aa
DEF-002	Request status not updating on user side after staff accepted/updated request	Visually found by Mohau	REQ-005	Medium	Fixed	76c6fee
DEF-003	When the server restarted, the requester was still signed in with his account.	Visually found by Benno	REQ-017	Medium	Fixed	76c6fee
DEF-004	Closed status change not working	Visually found by Mohau	REQ-010	High	Fixed	76c6fee

30.2 Static and security findings

Finding	Tool	Severity	Handling	Link
4 variables were declared but never used.	ESLint	Low	Fixed	d0939aa
The function was imported but never used	ESLint	Low	Fixed	d0939aa

30.3 Quality metrics interpretation

Metric	Value	What it shows	What it does not show
Tests passed / total	26/40	Tested features work on the release candidate	Features that have no tests.
Lint warnings and errors	0	Dependency and maintainability issues	Features do not work
High / moderate dependency alerts	22	Known vulnerabilities at scan time	Unknown vulnerabilities

31. Release Readiness

31.1 Release Evidence Summary

Evidence area	Result	Interpretation	Concern / action
Build and CI	Pass	The checks selected in the successful workflow completed. This supports those checks only, it does not establish that every requirement or test passed.	Link the successful run to submitted commit. A mandatory build or quality-gate failure block progression until resolved.
Automated regression	Partial	Local milestone run contained 10 passed checks, 13 failed and 5 todo entries. The team now reports fixes but corrective results have not been linked here.	To retain the original failures and link the corresponding passing retests. Confirm that critical regression cases run against the final candidate.
API / integration	Pass	All Api/integration tests work and application communicates flawlessly.	Email verification had to be deferred and will be subject to third party availability and compatibility with deployment and staging
System / E2E	Partial	The revised local API journey passed. Complete browser and staging journeys have not been evidenced in this record, the original verification journey remains deferred.	Link any completed team system tests and demonstrate the required critical journeys on the identified candidate.
Static / security	Partial	No code smells or maintainability issues were discovered; dependency issues are present. Some warnings were provided; indicating vulnerabilities.	Link static/security outputs, the disposition of significant findings and permission retests.
Performance	Pass	Performance requirement met on local environment and on CI, this does not mean it would perform to the same standard on staging or deployment environment (we couldn't test those environments due to free tier)	Retest will be needed on staging environment (in future)
Known defects	Closed	Known defects were found and resolved. Unknown defects will be evaluated once found. All major defects resolved	Not all defects have been identified, email verification deference could cause spam account which could bloat the database and cause performance issues if cyber attack has to happen.
Acceptance evidence	Partial	Executed tests support selected acceptance criteria. Email confirmation remains deferred, and complete acceptance coverage must be checked against the final RTM.	Link each critical acceptance criterion to its current result. Record acceptance of deferred scope and outstanding limitations.

31.2 Classification

Aspect	Detail
Release candidate	f313e2818d4ecb571f1b74244c3db7b38cd8c901
Classification	READY / CONDITIONALLY READY / NOT READY
Reasoning	After the final commit for this release candidate for milestone 3, a regression occurred which resulted in 10 failed tests, and in addition to that, those tests were not present during the previous commit which only contained 1 test that passed; this leaves a conclusion that all the tests were not progressively tested and contributed correctly for past merges. A single API journey has passed, but TC-017 was not automated and was recorded manually by one team member (Benno) which includes the staff journey, from signing in, viewing worklist, and accepting or assigning requests and providing feedback. In addition to that, the staging URL has been provided, and any real user may use it; however, a real automated test was not done to prove the staging environment functions as intended and will only demonstrate it with one team member. Despite the ESLint static analysis not providing any warnings regarding code smells, maintainability issues, or duplicated logic, its corresponding associative check, the secret/dependency scanner, has found 22 vulnerabilities, 19 Moderate, 3 High. This issue stems from database entities that are not recognized in the CI pipeline scanner, and dependencies that the platforms rely on are either outdated or unreliable to use. Also, even if the "npm audit" was provided as a quick fix, the platform itself still requires sufficient investigation in the CI pipeline to ensure it scans the right defects and not over-scan issues that will not affect the platform. Despite the executed tests covering their acceptance criteria, there is a obscene number of tests that has not been properly verified, which are 14/30 requirements, which is 47% of all requirements and 8 of the verified requirements are verified using manual approach, which were demonstrated by TC-017, the staff journey. In addition to that, given

	the email verification deferment, this will also nullify any tests that might have tried to acquire the functionality of the removed email module; this needs to be properly replaced since STMP is not supported by Render free tier. Due to surprise failed tests for last commit, non-automated staging environment, unhandled vulnerabilities, and many unverified requirements, the CivicConnect product will be a NO READY decision for milestone 3

31.3 Blockers and accepted limitations

Ref	Item	Type	Business impact	Decision / owner
RR-01	Earlier category, lifecycle and permission failures lack linked corrective retests.	**Evidence blocker.**	The reported fixes cannot yet support closure or a reliable readiness judgement.	Relevant module owners provide fix references; Team records the corresponding test evidence.
RR-02	Email verification deferred.	**Scope limitation; acceptance must be recorded.**	Email ownership is not confirmed, increasing exposure to invalid or abusive registrations.	Team confirms scope acceptance; Team coordinates verification follow-up. Brevo remains under consideration.
RR-03	Standby database and automated failover not implemented.	**Known limitation; acceptance depends on agreed scope and recovery requirements.**	Database failure may interrupt service beyond the required recovery time.	Team retain TD-001 and document recovery arrangements and future failover work.
RR-04	Performance not verified in staging/deployment.	**Evidence limitation.**	Actual deployed response times and error rates may differ from local and CI results.	Team performance/deployment owners arrange representative testing and record the remaining uncertainty.
RR-05	Final candidate and complete system evidence not identified in this record.	**Submission/readiness evidence gap.**	Results may refer to different versions, weakening traceability and release confidence.	Team leader coordinates the final baseline; contributors link their evidence to that version.

31.4 Residual risk statement

Residual risk	Why it remains	Current mitigation	Next action / owner
Performance under representative deployed load remains uncertain.	Local and CI environments do not reproduce all deployment conditions.	Team-reported performance exercise provides evidence for the tested workload only.	Team performance/deployment owners repeat testing in a representative environment and compare results with the agreed target.
Standby database failover remains unverified -R05, REQ-028, ASR-004.	Standby infrastructure and automated failover have not been implemented.	The recovery approach is documented in §8.5.4; a successful rehearsal is not established by that documentation alone.	**Benno:** verify backup availability, rehearse restoration and record recovery time and data-loss implications.
Accounts may be created without confirmed email ownership -R14/R16 where applicable.	Verification is deferred from the current registration flow.	The limitation is recorded; no equivalent ownership-confirmation control is demonstrated.	**Tammy with the authentication owner:** evaluate the provider, restore verification and define treatment of accounts created during the bypass.
Restored email delivery may fail or behave differently in deployment.	Replacement-provider selection, configuration, quotas and deployment compatibility remain unverified.	A mail adapter exists for provider isolation, and earlier local delivery provided limited feasibility evidence.	**Tammy with the deployment owner:** test delivery and failure handling through the selected provider before

			claiming the feature is restored.

32. Technical Debt Register

ID	Description	Origin	Why deferred	Consequence if ignored	Priority	Action / owner
TD-001	Standby database and automated failover not implemented	REQ-028, NFR-010, ASR-004, R05; D07	Time constraint to test and implement the failover; uncertainty if payment is required.	Outage beyond RTO if the database fails	Medium	Benno
TD-002	Real DB concurrency and recovery unverified	Mocks cannot prove locks, constraints, rollback or migration reversal. No backup/recovery rehearsal established.	Methods for the concurrency were never implemented and verified yet.	Transactions, row locks and additive migration in code.	High; database owner with auth owner.	Test simultaneous confirmation/resend and forced failure; rehearse restoration on a test copy; record results.
TD-003	Mocke assets and mixed run-time configuration	Mocks support tests but are statically imported; disabling demo mode does not prove their exclusion from the production bundle.	_	Live auth uses HTTP; request mode set false; demo shortcuts removed.	Low; frontend owner.	Keep fixtures until imports/tests are refactored. If needed, separate test/demo entry points and inspect production output.
TD-004	Deployment security/configuration mismatch	PED claims restricted CORS and explicit DB SSL/timeouts that are not configured in uploaded source. Mail environment guard couples real SMTP to production mode.	Due to the time constraint and the late discovery of bugs due to late development start date.	Secrets externalised; SMTP TLS options present; local testing worked.	High before external exposure; deployment owner.	Review CORS, proxy trust, DB TLS/timeouts and mail-mode separation. Record verified environment behaviour and update PED.
TD-005	Broader notification scope not evidenced	Signup adapter does not demonstrate status email, channel Strategy, Observer, opt-out or two-channel acceptance.	Development and integration time for email integration took longer than expected; focus now is only on email verification for milestone 4.	Shared mail adapter available for reuse.	Scope-dependent; notification/request owner.	Check current team implementation. Complete in-scope requirements or obtain explicit change; do not label mandatory gaps optional.
TD-006	Documentation and change control drift	D13/D16, IDs, version numbers, pipeline and acceptance criteria disagree. Review and defense can rely on wrong claims.	Due to the time constraint and the late discovery of bugs due to late development start date.	This update pack and existing PED/registers.	High for M3; document/requirement owners.	Merge reviewed corrections, attach evidence and approve one consistent baseline. Finish existing incomplete AI entries.

TD-007	Email verification middleware and SMTP delivery are disabled in the production registration flow	Infrastructure limitations on Render's free tier	Integrating, configuring, and domain-authenticating a third-party transactional email API (such as SendGrid, Resend, or Mailgun) falls outside the immediate scope and time constraints for the Milestone 3 deployment deadline	Unverified users can access the system, increasing the risk of spam accounts and complicating password reset flows in the future.	High	Evaluate and integrate a third-party email API that bypasses standard SMTP port blocks for M4, and re-enable the verification middleware.
TD-008	TC-006 and TC-007 excluded from automated CI pipeline execution (although passed locally due to direct database access)	Integration Testing / CI Pipeline	The 9-entity relational mapping relies on strict Role-Based Access Control (RBAC) mock states. Hardcoding this deep relational tree (users, staff_category mappings) into an ephemeral container proved too brittle, causing false-positive 500 errors during CI.	Automated CI runs will not verify staff jurisdiction filtering or verification token generation, relying strictly on local test execution.	Medium	Implement a robust ORM-based database seeding utility to restore these to CI / Mohau Drew Modiselle

33. Baseline Sign-Off

Project	CivicConnect
Baseline Type	M3
Version	3.0
Base	3.0
Scope reviewed	YES / NO
Requirements/traceability checked	YES / NO
Risk review completed	YES / NO
Repository/governance controls checked	YES / NO
Outcome	ACCEPTED / CONDITIONALLY ACCEPTED / REVISION REQUIRED

References

Airport Company South Africa, n.d. *Customer Relationship Management Solution*. [Document] Available at: https://www.airports.co.za/Documents/Annexure%20A%20Customer%20Relationship%20Management_Scope%20of%20Work_14%20October%202022.pdf [Accessed 7 September 2026].

Bailis, P., Venkataraman, S., Franklin, M.J., Hellerstein, J.M. and Stoica, I. (2012) 'Quantifying eventual consistency with PBS', *Proceedings of the VLDB Endowment*, 5(11), pp. 1428–1439.

Bass, L., Clements, P. and Kazman, R. (2021) Software Architecture in Practice. 4th edn. Boston: Addison-Wesley.

Belgium Campus. (2026a). SEN381 Software Engineering 381: Integrated Team Project - CivicConnect Master Project Brief, Academic Year 2026. Internal course document.

Belgium Campus. (2026b). SEN381 Software Engineering 381: Milestone 1 - Engineering Foundation & Requirements Baseline, Academic Year 2026. Internal course document.

Bijlsma, L.A., Kok, A.J.F., Passier, H.J.M., Pootjes, H.J. and Stuurman, S. (2022) 'Evaluation of design pattern alternatives in Java', *Software: Practice and Experience*, 52(5), pp. 1305–1315. doi: 10.1002/spe.3061.

Çetin, & Eldenk, D., 2025. Incidents During Microservice Decomposition: A Case Study. In Babar, , Tosun, , Wagner, & Stray, , eds. *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. Istanbul, 2025. Association for Computing Machinery.

Cheat Sheets Series Team, 2026. Authentication Cheat Sheet. [Online] Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html [Accessed 10 October 2026].

City and County of SAN FRANCISCO, 2023. *Citywide Technology Resilience Standard*. [Online] Available at: https://www.sf.gov/reports--february-2023--citywide-technology-resilience-standard [Accessed 2 October 2026].

Datadog, 2025. *Failure is inevitable: Learning from a large outage, and building for reliability in depth at Datadog*. [Online] Available at: https://www.datadoghq.com/blog/engineering/rethinking-reliability/ [Accessed 8 September 2026].

De Iasio, A. and Zimeo, E. (2021) 'A framework for microservices synchronization', *Software: Practice and Experience*, 51(8), pp. 1726–1742.

Fowler, M. (2015) MonolithFirst. Available at: https://martinfowler.com/bliki/MonolithFirst.html

Gamage, D., Tiwari, D., Monperrus, M. and Baudry, B. (2026) 'The design space of lockfiles across package managers', *Empirical Software Engineering*, 31(63), pp. 4–5.

GeeksforGeeks, 2026. *Creating a New Branch in Git*. [Online] Available at: https://www.geeksforgeeks.org/git/how-to-create-a-new-branch-in-git/ [Accessed 9 September 2026].

GitHub Docs, 2026. *Approving a pull request with required reviews*. [Online] Available at: https://docs.github.com/en/pull-requests/how-tos/review-pull-requests/approving-a-pull-request-with-required-reviews [Accessed 9 September 2026].

Hellerstein, J.M., Stonebraker, M. and Hamilton, J. (2007) 'Architecture of a Database System', *Foundations and Trends in Databases*, 1(2), pp. 141–259.

Heroku, n.d. *Removal of Heroku Free Product Plans FAQ*. [Online] Available at: https://help.heroku.com/RSBRUH58/removal-of-heroku-free-product-plans-faq [Accessed 8 September 2026].

Hofisi, C., & Chigova, L. (2023). Integrating digitization into basic service delivery frameworks to eliminate inefficiencies in local government.

Horn, et al., 2023. Native vs Web Apps: Comparing the Energy Consumption and Performance of Android Apps and their Web Counterparts. In *International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. Melbourne, 2023. IEEE.

Marquis, A., 2024. From Theory to Practice: Implementing Effective Role-Based Access Control Strategies to Mitigate Insider Risks in Diverse Organizational Contexts. *Journal of Engineering Research and Reports*, 26(5), p.138.

Mashau, P. (2023). South African municipalities balancing citizen aspirations with structural and financial constraints.

Mashau, N.L., & Kroeze, J.H. (2023). Municipal adoption challenges in South African digital transformation.

Naghdipour, A., Hasheminejad, S.M.H. and Barmaki, R.L. (2023) 'Software design pattern selection approaches: A systematic literature review', *Software: Practice and Experience*, 53(4), pp. 1091–1122. doi: 10.1002/spe.3176.

Nazário, F.C., Bonifácio, R. and Pinto, G. (2025) 'Mitigating Configuration Differences Between Development and Production Environments: A Catalog of Strategies', *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE 2025)*. New York: ACM.

Netshirando, M., & Ngoepe, M. (2024). Local authority trust and platform security determining e-government service consumption after digital transformation.

Nielsen, J. (2024) 10 Usability Heuristics for User Interface Design. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/

NIST (2025) SP 800-63B-4 Digital Identity Guidelines: Authentication and Authenticator Management. Available at: https://pages.nist.gov/800-63-4/sp800-63b.html

Nodemailer (n.d.) Nodemailer documentation. Available at: https://nodemailer.com/

Nookala, G. (2023) 'Microservices and Data Architecture: Aligning Scalability with Data Flow', *International Journal of Digital Innovation*.

OWASP (n.d.) Authentication Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP (n.d.) Forgot Password Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html

Pacific Certifications, 2026. *ISO/IEC 25010:2023 Guide to Software Product Quality Standards*. [Online] Available at: https://blog.pacificcert.com/iso-25010-software-product-quality-model/ [Accessed 6 September 2026].

Render, 2025. *Free web services will no longer allow outbound traffic to SMTP ports*. [Online] Available at: https://render.com/changelog/free-web-services-will-no-longer-allow-outbound-traffic-to-smtp-ports [Accessed 15 October 2026].

Richards, M. and Ford, N. (2020) Fundamentals of Software Architecture. Sebastopol: O'Reilly Media.

Saidani, I., Ouni, A. and Mkaouer, M.W. (2022) 'Improving the prediction of continuous integration build failures using deep learning', *Automated Software Engineering*, 29(21), p. 6.

Santos, J., da Costa, D.A. and Kulesza, U. (2022) 'Investigating the Impact of Continuous Integration Practices on the Productivity and Quality of Open-Source Projects', *ESEM '22: Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. Helsinki: ACM.

W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. Available at: https://www.w3.org/TR/WCAG22/

Weideman, B., Modiselle, M.D. and Fourie, T. (2026) SEN381 Assignment 2: Design, Persistence, Integration and CI Research. Belgium Campus ITversity.

Peer Review

Values:	5 = Excellent	4 = Above Average	3 = Average	2 = Below Average	1 = Poor

Attribute	Tammy Fourie		Benno Weideman		Mohau Drew Modiselle	
Participated in Group Discussions	5	5	5	5	5	5
Helped keep the group on task	4	5	4	5	4	5
Contribution to useful ideas	5	5	5	5	5	5
Quality of completed work	5	5	5	5	5	5
Total	19	20	19	20	19	20